

*Communauté Économique et Monétaire de
l'Afrique Centrale
(CEMAC)*



*Institut Sous-régional de Statistique
et d'Économie Appliquée
Organisation Internationale*

BP : 294 Yaoundé - Tél : 222 220 134

Cameroun AI Enthousiaste



*Direction du Développement et de la
Recherche*

ONG camerounaise

BP : 83000 Toulon, France

Site web : <https://kmer-ai.org/>

*Mémoire professionnel de fin d'étude présenté en vue de l'obtention du diplôme d'Ingénieur
Statisticien Économiste(ISE)*

OPTION : Data Science et Marketing

CONCEPTION D'UN PIPELINE HYBRIDE DE RECHERCHE SÉMANTIQUE POUR LES RECOMMANDATIONS D'EMPLOI-COMPÉTENCES, FONDÉ SUR LES LLMs ET LES GRAPHES DE CONNAISSANCES

Stage effectué du 10 Mars au 30 juin 2026

Rédigé par :

NGOULOU NGOUBILI Irch Defluviaire

Élève Ingénieur Statisticien Économiste cycle long – 5^e année

Soutenu publiquement le 07 Juillet 2026 devant le jury composé de :

Examineur

Pr MELATAGIA Paulin

Enseignant associé à l'ISSEA

Président

Dr WANSOUWE WANBITCHING

Enseignant permanent à l'ISSEA

Encadrante

Dr FOUTSE Yuehgoh

Présidente de KmerAi

Session de Juillet 2026

DÉDICACE

À mes parents,

LIKIBI NGALI Bérangère

et

KIVOUNDAKA Pierre

*Pour votre amour, vos sacrifices et les valeurs que vous
m'avez transmises.*

*Votre courage, votre confiance et votre soutien ont
accompagné chacun de mes pas.*

*Ce travail porte l'empreinte de vos efforts et de vos
espérances.*

**Je vous le dédie avec une profonde
reconnaissance.**

REMERCIEMENTS

Ce mémoire est l'aboutissement d'un travail exigeant, rendu possible par le soutien, les conseils et la disponibilité de nombreuses personnes. Je rends avant tout grâce à Dieu pour la force, la persévérance et les circonstances favorables qui m'ont permis de mener ce travail à son terme. J'adresse ensuite mes sincères remerciements à :

- **Dr. FOUTSE Yuehgoh**, Présidente et Directrice exécutive adjointe de KmerAI, mon encadrante professionnelle, pour son accompagnement, sa confiance et ses orientations tout au long de ce stage ;
- **M. MBOU SOB Ulrich**, Directeur de la Recherche et Développement de KmerAI, pour sa disponibilité, ses remarques pertinentes et son appui dans la conduite du projet ;
- **Dr. Marcel OPOUMBA**, Directeur Général de l'ISSEA, pour la rigueur académique promue au sein de l'institution et pour la qualité de la formation reçue ;
- **Dr. Dieudonné KINKIÉLÉLÉ**, Coordonnateur de la filière ISE, pour sa rigueur, son engagement envers notre promotion et l'exemple professionnel qu'il représente ;
- **Mme. MBIA NDI Marie Thérèse**, mon encadrante académique, pour ses précieux conseils, ses observations constructives et le temps consacré à la relecture de ce mémoire ;
- à l'ensemble du corps enseignant de l'ISSEA, pour les connaissances transmises, les exigences méthodologiques et la qualité de l'encadrement reçu durant ma formation ;
- mon oncle, **LIKIBI LOUFOUMA Richard**, pour sa présence, son soutien constant et ses encouragements ;
- mes amis **DONGMO Wandji Chelsie Carel**, **KOULOU Crépin** et **ONDAY ELISA Ruth Vanessa**, pour leur disponibilité, leurs encouragements et leur accompagnement dans l'élaboration de ce travail ;
- la communauté des étudiants congolais de l'ISSEA (MECI), pour l'organisation des pré-soutenances, les séances de soutien et l'esprit de solidarité entretenu ;
- toutes les personnes qui ont contribué, de près ou de loin, à la réalisation de ce projet et dont les noms ne figurent pas dans ce document. Qu'elles trouvent ici l'expression de ma profonde gratitude.

SOMMAIRE

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	v
Liste des tableaux	vii
Liste des figures	viii
Avant-propos	ix
Résumé	x
Abstract	xi
Introduction générale	1
I CADRE THÉORIQUE ET EXPÉRIMENTAL	5
1 FONDEMENTS THÉORIQUES ET REVUE DE LA LITTÉRATURE SUR L'IA GÉNÉRATIVE ET LES SYSTÈMES DE RECOMMANDATION	6
1.1 Repères conceptuels de l'étude	6
1.2 Fondements économiques et problématique d'appariement	8
1.3 Systèmes de recommandation et person-job fit	9
1.4 LLMs, embeddings et graphes de connaissances	12
1.5 RAG, GraphRAG et orchestration agentique	15
1.6 Positionnement de l'étude	19
2 SOURCES DE DONNÉES ET APPROCHE MÉTHODOLOGIQUE	21
2.1 Cadre méthodologique et sources de données	21
2.2 Représentation sémantique et graphe de connaissances	24

2.3	Orchestration agentique et génération contrôlée	27
II	PRÉSENTATION DES RÉSULTATS ET DISCUSSION	29
3	CONSTRUCTION DU SYSTÈME HYBRIDE EMPLOI-COMPÉTENCES	30
3.1	Données exploitables et contraintes de qualité du système	30
3.2	Adaptation du modèle d’embeddings au domaine emploi-compétences	33
3.3	Mémoire vectorielle pgvector pour le retrieval	37
3.4	Neo4j : mémoire relationnelle du matching	41
3.5	Score hybride et reranking des recommandations	45
3.6	Orchestration agentique du système par LangGraph	46
4	ÉVALUATION DE LA PERFORMANCE INTERNE DU SYSTÈME	50
4.1	Protocole expérimental et métriques retenues	50
4.2	Retrieval sémantique et apport mesuré du graphe	51
4.3	Validation fonctionnelle du GraphRAG	53
4.4	Reranking hybride et calibration du score	56
4.5	Contrôle de génération par critic	60
5	SYNTHÈSE ET DISCUSSION DES RÉSULTATS	64
5.1	Résultats majeurs de l’étude	64
5.2	Confrontation avec la littérature	65
5.3	Discussion des hypothèses de recherche	65
5.4	Portée, limites et conditions d’usage	66
	Conclusion générale et perspectives	68
	Références bibliographiques	II
A	ANNEXES	VII

LISTE DES SIGLES ET ABRÉVIATIONS

ANN :	Approximate Nearest Neighbors
API :	Application Programming Interface
BERT :	Bidirectional Encoder Representations from Transformers
BI :	Business Intelligence
CEMAC :	Communauté Économique et Monétaire de l'Afrique Centrale
CV :	Curriculum Vitae
DCG :	Discounted Cumulative Gain
ESCO :	European Skills, Competences, Qualifications and Occupations
FAISS :	Facebook AI Similarity Search
FNE :	Fonds National de l'Emploi
FRED :	Federal Reserve Economic Data
GAT :	Graph Attention Network
GCN :	Graph Convolutional Network
GNN :	Graph Neural Network
GPT :	Generative Pre-trained Transformer
GPU :	Graphics Processing Unit
HNSW :	Hierarchical Navigable Small World
IA :	Intelligence Artificielle
IDCG :	Ideal Discounted Cumulative Gain
ISE :	Ingénieur Statisticien Économiste
ISSEA :	Institut Sous-régional de Statistique et d'Économie Appliquée
IVF :	Inverted File Index
KG :	Knowledge Graph
LLM :	Large Language Model
LoRA :	Low-Rank Adaptation
LSTM :	Long Short-Term Memory

MAP :	Mean Average Precision
MEPC :	Nomenclature Camerounaise des Métiers, Emplois et Professions
MRR :	Mean Reciprocal Rank
NCF :	Nomenclature Camerounaise des Formations
NCM :	Nomenclature Camerounaise des Métiers
NDCG :	Normalized Discounted Cumulative Gain
NLP :	Natural Language Processing
OIT :	Organisation Internationale du Travail
ONG :	Organisation Non Gouvernementale
QLoRA :	Quantized Low-Rank Adaptation
RAG :	Retrieval-Augmented Generation
RNN :	Recurrent Neural Network
RWKV :	Receptance Weighted Key Value
SEO :	Search Engine Optimization
SGPT :	Sentence Generative Pre-trained Transformer
SQL :	Structured Query Language
SR :	Système de Recommandation

LISTE DES TABLEAUX

Tableau 1.1	Comparaison des principales approches de recommandation	11
Tableau 2.1	Sources de données mobilisées et utilisation méthodologique	23
Tableau 3.1	Protocole de construction des paires de fine-tuning	33
Tableau 3.2	Hyperparamètres du fine-tuning SentenceTransformer	34
Tableau 3.3	Distribution des embeddings par type d'entité	39
Tableau 3.4	Résultats de calibration des poids du score hybride	46
Tableau 4.1	Benchmark des modèles d'embeddings sur 473 requêtes de test	51
Tableau 4.2	Comparaison retrieval : pgvector seul versus pgvector + graphe	52
Tableau 4.3	Pondérations retenues selon la fonction objectif du score hybride	57
Tableau 5.1	Lecture des hypothèses de recherche au regard des résultats	66
Tableau A.1	Labels de nœuds et relations structurantes du graphe	IX
Tableau A.2	Volume d'entités indexées dans la base vectorielle pgvector	X
Tableau A.3	Pipeline général implémenté : étapes, entrées et sorties	X
Tableau A.4	Correspondance entre objectifs spécifiques et choix méthodologiques	XI
Tableau A.5	Volumes réellement exploités par les modules de recommandation	XII
Tableau A.6	Comparaison directe entre le baseline et le modèle fine-tuné	XII
Tableau A.7	Synthèse des traitements de nettoyage utiles au moteur de recommandation	XIII
Tableau A.8	Diagnostic statistique du chunking documentaire	XIII
Tableau A.9	Couverture documentaire et duplication des chunks	XIV
Tableau A.10	Cohérence sémantique intra-chunk	XIV
Tableau A.11	Hétérogénéité des degrés par famille de noeuds	XV
Tableau A.12	Résumé du déplacement de rang après reranking Neo4j	XVI

TABLE DES FIGURES

Figure 1.1	Évolution simplifiée des modèles de langage et des représentations textuelles	13
Figure 1.2	Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG	16
Figure 2.1	Pipeline hybride proposé de recherche sémantique et de recommandation emploi-compétences	22
Figure 3.1	Répartition des offres par source après nettoyage	31
Figure 3.2	Compétences fréquentes et disponibilité descriptive	32
Figure 3.3	perte, généralisation, métriques de validation et calendrier du learning rate	35
Figure 3.4	Comparaison des modèles d’embeddings : qualité de classement, rappel et latence	36
Figure 3.5	Projection UMAP des embeddings indexés	36
Figure 3.6	Schéma physique de la base pgvector	38
Figure 3.7	Pipeline de chunking structurel des référentiels documentaires	40
Figure 3.8	Distribution des tailles, sources et couverture du chunking documentaire	41
Figure 3.9	Schéma logique du graphe de connaissances	42
Figure 3.10	Tableau de bord statistique du graphe Neo4j	43
Figure 3.11	Distribution des degrés dans la projection matching	44
Figure 3.12	Distribution comparée des scores pgvector et Neo4j utilisés dans le reranking	45
Figure 3.13	Workflow agentique orchestré dans LangGraph Studio	47
Figure 4.1	Comparaison des modèles d’embeddings sur le jeu de test	52
Figure 4.2	Effet de l’enrichissement graphe sur les métriques de retrieval	53
Figure 4.3	Diagnostics fonctionnels des chemins Neo4j mobilisés par le GraphRAG	54
Figure 4.4	Distribution des déplacements de rang après reranking par le graphe	56
Figure 4.5	Comparaison des pondérations retenues selon la fonction objectif	59
Figure 4.6	Distribution des scores du critic avec contexte réel et contexte mélangé	61
Figure 4.7	Effet du seuil du critic sur les décisions accept/revise	62
Figure A.1	Interface Streamlit du chatbot Agentic GraphRAG	VII
Figure A.2	Documentation interactive des endpoints FastAPI	VIII
Figure A.3	Localisation nettoyée des offres	XI
Figure A.4	Distribution de la cohérence sémantique intra-chunk par source	XIV
Figure A.5	Composition du graphe par types de noeuds et relations	XV

AVANT-PROPOS

L'Institut Sous-régional de Statistique et d'Économie Appliquée (ISSEA) est une institution spécialisée de la Communauté Économique et Monétaire de l'Afrique Centrale (CEMAC). Il assure la formation et le perfectionnement des cadres ressortissants des pays africains dans les domaines de la statistique et de l'économie appliquée.

Outre la formation continue, l'ISSEA dispose de cinq principales filières : les Ingénieurs Statisticiens Économistes (ISE classique et ISE cycle long), les Ingénieurs d'Application de la Statistique (IAS), les Analystes Statisticiens (AS), les Techniciens Supérieurs de la Statistique (TSS) et les Agents Techniques de la Statistique (ATS). Dans le cadre de ses programmes de formation continue, l'Institut propose également des formations de master et de licence professionnelle, notamment en Statistiques agricoles, en Base de données et Big Data, en Gestion et traitement statistique des données, ainsi qu'en Data Science et modélisation statistique.

Afin de permettre aux étudiants de mettre en pratique les connaissances théoriques et académiques acquises au cours de leur formation, l'ISSEA intègre des stages académiques et professionnels dans ses cursus. Dans la filière ISE cycle long, un premier stage est effectué en deuxième année pour une durée minimale de deux mois au sein d'une structure publique, parapublique ou privée. Le second stage, réalisé en cinquième année, marque la fin de la formation. D'une durée comprise entre trois et six mois, il donne lieu à la rédaction d'un mémoire professionnel en vue de l'obtention du diplôme d'Ingénieur Statisticien Économiste.

Le présent mémoire a ainsi été élaboré à l'issue d'un stage professionnel effectué à la Direction de la Recherche et Développement de KmerAI, du 10 mars au 30 juin 2026. Les travaux réalisés dans ce cadre portent sur le thème suivant : **CONCEPTION D'UN PIPELINE HYBRIDE DE RECHERCHE SÉMANTIQUE POUR LES RECOMMANDATIONS D'EMPLOI-COMPÉTENCES, FONDÉ SUR LES LLMs ET LES GRAPHEs DE CONNAISSANCES**

RÉSUMÉ

Dans les services numériques d'emploi, le rapprochement entre offres, profils candidats et compétences est freiné par l'hétérogénéité des descriptions, la variabilité des intitulés et le manque d'explicabilité des outils fondés sur les seuls mots-clés. Dans ce contexte, ce mémoire propose et opérationnalise un pipeline de recommandation emploi-compétences par recherche hybride, fondé sur les LLM et les graphes de connaissances. Le pipeline articule la normalisation des données et leur alignement avec ESCO, MEPC et NCF, l'encodage sémantique par SentenceTransformer, le rappel top-K dans PostgreSQL-pgvector, la vérification relationnelle dans Neo4j, le reranking, l'analyse du *skill gap* et la génération d'explications ancrées dans les preuves récupérées. Une implémentation pilote sur 13 957 offres, 41 298 candidats, 72 643 embeddings et 142 fragments documentaires documente la faisabilité technique de cette chaîne. Les diagnostics internes rapportent un Recall@10 de 0,8520 et un NDCG@10 de 0,6336 pour le retrieval sémantique ; après reranking relationnel, le NDCG@10 passe de 0,5575 à 0,7392 et le MRR@10 de 0,6908 à 0,8093. Ces chiffres ne constituent ni un taux de recrutement ni une validation métier, car ils reposent sur un proxy de pertinence. Le critic, calibré à 0,46, contrôle l'ancrage lexical de la réponse dans ses preuves sans juger la vérité métier. La contribution du mémoire est une architecture modulaire, traçable et explicable ; sa validation finale exige un jeu annoté indépendant et des tests avec des professionnels de l'emploi.

Mots-clés : pipeline hybride ; recherche sémantique ; recommandation emploi-compétences ; graphe de connaissances ; intelligence artificielle générative ; écart de compétences.

ABSTRACT

Digital employment services must match job offers, candidate profiles and skills described with heterogeneous vocabularies and uneven levels of detail. In this context, this thesis proposes and operationalises a job–skill recommendation pipeline based on hybrid retrieval, LLMs and knowledge graphs. The pipeline combines data normalisation and alignment with ESCO, MEPC and NCF; SentenceTransformer encoding; top- K semantic retrieval in PostgreSQL–pgvector; relational verification in Neo4j; reranking; skill-gap analysis; and evidence-grounded explanation generation. A pilot implementation using 13,957 job offers, 41,298 candidate profiles, 72,643 entity embeddings and 142 documentary chunks demonstrates the technical feasibility of the pipeline. The implementation and evaluation chapters report internal diagnostics: Recall@10 of 0.8520 and NDCG@10 of 0.6336 for semantic retrieval; NDCG@10 rising from 0.5575 to 0.7392 and MRR@10 from 0.6908 to 0.8093 after relational reranking. These figures are not hiring outcomes or business validation, because they rely on a relevance proxy. The generation critic, calibrated at 0.46, measures lexical grounding in retrieved evidence rather than job-domain truth. The defensible contribution is therefore a modular, explainable and auditable pipeline; independent expert annotation and user studies remain necessary for final validation.

Keywords: hybrid pipeline; semantic search; job–skill recommendation; knowledge graph; generative artificial intelligence; skill gap.

INTRODUCTION GÉNÉRALE

Contexte et justification

Les plateformes d'emploi doivent rapprocher des offres, des profils et des référentiels de compétences qui ne mobilisent ni les mêmes vocabulaires ni le même niveau de détail. Les intitulés de poste sont fréquemment ambigus, les compétences sont parfois implicites ou exprimées par des synonymes, et les descriptions incomplètes limitent la pertinence de filtres exacts. Il en résulte un double risque : écarter des profils réellement compatibles et produire des recommandations dont le raisonnement reste opaque. Les modèles d'appariement montrent que ces frictions informationnelles compliquent la rencontre entre candidats et emplois [28, 34].

La recherche sémantique réduit une partie de cette variabilité en rapprochant des formulations de sens voisin ; elle ne permet toutefois pas, à elle seule, de vérifier qu'un candidat détient les compétences indispensables, le niveau requis ou une trajectoire de formation cohérente. À l'inverse, un graphe de connaissances rend ces relations explicites, mais ne constitue pas seul un mécanisme de rappel efficace dans des collections textuelles volumineuses. La justification de l'étude réside donc dans leur complémentarité : la recherche hybride permet de constituer un vivier sémantiquement pertinent, puis de contrôler, classer et expliquer ce vivier au moyen des relations candidat-compétence-offre-formation.

KmerAI constitue le contexte applicatif de l'étude et la source des données pilotes. Le mémoire ne vise pas à établir un diagnostic général du marché du travail camerounais ; il vise à proposer et à opérationnaliser une chaîne de recommandation emploi-compétences reproductible, explicable et soumise au jugement humain. Cette chaîne relie la normalisation, l'indexation vectorielle, le graphe de connaissances, le reranking et la génération contrôlée par LLM.

Problématique

Le problème ne se réduit pas à apparier un profil à une offre. Une recommandation exploitable doit d'abord restituer des candidats ou offres sémantiquement proches, puis vérifier les exigences explicites – compétences, niveau, secteur ou parcours – et enfin fournir des preuves compréhensibles. Les modèles fondés uniquement sur des mots-clés échouent sur les équivalences de formulation ; les modèles fondés uniquement sur les embeddings peuvent ignorer une compétence obligatoire ; un LLM seul peut produire une explication plausible sans preuve

locale.

Les référentiels ESCO, MEPC et NCF fournissent des points d’ancrage, mais ils ne deviennent utiles qu’après normalisation, alignement et explicitation des relations dans un graphe. Le problème de recherche est donc architectural : comment articuler ces briques sans confondre rappel sémantique, contrôle relationnel, génération textuelle et décision humaine ?

La question centrale est la suivante :

Question centrale de recherche

Comment proposer et opérationnaliser un pipeline de recommandation emploi-compétences par recherche hybride, combinant rappel sémantique, vérification par graphe de connaissances et génération contrôlée par llm, afin de produire des classements traçables et des explications fondées sur des preuves ?

Cette question générale se décline en quatre questions de recherche, directement associées aux objectifs spécifiques :

1. Comment normaliser les offres et les profils, puis les aligner avec les référentiels ESCO, MEPC et NCF, afin de rendre comparables les métiers, les compétences et les niveaux de formation ?
2. Comment concevoir une recherche sémantique fondée sur l’encodage et l’indexation vectorielle afin de retrouver, pour un profil donné, les K offres candidates les plus proches sur le plan sémantique malgré les variations de vocabulaire ?
3. Comment exploiter les relations du graphe entre candidat, offre, compétence, métier et formation afin de vérifier les exigences, d’identifier les compétences acquises et manquantes, puis de réordonner les offres de manière traçable ?
4. Comment encadrer l’usage du LLM afin que chaque explication soit fondée sur les preuves récupérées, et comment distinguer les diagnostics techniques d’une évaluation de la pertinence métier fondée sur une annotation indépendante ?

Objectifs spécifiques de l’étude

L’objectif général de cette étude est de **proposer et d’opérationnaliser un pipeline de recommandation emploi-compétences par recherche hybride**. Ce pipeline combine un rappel sémantique, une vérification relationnelle et une génération contrôlée ; il constitue un outil d’aide à la décision et n’automatise ni le recrutement ni l’orientation. Plus spécifiquement, il s’agit de :

- **Structurer les données d'entrée.** Définir et opérationnaliser un contrat de données permettant de normaliser les offres et les profils, puis de les aligner avec ESCO, MEPC et NCF.
- **Mettre en place la recherche sémantique.** Concevoir et opérationnaliser l'encodage des textes, l'indexation vectorielle et la récupération des offres candidates les plus proches pour un profil donné.
- **Vérifier la compatibilité et classer les offres.** Formaliser une couche relationnelle fondée sur un graphe de connaissances afin de contrôler les exigences de l'offre, d'identifier les compétences acquises, requises et manquantes, puis de réordonner les offres de manière explicable.
- **Produire une explication contrôlée et préparer son évaluation.** Encadrer la génération par LLM à partir des preuves récupérées et définir un protocole distinguant les diagnostics techniques, l'annotation métier indépendante et les tests utilisateurs.

Ces objectifs portent sur la conception et l'opérationnalisation du pipeline. Ils ne visent ni à mesurer un taux de recrutement ni à conclure à une utilité métier généralisable sans annotation indépendante.

Intérêt de l'étude

L'intérêt de cette étude réside dans la séparation des responsabilités. La mémoire vectorielle fournit un rappel sémantique rapide ; le graphe porte les contraintes et les preuves relationnelles ; le LLM n'intervient qu'après récupération pour router une requête ou formuler une explication ancrée. Cette architecture est plus défendable qu'un score opaque ou qu'une réponse générée sans source : elle rend inspectables la recommandation, le *skill gap* et les limites de la donnée. Sa valeur attendue est méthodologique et transférable ; les résultats pilotes ne doivent pas être interprétés comme une mesure généralisable de l'emploi au Cameroun.

Propositions de conception

Afin de répondre à la problématique, la proposition repose sur trois exigences vérifiables, et non sur des promesses d'impact métier :

- **P1** : les données doivent être normalisées et ancrées dans des référentiels avant tout rapprochement sémantique ;
- **P2** : le vivier sémantique doit être contrôlé par des relations explicites entre candidat, compétences, offre et formation ;
- **P3** : toute réponse générée doit être associée à ses sources, ses scores et une possibilité de

révision ; elle reste soumise au jugement humain.

Méthodologie de l'étude

La démarche suit une logique de conception de pipeline. Les entrées sont les offres, profils, requêtes utilisateur et référentiels. Elles sont successivement normalisées, alignées et transformées en deux mémoires complémentaires : des embeddings interrogeables pour le rappel top- K et un graphe de connaissances pour les contraintes, les chemins explicatifs et le *skill gap*. Le reranking utilise ces signaux sans laisser le LLM décider seul du classement. Le LLM route une demande, appelle des outils autorisés, puis formule une réponse à partir d'un contexte et de preuves traçables. Les chapitres 3 et 4 conservent l'implémentation pilote et l'évaluation interne de cette proposition ; leurs métriques mesurent la cohérence des sorties et non une efficacité métier généralisable. La validation finale prévue requiert un jeu annoté indépendant, une séparation stricte entre calibration et test, ainsi que des évaluations par recruteurs, conseillers et candidats.

Plan du travail

Le mémoire est organisé en deux parties comprenant cinq chapitres. La première établit les fondements théoriques et spécifie le pipeline, de la préparation des données à l'orchestration contrôlée. La seconde documente son implémentation pilote et évalue ses comportements internes avant une discussion critique des résultats, des limites et du protocole de validation métier à venir. Une conclusion générale rappelle enfin les apports de la proposition, ses conditions de validité et ses perspectives.

Première partie

CADRE THÉORIQUE ET EXPÉRIMENTAL

FONDEMENTS THÉORIQUES ET REVUE DE LA LITTÉRATURE SUR L'IA GÉNÉRATIVE ET LES SYSTÈMES DE RECOMMANDATION

Ce chapitre situe le mémoire dans la littérature sur l'appariement emploi-compétences, les systèmes de recommandation, l'intelligence artificielle générative, les graphes de connaissances et la génération augmentée par récupération. Il ne vise pas à reprendre l'ensemble des raisonnements des ouvrages ou articles consultés. Il sélectionne les résultats qui éclairent directement le thème du stage : concevoir un système capable de recommander des offres, d'identifier des écarts de compétences et de produire des explications fondées sur des données structurées et non structurées.

1.1 Repères conceptuels de l'étude

1.1.1 Vocabulaire opérationnel : du matching à l'Agentic RAG

Avant d'entrer dans la revue de littérature, il est nécessaire de stabiliser les concepts utilisés dans la suite du mémoire. Cette clarification évite une confusion fréquente : employer indifféremment IA générative, LLM, RAG, GraphRAG, moteur de recommandation et chatbot. Dans ce travail, ces termes désignent des fonctions complémentaires, mais non interchangeables.

Matching emploi-compétences.

Le matching emploi-compétences désigne le processus qui consiste à rapprocher un profil candidat d'une offre, d'un métier ou d'une trajectoire de formation à partir de caractéristiques observables : compétences, niveau de formation, expérience, secteur, localisation et objectif professionnel. Il est traité ici comme un problème d'appariement économique, mais aussi comme un problème de représentation des connaissances.

Système de recommandation.

Un système de recommandation est un dispositif algorithmique qui classe des éléments

selon leur pertinence probable pour un utilisateur ou un cas d'usage [37]. Dans ce mémoire, les éléments recommandés peuvent être des offres d'emploi, des compétences, des métiers ou des pistes de formation. La pertinence ne renvoie donc pas seulement à une préférence ; elle renvoie aussi à une compatibilité professionnelle explicable.

Skill gap.

Le *skill gap*, ou écart de compétences, correspond à la différence entre les compétences déjà détenues par un candidat et celles attendues pour une offre ou un métier cible. Il permet de dépasser une logique binaire compatible/non compatible : un candidat peut être proche d'un poste tout en nécessitant une montée en compétences.

Embedding et recherche sémantique.

Un embedding est une représentation vectorielle d'un texte, d'un profil, d'une offre ou d'une compétence. La recherche sémantique utilise cette représentation pour retrouver des objets proches en sens, même lorsque les formulations diffèrent. Elle répond à un problème récurrent des données d'emploi : des expressions comme « data analyst », « analyste de données » et « reporting statistique » peuvent désigner des zones de compétences voisines.

Base de données vectorielle.

Une base de données vectorielle stocke des représentations numériques d'objets et permet de retrouver rapidement ceux dont les vecteurs sont les plus proches d'une requête. Elle sert donc à rechercher des textes similaires par leur sens, et non seulement par des mots identiques. Dans ce mémoire, cette fonction est mise en œuvre avec PostgreSQL et son extension pgvector [33]. Elle présélectionne les offres, métiers, compétences ou documents proches, sans décider seule de leur compatibilité professionnelle.

Graphe de connaissances Neo4j.

Un graphe de connaissances représente des entités et leurs relations : candidat, offre, compétence, métier, secteur, niveau, formation et référentiel [19]. Dans ce mémoire, Neo4j sert à représenter les relations explicites entre ces entités. Il permet de répondre à des questions que la seule similarité vectorielle ne traite pas correctement : quelles compétences une offre exige-t-elle ? quelles compétences le candidat possède-t-il ? quel niveau de formation est demandé ? quels chemins relient un métier à une formation ?

IA générative, LLM et prompting.

L'intelligence artificielle générative désigne les modèles capables de produire du contenu, notamment du texte. Un LLM est un grand modèle de langage entraîné sur de grands corpus

textuels et capable de générer, reformuler ou synthétiser des réponses. Dans ce mémoire, le LLM n'est pas le moteur de vérité du système. Il intervient comme générateur contrôlé : il reçoit un contexte construit à partir de pgvector, Neo4j et des documents, puis rédige une réponse en français. Le prompting désigne l'ensemble des consignes données au modèle pour encadrer son comportement, limiter les inventions et structurer la réponse.

RAG, GraphRAG et Agentic RAG.

La RAG combine récupération d'information et génération : le système recherche d'abord des éléments pertinents, puis les transmet au LLM pour produire une réponse [24]. Le GraphRAG étend cette logique en ajoutant des faits et chemins issus d'un graphe de connaissances. L'Agentic RAG ajoute une couche d'orchestration : l'agent analyse l'intention, choisit les outils, récupère les données, construit le contexte, génère la réponse et peut demander une révision si le contexte est insuffisant. Dans ce mémoire, cette orchestration est assurée par LangGraph.

Critic et traçabilité.

Le critic est un mécanisme de contrôle qui examine si la réponse générée semble suffisamment couverte par le contexte récupéré. Il ne remplace pas une validation humaine, mais il réduit le risque d'une réponse fluide et insuffisamment fondée. La traçabilité désigne la capacité à afficher les outils mobilisés, les sources consultées et le chemin suivi par le workflow. Elle est indispensable dans un système d'aide à la décision, car une recommandation professionnelle doit être vérifiable.

1.2 Fondements économiques et problématique d'appariement

1.2.1 Marché du travail, recherche d'emploi et frictions

Le problème traité dans ce mémoire relève d'abord de l'économie du travail. Les modèles de recherche et d'appariement montrent que le chômage peut coexister avec des postes vacants lorsque la rencontre entre travailleurs et entreprises est coûteuse, lente ou imparfaitement informée [28, 34]. Cette lecture est importante : un système de recommandation emploi-compétences n'est pas seulement un outil informatique ; c'est un dispositif d'intermédiation qui cherche à réduire les coûts de recherche et à améliorer la qualité des mises en relation.

La littérature sur l'asymétrie d'information renforce ce diagnostic. George A. Akerlof montre que l'incertitude sur la qualité peut dégrader les échanges lorsque les acteurs ne disposent pas de signaux fiables [2]. Sur le marché de l'emploi, cette asymétrie apparaît des deux côtés : le

recruteur ne voit pas toujours les compétences réelles du candidat, et le candidat ne connaît pas toujours les exigences effectives du poste. La théorie du signalement de Spence explique alors le rôle des diplômes, certifications et expériences comme signaux observables [38]. Mais ces signaux restent incomplets lorsque les métiers évoluent rapidement, notamment dans les domaines numériques où les compétences techniques sont souvent décrites par des formulations instables.

1.2.2 Capital humain, tâches et inadéquation des compétences

La théorie du capital humain rappelle que la formation et l'expérience sont des investissements productifs [10]. Toutefois, la possession d'un capital humain ne garantit pas l'appariement. Encore faut-il que les compétences acquises soient visibles, comparables et alignées sur les compétences demandées. Autor montre que le progrès technique transforme surtout les tâches composant les métiers, et non uniquement les intitulés d'emploi [5]. Pour un système de recommandation, cela impose de descendre au niveau des compétences et des tâches plutôt que de s'arrêter aux titres de poste.

L'apport de cette littérature est de montrer que l'inadéquation emploi-compétences relève autant de la circulation imparfaite de l'information que d'un manque de formation. Sa limite, pour notre objet, est qu'elle ne fournit pas directement les outils de traitement des données nécessaires pour extraire, normaliser, comparer et expliquer les compétences. La section suivante examine donc les systèmes de recommandation comme réponse algorithmique à ce problème d'appariement.

1.3 Systèmes de recommandation et person-job fit

1.3.1 Approches par contenu, collaboratives et hybrides

Les systèmes de recommandation cherchent à classer des items selon leur pertinence probable pour un utilisateur. La littérature distingue classiquement les approches fondées sur le contenu, le filtrage collaboratif et les approches hybrides [1, 37]. Dans le contexte de ce mémoire, l'utilisateur peut être un candidat, un conseiller emploi ou une plateforme ; les items peuvent être des offres, compétences, métiers ou formations.

Cette typologie doit toutefois être actualisée. Les approches profondes de recommandation prolongent le filtrage par contenu en encodant les descriptions d'offres et de profils dans des représentations contextuelles. BERT fournit des représentations sensibles au contexte, tandis que Sentence-BERT rend ces représentations directement comparables pour la recherche par

similarité [13, 36]. Dans un domaine où les intitulés sont hétérogènes, cette couche sémantique peut rapprocher « analyste de données », « data analyst » et des descriptions de compétences voisines ; elle ne démontre cependant pas, à elle seule, la compatibilité professionnelle.

Les approches fondées sur la connaissance complètent ce signal en imposant une structure explicite : ontologies, référentiels et graphes relient métiers, compétences, formations et profils. Des travaux récents en ressources humaines illustrent cette direction : HRGraph combine extraction par LLM et graphe de connaissances pour des tâches de recommandation d’emploi [42], tandis que Yang et Shen articulent graphe de connaissances, GCN, apprentissage par renforcement et filtrage collaboratif profond pour la prédiction de compétences [43]. Ces résultats ne sont pas transposables automatiquement au cas d’étude ; ils motivent plutôt une architecture qui sépare rappel sémantique, contrôle relationnel et décision humaine.

Le tableau 1.1 compare ces familles dans le contexte de l’emploi [32, 22, 17, 12].

Table 1.1 – Comparaison des principales approches de recommandation

Approche	Principe	Avantages	Inconvénients	Limites dans cette étude
Par contenu	Compare les caractéristiques des profils et des offres.	Fonctionne sans historique de clics et permet d'expliquer les ressemblances.	Dépend de la qualité des descriptions et peut rester trop proche du profil initial.	Les formulations varient entre sources et certaines descriptions sont incomplètes.
Collaborative	Utilise les comportements passés d'utilisateurs semblables.	Peut découvrir des préférences qui ne sont pas écrites dans les profils.	Exige beaucoup d'interactions et gère mal les nouveaux utilisateurs ou les nouvelles offres.	Les historiques de clics, candidatures et recrutements sont insuffisants.
Sémantique profonde	Encode offres et profils par des représentations contextuelles (BERT et SBERT).	Dépasse les synonymes, variantes d'intitulés et proximités lexicales.	Sensible au domaine, aux données d'entraînement et à la qualité des labels.	Sert au rappel initial ; une similarité forte ne constitue pas une preuve de compatibilité.
Fondée sur la connaissance	Exploite ontologies, référentiels ou graphe métier-compétence-formation.	Rend les contraintes et les explications relationnelles explicites.	Dépend de la couverture, de la qualité et de la maintenance des relations.	Le graphe vérifie et explique le vivier ; il ne remplace pas les données manquantes.
Hybride	Combine plusieurs signaux : textes, relations et règles métier.	Compense les faiblesses d'une source et améliore l'explication.	Demande de calibrer les signaux et rend le système plus complexe.	Sa qualité dépend encore de la fiabilité des données et des relations du graphe.

Source : Nos travaux, synthèse de Pazzani et Billsus (2007), Koren et al. (2009), He et al. (2017) et Burke (2002).

1.3.2 Spécificité du person-job fit

La recommandation d'emploi se distingue de la recommandation de films ou de produits. Une offre n'est pas seulement un item susceptible de plaire ; elle impose des contraintes de qualification, d'expérience, de localisation, de disponibilité et de compétences. La littérature récente sur le *person-job fit* insiste sur cette double sélection : le candidat doit être intéressé

par l'offre, mais l'employeur doit aussi pouvoir considérer le profil comme compatible [44]. La recommandation devient donc un problème bilatéral.

Cette spécificité change l'interprétation des scores. Dans le commerce électronique, une recommandation peut viser la probabilité de clic ou d'achat. Dans l'emploi, elle doit distinguer trois situations :

- le candidat est immédiatement compatible avec le poste ;
- le candidat est proche du profil attendu, mais doit encore acquérir certaines compétences ;
- le candidat est trop éloigné du poste cible pour une recommandation immédiate.

La notion de *skill gap* devient alors centrale.

Les travaux récents mobilisant les graphes et les LLM dans les ressources humaines vont dans ce sens. HRGraph propose par exemple d'exploiter des graphes de connaissances enrichis par LLM pour la recommandation d'emploi [42]. D'autres travaux s'intéressent à la prédiction des compétences et à la construction de graphes de talents pour les ressources humaines [43]. Ces contributions confirment l'intérêt des graphes pour représenter les relations entre compétences, métiers et profils, mais elles ne résolvent pas automatiquement la question de l'ancrage local : nomenclatures camerounaises, données réelles de plateforme, contraintes de formation et explication opérationnelle.

1.4 LLMs, embeddings et graphes de connaissances

1.4.1 Grands modèles de langage et génération contrôlée

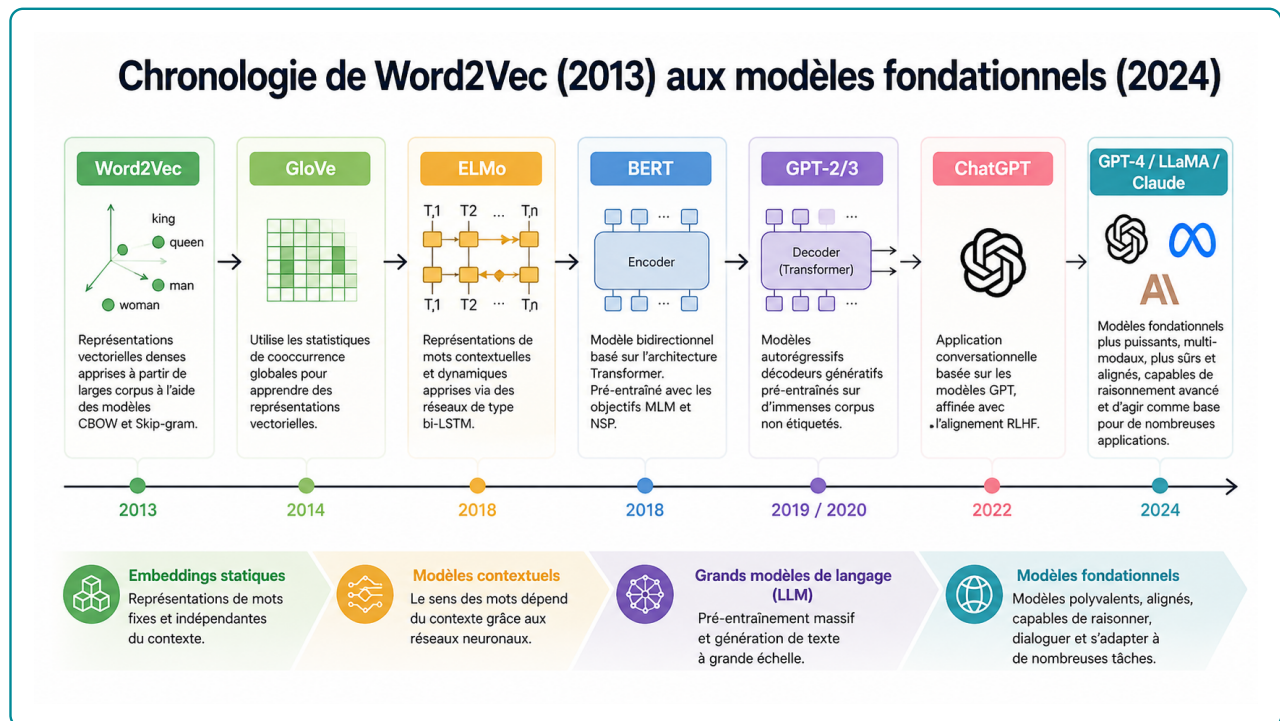
Les grands modèles de langage ont transformé le traitement automatique du langage naturel en permettant des tâches de génération, synthèse, reformulation et raisonnement assisté. Leur rupture technique vient principalement de l'architecture Transformer proposée par Vaswani et al., fondée sur le mécanisme d'attention [40]. Avant cette rupture, les représentations textuelles étaient passées des sacs de mots aux embeddings denses comme Word2Vec, qui apprennent des vecteurs capturant des régularités sémantiques et syntaxiques [27]. Les mécanismes d'attention introduits en traduction neuronale ont ensuite permis aux modèles de pondérer dynamiquement les parties pertinentes d'une séquence [6].

Deux grandes familles de modèles sont utiles pour ce mémoire. La première regroupe les modèles de représentation, comme BERT et Sentence-BERT. BERT apprend des représentations contextuelles bidirectionnelles adaptées à la compréhension du texte [13]. Sentence-BERT

adapte cette logique pour produire des embeddings de phrases comparables efficacement par similarité cosinus [36]. La seconde famille regroupe les modèles génératifs, comme GPT, T5 ou les modèles instruction-tuned. GPT-3 a montré qu'un modèle de grande taille pouvait effectuer des tâches variées à partir de quelques exemples dans le prompt [11]. T5 a proposé une formulation unifiée des tâches NLP comme transformation texte-vers-texte [35]. L'apprentissage par retour humain a ensuite amélioré la capacité des modèles à suivre des instructions [31].

L'acquis de cette littérature est considérable : les LLM savent reformuler, synthétiser et produire des explications lisibles. Sa limite, pour notre problématique, tient au statut de la vérité : un modèle génératif peut produire une réponse plausible sans être fondée sur les données locales. Dans un système d'emploi, cette limite impose une génération contrôlée par des sources récupérées.

Figure 1.1 – Évolution simplifiée des modèles de langage et des représentations textuelles



Source : Nos travaux, schématisation conceptuelle à partir de Mikolov et al. (2013)

1.4.2 Embeddings, recherche sémantique et bases vectorielles

Après la clarification conceptuelle, l'enjeu est de savoir dans quelles conditions les représentations vectorielles sont fiables pour la recherche d'information. Les benchmarks montrent que les modèles d'embeddings doivent être évalués sur des tâches variées, car un bon modèle généraliste peut être insuffisant dans un domaine spécialisé [39, 29]. Cette observation justifie

l'adaptation d'un modèle SentenceTransformer au vocabulaire emploi-compétences.

Techniquement, un bi-encodeur transforme séparément la requête q et un document d en vecteurs $e(q)$ et $e(d)$. Après normalisation L_2 , leur proximité est calculée par similarité cosinus :

$$s_{sem}(q, d) = \frac{e(q)^\top e(d)}{\|e(q)\|_2 \|e(d)\|_2}.$$

Le moteur restitue un vivier top- K plutôt qu'une décision finale. Pour conserver une latence praticable, ce vivier est retrouvé par recherche approximative de plus proches voisins : avec HNSW, les paramètres de construction M et $ef_{construction}$ règlent la richesse de l'index, alors que ef_{search} arbitre, à l'interrogation, entre rappel et temps de réponse [26, 20]. Le fine-tuning contrastif est donc pertinent seulement s'il améliore le classement sur des paires emploi-profil tenues à l'écart de l'apprentissage.

La recherche sémantique résout une partie du problème de vocabulaire, mais elle reste une approximation. Deux textes proches peuvent cacher une incompatibilité : niveau insuffisant, compétence obligatoire absente, localisation incompatible ou métier trop éloigné. La similarité vectorielle doit donc être traitée comme un signal de présélection, non comme une décision finale.

La littérature sur la recherche approximative de plus proches voisins montre par ailleurs que le passage à l'échelle exige des structures d'indexation efficaces. HNSW fournit un cadre robuste pour la recherche vectorielle approximative [26], tandis que FAISS a popularisé la recherche vectorielle à grande échelle [20]. Dans ce mémoire, l'utilisation de PostgreSQL avec pgvector répond à une logique pragmatique : conserver les embeddings dans une base relationnelle exploitable, interrogeable et intégrable au reste du pipeline [33].

1.4.3 Graphes de connaissances et recommandation relationnelle

Hogan et al. montrent que les graphes de connaissances servent à structurer des informations hétérogènes, à rendre explicites des relations sémantiques et à soutenir des tâches de recherche ou de raisonnement [19]. Dans le domaine emploi-compétences, cette représentation complète la recherche vectorielle : elle rend visibles les relations entre candidats, offres, compétences, métiers, formations et niveaux.

La recommandation par graphe a également connu des avancées importantes. Les graph neural networks ont fourni des outils pour propager de l'information dans des graphes [21, 41]. LightGCN a montré que, dans la recommandation collaborative, une architecture simplifiée de propagation sur graphe pouvait être très efficace [18]. Ces travaux établissent l'intérêt

de la structure relationnelle pour la recommandation.

Cependant, le présent mémoire ne cherche pas principalement à entraîner un GNN. Le choix méthodologique est différent : utiliser un graphe de connaissances comme support d'explication, de contraintes et de calcul du skill gap. Ce choix est plus adapté au contexte disponible. Les données d'interaction ne sont pas assez riches pour justifier un modèle collaboratif profond, tandis que les relations métier-compétence-formation sont directement interprétables.

1.5 RAG, GraphRAG et orchestration agentique

1.5.1 De la RAG au GraphRAG

La RAG, combine une étape de recherche d'information et une étape de génération. Lewis et al. montrent que l'accès à une mémoire externe améliore les tâches intensives en connaissances, car le modèle ne dépend plus uniquement de ses paramètres internes [24]. Les synthèses récentes sur la RAG insistent sur les mêmes enjeux : qualité de la récupération, actualisation des connaissances, évaluation de la factualité et articulation entre retriever et générateur [15].

Dans une RAG native, le système récupère des documents proches d'une requête, puis les transmet au LLM. Cette architecture est utile, mais elle reste centrée sur des passages textuels. Pour le matching emploi-compétences, le contexte doit aussi intégrer des relations : compétences possédées, compétences requises, niveau de formation, métier cible, secteur et formation possible. Le GraphRAG étend alors la récupération vers des informations structurées par graphe.

Figure 1.2 – Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG

Différence fonctionnelle entre RAG, GraphRAG et Agentic RAG			
Aspect	RAG	GraphRAG	Agentic RAG
 Approche principale	Récupération de documents pertinents pour augmenter la réponse du LLM. 	Exploite la structure des données et les relations sémantiques (graphe). 	Utilise un agent autonome pour planifier, rechercher, agir et synthétiser. 
 Représentation des données	Documents non structurés (textes, PDFs, etc.) 	Grappe de connaissances (entités, relations, événements). 	Documents + outils + mémoire + instructions pour l'agent. 
 Récupération d'information	Recherche sémantique par similarité vectorielle (embeddings). 	Navigation dans le graphe (requêtes sur entités et relations). 	L'agent décide quoi chercher, où chercher et comment combiner les infos. 
 Raisonnement	Limité au contexte récupéré + capacités du LLM. 	Raisonnement basé sur les relations et la structure du graphe. 	Raisonnement multi-étapes, planification et adaptation autonome. 
 Orchestration	Pipeline fixe : Retrieval → Augmentation → Generation 	Pipeline avec requêtes sur le graphe + génération. 	Boucle agentique : Planifier → Agir → Observer → Réajuster → Répondre. 
 Cas d'usage typiques	FAQ, recherche documentaire, chatbots de base. 	Analyse relationnelle, détection d'insights, contexte complexe. 	Tâches complexes, multi-sources, prise de décision, workflows autonomes. 

Source : Nos travaux sur drawio

La figure 1.2 distingue trois niveaux fonctionnels. La RAG récupère surtout des documents proches d’une requête et les transmet au LLM ; elle reste donc efficace pour une recherche documentaire, mais son raisonnement dépend du contexte retrouvé. Le GraphRAG ajoute une représentation relationnelle : les entités et leurs liens permettent de naviguer entre compétences, métiers, offres, niveaux et formations. L’Agentic RAG va plus loin : il ne se limite pas à récupérer une information, il planifie les étapes, choisit les outils, observe les résultats, ajuste le contexte et produit une réponse. Cette distinction justifie le choix du mémoire : combiner retrieval vectoriel, graphe Neo4j et orchestration LangGraph pour traiter des requêtes emploi-compétences plus complexes qu’une simple recherche de documents.

1.5.2 Agentic RAG, critic et traçabilité

Les travaux ReAct montrent l’intérêt d’alterner raisonnement et action outillée dans les modèles de langage [45]. Self-RAG va plus loin en introduisant une logique d’auto-réflexion pour décider quand récupérer de l’information et comment critiquer la réponse [4]. Pour ce mémoire, ces travaux ne doivent pas être interprétés comme une autorisation à laisser un agent

décider librement. Ils justifient plutôt une orchestration contrôlée : classifier l'intention, choisir les sources, récupérer les preuves, construire le contexte, générer, puis vérifier si la réponse est suffisamment fondée.

Le manque de la littérature est ici opérationnel. Beaucoup de travaux décrivent des architectures générales, mais la fiabilité dépend de détails concrets : qualité des chunks, schéma du graphe, robustesse des requêtes, traçabilité des sources, gestion des cas sans résultat et critères de révision.

Dans le pipeline proposé, cette exigence se traduit par un état explicitement typé : requête, intention, plan d'outils, résultats de recherche, preuves, réponse, score du critic et trace. Les transitions sont bornées : après un top- K vectoriel, le graphe vérifie les relations autorisées ; une requête Text2Cypher est limitée au schéma et à la lecture seule ; le critic ne peut ouvrir qu'une révision limitée avant l'acceptation ou le refus de répondre. Cette conception distingue le rôle du LLM – interpréter ou formuler – de celui des sources, qui portent les preuves. Elle limite les boucles incontrôlées et rend chaque recommandation audit-able.

1.5.3 Évaluer la performance et la factualité

L'évaluation d'un système emploi-compétences ne peut pas se limiter à la fluidité de la réponse générée. Elle doit d'abord mesurer la qualité de la récupération d'information : le système retrouve-t-il les offres, compétences ou documents réellement pertinents ? La littérature en recherche d'information utilise pour cela des métriques de classement comme Precision@ k , Recall@ k , MRR et NDCG [39, 29]. Ces métriques supposent l'existence d'un ensemble de résultats pertinents, noté $Rel(q)$, pour une requête q , et d'une liste classée de résultats retournés par le système.

La **Precision@ k** mesure la proportion de résultats pertinents parmi les k premiers résultats retournés. Si $R_k(q)$ désigne l'ensemble des k premiers éléments proposés pour la requête q , alors :

$$Precision@k(q) = \frac{|R_k(q) \cap Rel(q)|}{k}. \quad (1.1)$$

Cette métrique répond à la question suivante : parmi les premières recommandations affichées, combien sont réellement utiles ? Dans ce mémoire, Precision@10 porte sur les dix offres placées en tête pour chaque candidat. Sa limite est qu'elle ne mesure pas si le système a retrouvé tous les bons résultats disponibles.

Le **Recall@ k** mesure au contraire la proportion des éléments pertinents retrouvés dans les

k premiers résultats :

$$Recall@k(q) = \frac{|R_k(q) \cap Rel(q)|}{|Rel(q)|}. \quad (1.2)$$

Cette métrique répond à la question : parmi toutes les offres ou compétences pertinentes, quelle part le système a-t-il réussi à faire remonter ? Elle est importante pour ne pas éliminer trop tôt des offres utiles. En recommandation emploi, un faible Recall@k peut signifier que le moteur rate des opportunités pertinentes pour le candidat.

Le **MRR** (*Mean Reciprocal Rank*) évalue la position du premier résultat pertinent. Pour une requête q , si $rank_q$ est le rang du premier résultat pertinent, alors le score associé est :

$$RR(q) = \frac{1}{rank_q}. \quad (1.3)$$

Sur un ensemble de N requêtes, le MRR est :

$$MRR = \frac{1}{N} \sum_{q=1}^N \frac{1}{rank_q}. \quad (1.4)$$

Cette métrique est utile lorsque l'on veut savoir si le système place rapidement au moins une bonne réponse. Dans une interface de recommandation, cela correspond à une logique pratique : l'utilisateur regarde souvent les premiers résultats avant de parcourir toute la liste.

Le **NDCG@k** (*Normalized Discounted Cumulative Gain*) tient compte non seulement de la présence des résultats pertinents, mais aussi de leur ordre et éventuellement de leur degré de pertinence. Si rel_i désigne le niveau de pertinence du résultat placé au rang i , alors :

$$DCG@k = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)}. \quad (1.5)$$

Le score est ensuite normalisé par le meilleur classement possible, noté $IDCG@k$:

$$NDCG@k = \frac{DCG@k}{IDCG@k}. \quad (1.6)$$

Cette métrique est particulièrement adaptée lorsque tous les résultats pertinents ne se valent pas. Par exemple, une offre parfaitement alignée avec les compétences du candidat devrait être mieux classée qu'une offre seulement partiellement compatible.

Dans cette étude, la métrique principale retenue pour comparer et calibrer le reranking est donc **NDCG@10**. Elle récompense simultanément la pertinence graduée et le bon ordre des dix premières offres, qui constituent la zone effectivement consultée par l'utilisateur. Recall@10 est

la métrique de garde-fou : un gain de NDCG ne serait pas acceptable s'il faisait disparaître une grande part des offres pertinentes. Precision@10 et MRR@10 restent secondaires, respectivement pour contrôler la qualité moyenne de la liste et la position de la première offre pertinente [39, 29]. Ces mesures sont interprétées comme des diagnostics internes tant qu'aucun jugement métier indépendant n'est disponible.

Dans une architecture RAG, l'évaluation ne s'arrête pas au classement des résultats. Elle doit aussi mesurer la qualité de la réponse générée à partir du contexte récupéré. RAGAS distingue notamment la fidélité au contexte, la pertinence de la réponse, la qualité de la récupération et l'usage des sources [14]. La **fidélité** vérifie si les affirmations de la réponse sont supportées par les éléments récupérés. La **pertinence de réponse** vérifie si la réponse traite effectivement la question posée. La **couverture des sources** vérifie si le contexte contient suffisamment d'informations pour répondre. Enfin, la **traçabilité** vérifie si les preuves ou citations sont identifiables.

Ces critères sont moins simples à formaliser que Precision@k ou Recall@k, car ils portent sur du langage naturel. Des approches comme G-Eval utilisent des LLM comme juges pour approximer l'évaluation humaine [25]. Cette pratique doit rester prudente : un juge automatique peut aider à comparer des variantes de système, mais il ne remplace pas une validation métier. Dans ce mémoire, l'évaluation doit donc combiner des métriques de retrieval pour pgvector et Neo4j, des critères de factualité pour la réponse générée, et une interprétation professionnelle de l'utilité des recommandations.

1.6 Positionnement de l'étude

Les travaux présentés dans cette revue montrent que le matching emploi-compétences ne peut pas être traité comme une simple recherche de mots-clés. Il exige à la fois une représentation sémantique des textes, une structuration des relations métier-compétence-formation et un mécanisme de génération capable d'expliquer les résultats sans s'éloigner des données disponibles. Cette étude se positionne à l'intersection de trois familles d'approches. La première est la recherche sémantique par embeddings, utilisée pour retrouver rapidement les offres, compétences ou documents proches d'une requête. La deuxième est le graphe de connaissances, mobilisé pour représenter les relations entre candidats, offres, compétences, métiers, secteurs et niveaux de formation. La troisième est l'orchestration agentique, qui organise le retrieval, l'enrichissement par le graphe, la génération et le contrôle de la réponse. La contribution du mémoire n'est donc pas de proposer un nouveau modèle de langage ou un nouvel algorithme

de recommandation générique. Elle consiste à concevoir et évaluer une architecture hybride adaptée au contexte emploi-compétences : pgvector sert à la présélection sémantique, Neo4j apporte le signal relationnel et LangGraph orchestre les étapes de décision. Ce positionnement répond à une limite de la littérature : les approches générales de RAG ou de recommandation expliquent rarement comment articuler, dans un même système opérationnel, données locales, référentiels métiers, graphe, scoring hybride et contrôle de la génération. Le système proposé reste un outil d'aide à la décision. Il ne remplace ni le recruteur, ni le conseiller emploi. Son rôle est de rendre les correspondances plus lisibles : quelles offres sont proches du profil, quelles compétences expliquent le rapprochement, quels écarts subsistent et quelles pistes de montée en compétence peuvent être envisagées.

Conclusion

Ce chapitre a établi le socle théorique du mémoire. Le matching emploi-compétences est d'abord un problème d'information imparfaite : les offres, profils, compétences et formations sont décrits avec des vocabulaires hétérogènes. Les systèmes de recommandation, les embeddings, les graphes de connaissances et la RAG apportent chacun une réponse partielle à ce problème, mais aucun ne suffit isolément. La revue conduit donc à une position claire : le système à concevoir doit combiner une mémoire vectorielle pour retrouver rapidement des objets proches, un graphe pour vérifier les relations métier-compétence-formation, et une orchestration agentique pour produire des réponses contrôlées.

SOURCES DE DONNÉES ET APPROCHE MÉTHODOLOGIQUE

Ce chapitre spécifie le pipeline proposé et les données qui l'alimentent. Il décrit la transformation des sources en représentations, puis les règles qui organisent leur interrogation ; l'implémentation pilote et les diagnostics internes correspondants sont conservés dans les chapitres suivants.

2.1 Cadre méthodologique et sources de données

Cette première section fixe le cadre de travail. Le pipeline est conçu comme un artefact d'aide à la décision, pas comme un modèle isolé ni comme un dispositif de décision automatique.

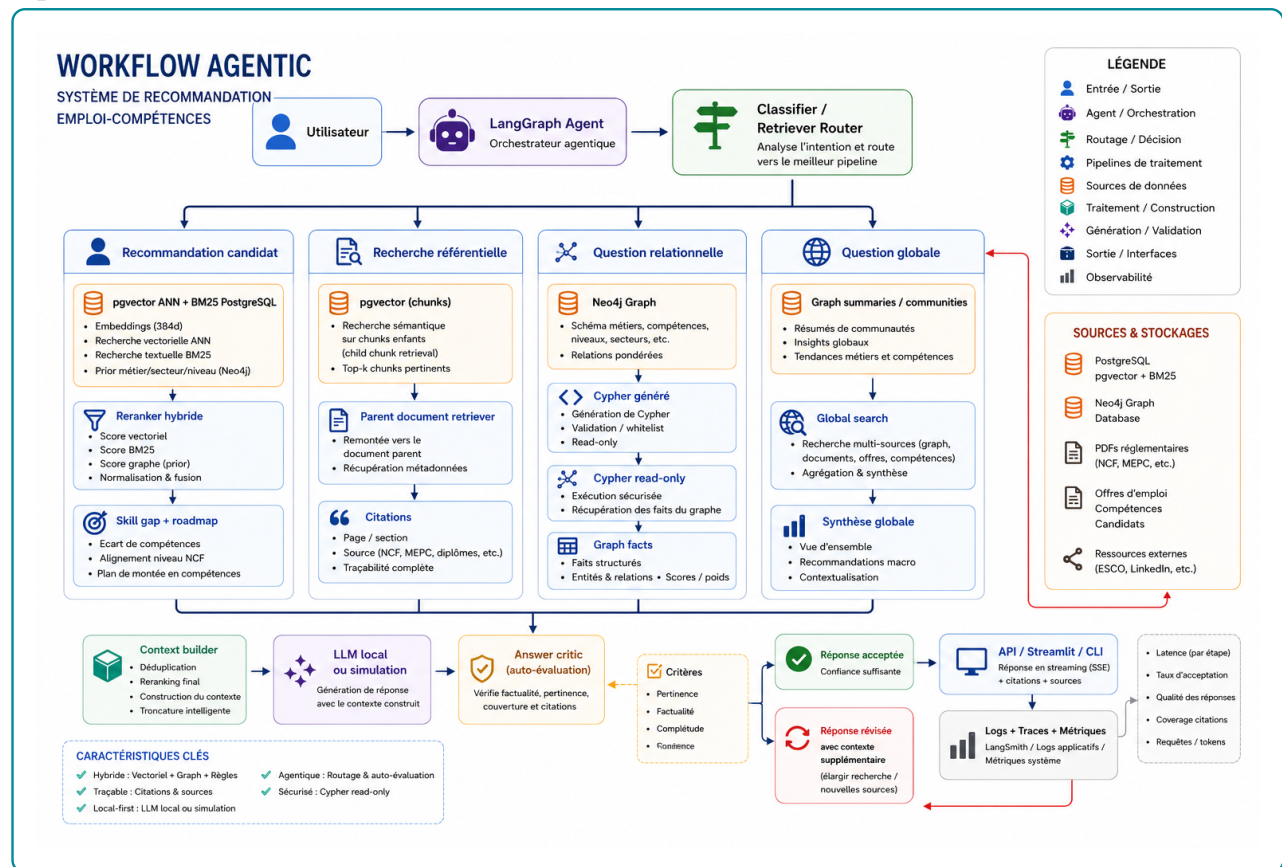
2.1.1 Démarche d'ingénierie appliquée retenue

Le pipeline suit une chaîne de rôles séparés. Les offres, les profils et les référentiels sont d'abord collectés, nettoyés et harmonisés. Les informations comparables – métiers, compétences, niveaux de formation et localisations – sont ensuite représentées dans une mémoire vectorielle et un graphe de connaissances. La mémoire vectorielle produit un vivier d'offres proches ; le graphe vérifie les compétences communes et manquantes ; le reranking classe les résultats ; le LLM formule enfin une explication à partir des preuves. Cette organisation répond à quatre exigences. Premièrement, chaque recommandation doit être reliée aux données utilisées. Deuxièmement, le classement ne doit pas dépendre d'un seul indice. Troisièmement, le pipeline doit signaler les compétences acquises et celles qui manquent. Enfin, la réponse rédigée doit rester fidèle aux informations retrouvées [24, 14]. LangGraph coordonne ces étapes dans un graphe d'états et conserve la trace des outils appelés [23]. Si les informations récupérées sont insuffisantes, la règle est de rechercher à nouveau ou de signaler une information insuffisante, jamais de compléter librement par une réponse non sourcée.

La figure 2.1 montre que le système ne traite pas toutes les questions de la même manière. Après réception de la demande, l'agent identifie le besoin de l'utilisateur et choisit le parcours adapté : recommander des offres à un candidat, consulter un référentiel, interroger les relations

du graphe ou produire une vue d'ensemble. Cette étape évite d'appeler inutilement toutes les sources pour chaque question.

Figure 2.1 – Pipeline hybride proposé de recherche sémantique et de recommandation emploi-compétences



Source : Nos travaux, synthèse du workflow méthodologique.

Dans le pipeline proposé, les éléments récupérés sont réunis dans un contexte commun. Pour une recommandation, le système rapproche d'abord le profil des offres, puis vérifie les compétences et les niveaux de formation avant de calculer les écarts. Pour une question documentaire, il récupère les passages utiles et leurs sources. Pour une question relationnelle, il utilise les liens du graphe. Le modèle de langage rédige enfin la réponse à partir de ces éléments. Un module de contrôle, appelé *critic*¹, décide si la réponse peut être acceptée, révisée une fois ou accompagnée d'un message d'information insuffisante. Le workflow se termine par l'affichage de la réponse dans l'API, le chatbot ou l'interface en ligne, avec les traces utiles à sa vérification.

1. Le *critic* vérifie si la réponse générée est suffisamment appuyée par les informations récupérées. Il ne remplace pas une validation humaine.

2.1.2 Sources de données et leur rôle dans le système

Les données se répartissent en deux groupes. Les données opérationnelles décrivent les offres et les candidats. Les référentiels ESCO, MEPC et NCF apportent les nomenclatures nécessaires pour interpréter les métiers, compétences et niveaux de formation.

Les offres décrivent les postes disponibles : titre, employeur, secteur, localisation, expérience, niveau attendu, compétences et description. Les profils candidats décrivent le diplôme, le niveau, l'expérience, les compétences, la mobilité et l'objectif professionnel. Ces informations sont utiles mais bruitées ; elles doivent donc être normalisées avant l'encodage et la construction du graphe.

Table 2.1 – Sources de données mobilisées et utilisation méthodologique

Données mobilisées	Nature	Utilisation dans la méthode
Offres d'emploi	Données opérationnelles collectées par KmerAI et FNE	Normalisation, extraction de compétences, génération de <code>text_to_embed</code> , indexation vectorielle, création des noeuds d'offres et calcul des scores de matching.
Profils candidats	Base locale de demandeurs	Normalisation du niveau, du diplôme, de la mobilité et du métier visé ; création des embeddings candidats ; analyse de compatibilité avec les offres.
ESCO	Référentiel métiers-compétences	Source de métiers, compétences, groupes ISCO et relations métier-compétence, utilisée pour structurer le graphe et enrichir les correspondances.
MEPC	Nomenclature camerounaise des métiers	Ancrage national des métiers, structuration hiérarchique et alignement partiel avec les groupes internationaux.
NCF	Nomenclature camerounaise des formations	Codification des niveaux et domaines de formation, utilisée pour la compatibilité candidat-offre et l'analyse de montée en compétences.
PDF réglementaires	Documents officiels fragmentés	Constitution de <code>DocChunk</code> pour la recherche référentielle et la citation des sources.

Source : Nos travaux

2.1.3 Préparation et normalisation des données

La normalisation prépare les données pour deux usages : l’encodage vectoriel et la création des relations Neo4j. Elle réduit les doublons, harmonise les champs textuels, sépare les valeurs multiples et transforme les compétences en listes exploitables.

Pour les candidats, le traitement porte surtout sur le diplôme, le niveau d’étude, l’expérience, les compétences, le secteur visé et la mobilité. Lorsque c’est possible, le niveau est rapproché de la NCF afin de ne pas confondre proximité textuelle et compatibilité de qualification.

Chaque offre et chaque candidat reçoit un champ `text_to_embed`. Il regroupe les informations utiles à la recherche sémantique : titre, compétences, secteur, niveau, description et objectif.

2.1.4 Alignement des référentiels métiers et formations

L’alignement rend comparables les annonces, les profils et les référentiels. La MEPC structure les métiers selon ses niveaux hiérarchiques ; la NCF structure les niveaux et domaines de formation ; ESCO apporte un vocabulaire international métier-compétence.

L’alignement repose sur les codes disponibles et les proximités de libellés. Il sert à enrichir la couverture et l’interprétabilité du système, sans prétendre établir une équivalence parfaite entre toutes les nomenclatures.

2.2 Représentation sémantique et graphe de connaissances

Cette section décrit les représentations utilisées par le système : embeddings pour la proximité textuelle, pgvector pour le retrieval, Neo4j pour les relations métier et score hybride pour le classement.

2.2.1 Choix du modèle d’embeddings et fine-tuning

Le modèle SentenceTransformer représente les offres, candidats, métiers, compétences et documents dans un espace vectoriel. Ce choix est adapté à la recherche sémantique, car les embeddings peuvent être comparés par similarité cosinus [36].

Le fine-tuning n’est pas génératif. Il adapte l’espace vectoriel au vocabulaire du domaine par apprentissage contrastif : rapprocher les paires pertinentes et éloigner les paires moins pertinentes. Les paires d’apprentissage sont construites à partir des relations et champs normalisés, puis le modèle adapté est évalué séparément du modèle de base.

2.2.2 Indexation vectorielle dans PostgreSQL et pgvector

Les embeddings sont stockés dans PostgreSQL avec pgvector, afin de combiner vecteurs et métadonnées dans la même base [33]. L'indexation de type HNSW sert à accélérer la recherche top-K [26].

pgvector joue deux rôles : retrouver les objets proches d'une requête et constituer un vivier initial avant le reranking par le graphe. Chaque résultat conserve son identifiant, son type, sa source, son score et le texte associé pour préserver la traçabilité.

2.2.3 Construction du graphe de connaissances sous Neo4j

Neo4j représente la couche relationnelle du système. Les noeuds correspondent aux offres, candidats, compétences, métiers, secteurs, localisations et référentiels. Les relations décrivent les liens utiles au matching, par exemple `REQUIERT` entre offre et compétence ou `POSSEDE` entre candidat et compétence.

Le graphe sert à vérifier les correspondances proposées par pgvector, expliquer les compétences communes ou manquantes et répondre à des questions relationnelles que la recherche vectorielle traite mal.

2.2.4 Score hybride et logique de skill gap

Le score de recommandation combine des signaux complémentaires : similarité sémantique, couverture des compétences, compatibilité de niveau et alignement sectoriel.

La forme générale du score est la suivante :

$$S_{hyb}(c, o) = \alpha S_{sem}(c, o) + \beta S_{graph}(c, o) + \gamma C_{ncf}(c, o) + \delta A_{sect}(c, o),$$

où c désigne un candidat, o une offre, S_{sem} la similarité sémantique, S_{graph} la couverture des compétences, C_{ncf} la compatibilité de niveau et A_{sect} l'alignement sectoriel. Le LLM n'intervient donc pas dans le calcul brut du classement ; il sert à synthétiser et expliquer.

2.2.5 Construction du proxy de pertinence pour l'évaluation

En l'absence d'annotations indépendantes de recruteurs, l'évaluation utilise un label faible nommé `proxy_relevance`. Ce proxy ne fait pas partie du score hybride utilisé par le moteur ; il sert uniquement de référence expérimentale pour comparer les classements.

Il ne s'agit pas d'une formule universelle de pertinence. La littérature sur l'évaluation du retrieval demande de rendre explicites les jugements de pertinence et permet d'utiliser des

gains gradués pour comparer des classements [39, 29] ; elle ne fixe pas les coefficients 0,45, 0,30 et 0,25. Ces coefficients sont donc des choix de conception transparents : le recouvrement lexical reçoit le poids principal car il est observé directement dans les métadonnées normalisées ; l'alignement sectoriel et le niveau NCF jouent un rôle complémentaire de contrainte. Le proxy doit être lu comme un label faible de cohérence interne, non comme une vérité terrain.

Pour un candidat c et une offre o , le proxy combine trois informations issues des métadonnées normalisées :

$$R_{\text{proxy}}(c, o) = 0,45L(c, o) + 0,30S(c, o) + 0,25N(c, o),$$

où $L(c, o)$ mesure le recouvrement lexical, $S(c, o)$ l'alignement sectoriel et $N(c, o)$ la compatibilité de niveau NCF. Le recouvrement lexical est défini par :

$$L(c, o) = \frac{|T_c \cap T_o|}{\max(\min(|T_c|, |T_o|), 1)}.$$

La compatibilité NCF vaut 1 si le niveau du candidat couvre le niveau demandé, 0,70 pour un écart d'un niveau, 0,40 pour deux niveaux et 0,10 au-delà. Si l'un des niveaux est absent, une valeur neutre de 0,45 évite de transformer une donnée manquante en incompatibilité certaine.

Le score continu R_{proxy} est utilisé comme gain gradué dans le calcul du NDCG. Pour Precision@10, Recall@10 et MRR@10, il est transformé en label binaire séparément pour chaque candidat. Une offre est déclarée pertinente lorsque :

$$R_{\text{proxy}}(c, o) \geq \max(0,35, Q_{0,70,c}),$$

où $Q_{0,70,c}$ est le 70^e percentile des scores du vivier du candidat. Le seuil relatif évite qu'un même seuil absolu traite identiquement des viviers dont les scores ont des distributions différentes. Il reste néanmoins un paramètre de conception ; une analyse de sensibilité des poids, des paliers NCF et du percentile doit précéder toute conclusion métier. Ce protocole est reproductible, mais il mesure une cohérence interne avec les métadonnées disponibles, pas une validation métier définitive.

Les pondérations sont calibrées avec Optuna, par optimisation bayésienne et *Tree-structured Parzen Estimator* [3]. Dans l'expérience retenue, la calibration porte sur deux signaux : score sémantique pgvector et score Neo4j.

$$S_{cal}(c, o) = w_{sem} S_{sem}(c, o) + w_{Neo4j} S_{Neo4j}(c, o), \quad w_{sem} + w_{Neo4j} = 1.$$

À chaque essai, Optuna propose w_{sem} , déduit $w_{Neo4j} = 1 - w_{sem}$, recalcule les scores, re-range les offres et évalue le classement. Trois objectifs sont comparés : NDCG@10, moyenne NDCG@10–Recall@10–Precision@10, et moyenne MRR@10–Precision@10. Les poids obtenus sont donc optimaux dans ce protocole expérimental, non des paramètres définitifs du marché du travail. L’analyse de *skill gap* complète le score en distinguant compétences acquises, requises et manquantes. Elle transforme le classement en verdict opérationnel : compatibilité immédiate, compatibilité après montée en compétence, vivier ou profil hors cible.

2.3 Orchestration agentique et génération contrôlée

Cette section présente le rôle du workflow agentique dans la méthode. Les détails d’implémentation sont développés au chapitre 3.

2.3.1 Architecture Agentic RAG et orchestration LangGraph

LangGraph représente l’agent comme un graphe d’états plutôt que comme une succession libre de prompts [23]. La requête est analysée, classée par cas d’usage, puis dirigée vers les outils adaptés : pgvector, Neo4j, recherche documentaire, score hybride ou diagnostic.

Ce choix relève de l’Agentic RAG : l’agent orchestre des outils spécialisés autour d’un état partagé [45, 4]. Dans ce mémoire, l’agent reste borné : il route, trace et contrôle ; il ne remplace pas les bases ni les règles métier.

2.3.2 Text2Cypher et rôle du LLM

Deux usages du LLM sont distingués. Text2Cypher transforme certaines questions relationnelles en requêtes Cypher, sous contrainte du schéma Neo4j. La génération finale formule ensuite une réponse française à partir du contexte récupéré. Cette séparation est importante : le LLM ne décide pas seul de la pertinence.

2.3.3 Modes d’accès et traçabilité des décisions

Trois modes d’accès sont prévus : une interface en ligne de commande pour les tests techniques, une démonstration conversationnelle avec Streamlit et une interface de programmation FastAPI pour l’intégration applicative. Ces interfaces exposent le même moteur. La traçabilité repose sur les traces du workflow, les identifiants des entités, les sources documentaires, les scores et les chemins du graphe. Une recommandation sans preuve n’est pas défendable dans

un contexte d'orientation ou de recrutement.

2.3.4 Protocole d'évaluation prévu du système

L'évaluation prévue sépare quatre niveaux : retrieval vectoriel, validation fonctionnelle du graphe, comparaison des classements et contrôle de la génération. Les métriques de ranking retenues incluent Precision@K, Recall@K, MRR@K, MAP@K et NDCG@K [39, 29]. Cette séparation évite une confusion classique dans les systèmes RAG : une réponse bien formulée ne prouve pas que les bonnes informations ont été retrouvées ; inversement, un bon retrieval ne garantit pas une bonne synthèse.

Conclusion

Ce chapitre a transformé la problématique en spécification de pipeline. Les données d'offres, de candidats et de référentiels sont d'abord normalisées, puis représentées dans deux mémoires complémentaires : pgvector pour le rappel sémantique et Neo4j pour les relations métier-compétence-formation. Le score hybride, la mesure du *skill gap* et l'orchestration agentique structurent ensuite la production de recommandations traçables. La méthodologie retient aussi une règle de prudence : les métriques de ranking et le critic évaluent la cohérence interne du prototype, mais ne remplacent pas une validation humaine métier. Le chapitre suivant documente l'opérationnalisation pilote de cette architecture et les choix techniques réalisés sous les contraintes de données et de ressources disponibles.

Deuxième partie

PRÉSENTATION DES RÉSULTATS ET DISCUSSION

CONSTRUCTION OPÉRATIONNELLE DU SYSTÈME HYBRIDE EMPLOI-COMPÉTENCES

Ce chapitre suit le workflow réellement implémenté : préparation des données, adaptation de l’encodeur, indexation dans pgvector, construction du graphe Neo4j et calcul du score hybride. Seuls les résultats nécessaires pour justifier ces choix techniques sont présentés ; leur évaluation comparative est réservée au chapitre 4.

3.1 Données exploitables et contraintes de qualité du système

3.1.1 Volumes disponibles après nettoyage

L’ETL produit les offres, profils et référentiels normalisés qui alimentent l’encodeur, pgvector et Neo4j. Le corpus compte 13 957 offres pour 41 298 candidats, soit environ trois profils pour une offre. Ce ratio ne prouve pas mécaniquement un biais : le risque apparaît lorsqu’un échantillonnage de couples candidat-offre ou un voisinage vectoriel expose davantage les secteurs, intitulés ou compétences les plus fréquents. Ces signaux de popularité peuvent alors masquer des différences utiles entre profils. Le rappel sémantique est donc suivi d’un contrôle relationnel qui vérifie les compétences requises plutôt que la seule fréquence ; les volumes complets figurent au tableau A.5 en annexe.

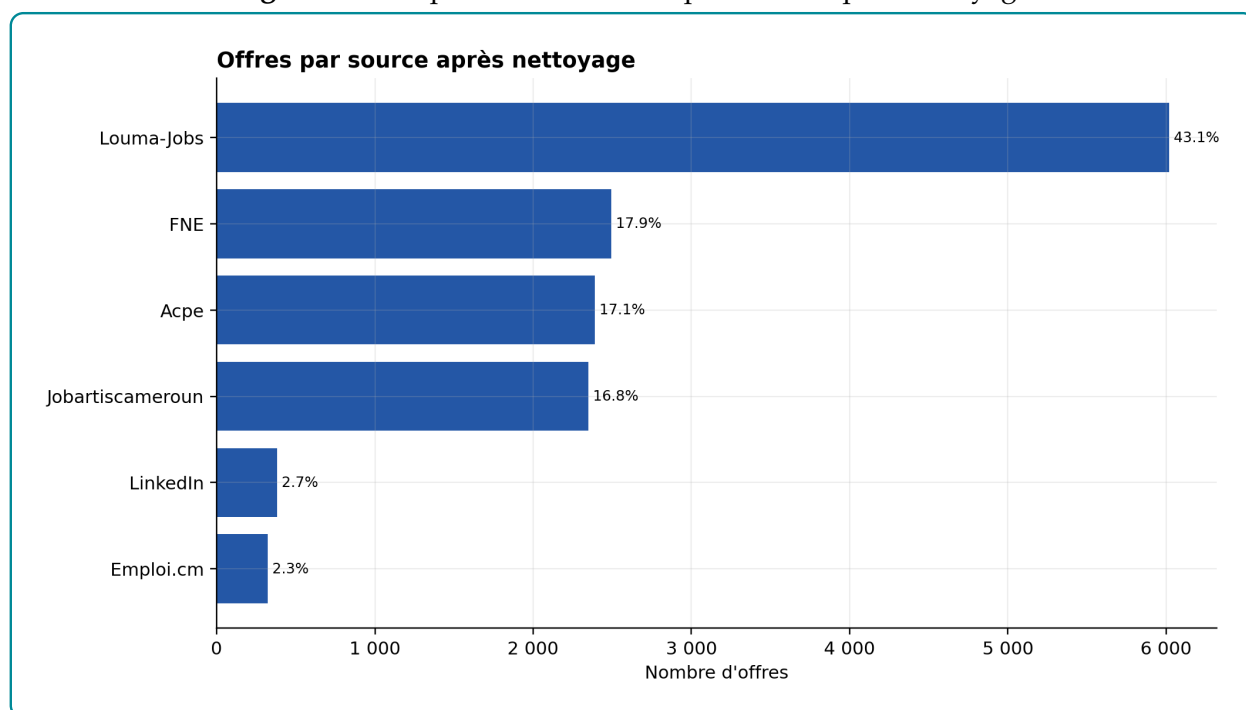
Les principales opérations appliquées sont :

- harmonisation de la casse, des espaces, des accents et des formats de texte ;
- suppression prudente des doublons strictement identiques ;
- normalisation des localisations, secteurs, niveaux et types de contrat ;
- conservation explicite des valeurs manquantes, sans inventer d’information ;
- composition d’un texte exploitable à partir du titre, des compétences et des métadonnées disponibles lorsque la description est insuffisante.

Les offres proviennent de plateformes scrapées et de sources institutionnelles. Louma-Jobs représente 43,14 % du corpus : les résultats peuvent refléter les caractéristiques de cette source

et ne décrivent pas exhaustivement le marché camerounais de l'emploi.

Figure 3.1 – Répartition des offres par source après nettoyage



Source : Nos travaux, traitements Python de nettoyage et de diagnostic

3.1.2 Qualité des variables utiles au matching emploi-compétences

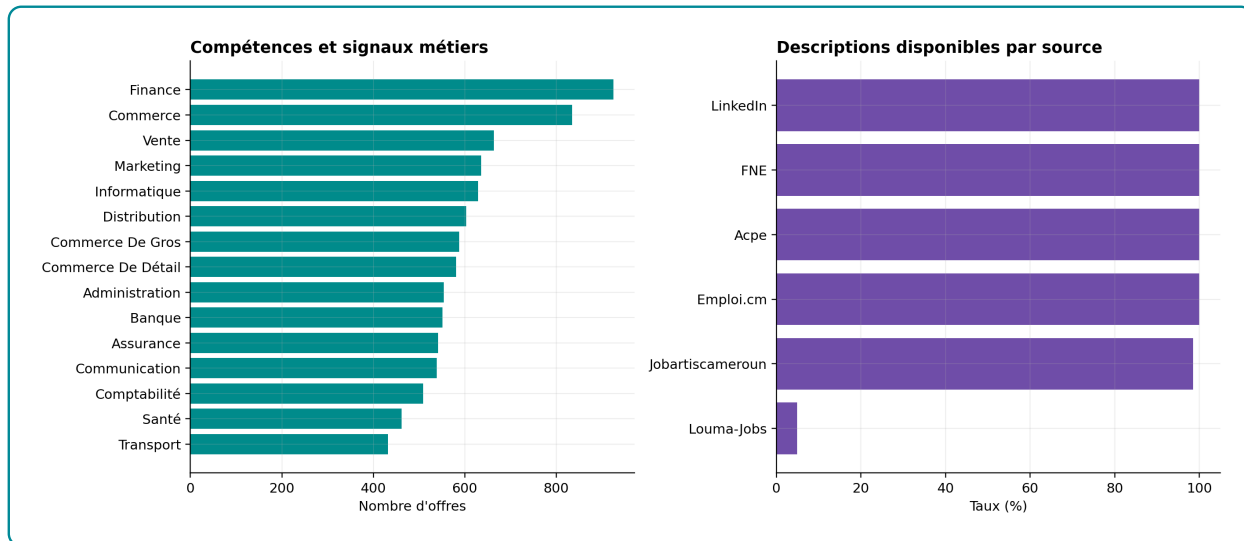
Le matching exploite la description, les compétences, la localisation, le secteur et le niveau. La source reste une variable de traçabilité et permet d'interpréter les écarts de qualité entre plateformes.

3.1.2.1 Localisation exploitable pour filtrer sans exclure

La localisation est utilisée comme filtre souple : 79,88 % des offres sont rattachées au Cameroun et 20,12 % à l'international. Une ville absente n'est pas imputée ; le pays disponible est conservé. Le diagnostic détaillé figure en annexe (figure A.3).

3.1.2.2 Compétences et descriptions : signal utile mais hétérogène

Figure 3.2 – Compétences fréquentes et disponibilité descriptive



Source : Nos travaux, traitements Python de nettoyage et de diagnostic.

Les libellés fréquents mélangent compétences observables et domaines d'activité ; ils sont utilisés comme indices de rapprochement, non comme preuve de maîtrise. Louma-Jobs ne fournit une description que pour 4,90 % de ses offres. Pour éviter qu'une description absente produise un vecteur pauvre, le texte encodé combine titre, compétences, secteur, localisation et autres métadonnées disponibles.

3.1.3 Nettoyage réalisé et limites conservées

Le nettoyage harmonise les formats sans inventer les valeurs absentes. Les localisations suivent la priorité ville-pays et les doublons potentiels sont audités sans suppression automatique, car deux annonces proches peuvent correspondre à des publications distinctes. Les règles détaillées figurent au tableau A.7 en annexe.

Le pipeline peut traiter de nouvelles offres et de nouveaux profils dès lors que les colonnes minimales sont respectées. Une nouvelle exécution applique les mêmes contrôles, régénère les fichiers normalisés, puis permet de réencoder et synchroniser les nouvelles entités dans les bases de données. Il s'agit d'une actualisation par lots contrôlée, non d'une ingestion en temps réel ; un changement du format d'une source scrapée exige d'adapter son mapping.

3.2 Adaptation du modèle d'embeddings au domaine emploi-compétences

Après l'ETL, les textes nettoyés servent à adapter l'encodeur avant leur indexation dans pgvector. L'objectif est d'améliorer le classement sémantique, non de générer une réponse.

La logique de cette section se résume en quatre étapes. Un encodeur compact est d'abord retenu pour représenter les textes ; 6 476 paires offre-description servent ensuite à l'apprentissage contrastif par *MultipleNegativesRankingLoss* ; le checkpoint est choisi à partir des métriques de validation ; enfin, les embeddings du modèle retenu alimentent l'index vectoriel. Les courbes, le benchmark et la projection qui suivent documentent chacune une étape de cette chaîne. Elles constituent des diagnostics du prototype, non une preuve directe de pertinence métier.

3.2.1 Encodeur retenu pour les paires contrastives

Table 3.1 – Protocole de construction des paires de fine-tuning

Élément	Implémentation observée
Modèle de départ	sentence-transformers/all-MiniLM-L6-v2, embeddings denses de 384d.
Rôle de sentence1	titre, secteur, contrat, niveau d'études, expérience, ville et compétences.
Rôle de sentence2	Détails nettoyés de l'annonce uniquement, c'est-à-dire le texte descriptif que le modèle doit rapprocher de la représentation structurée.
Filtrage informatif	Conservation des paires avec compétences disponibles, sentence1 d'au moins 40 caractères et sentence2 d'au moins 120 caractères ; le champ <code>ft_eligible</code> est appliqué lorsqu'il existe.
Total de paires	6 476 paires offre-description.

Source : Nos travaux, traitements Python et SentenceTransformer.

Le modèle all-MiniLM-L6-v2 a été retenu pour ses vecteurs de 384 dimensions, moins coûteux à encoder, stocker et rechercher que des représentations plus larges. Son fine-tuning adapte cet espace généraliste aux intitulés, secteurs, niveaux et compétences du corpus local sans modifier l'architecture de production.

sentence1 contient la représentation structurée de l'offre et sentence2 sa description. Les seuils de 40 et 120 caractères excluent les paires trop pauvres pour fournir un signal d'apprentissage exploitable.

3.2.2 Fine-tuning : contraintes de calcul et preuve de gain

3.2.2.1 Paramétrage contraint et dynamique d'apprentissage

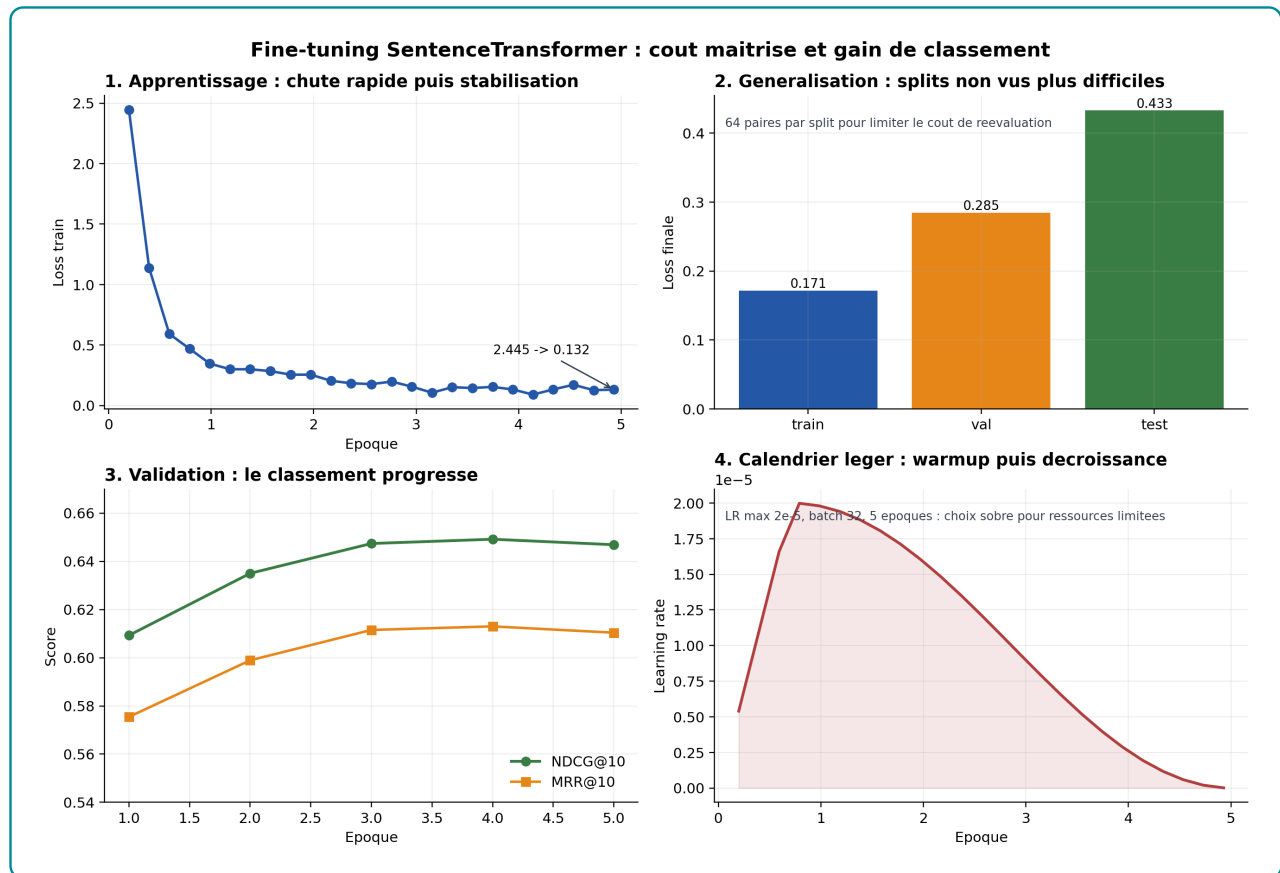
Table 3.2 – Hyperparamètres du fine-tuning SentenceTransformer

Hyperparamètre	Valeur	Justification
Nombre maximal d'époques	5	Plafond court retenu sous contrainte CPU pour observer la stabilisation sans prolonger inutilement l'entraînement. Le meilleur checkpoint apparaît autour de l'époque 4 ; la cinquième époque confirme le léger recul.
Taille de lot	32	Compromis entre mémoire disponible, temps de calcul et diversité des exemples comparés dans chaque lot.
Négatifs implicites	31 par exemple	Avec un lot de 32 paires, les 31 autres descriptions servent de contre-exemples sans créer un fichier supplémentaire de négatifs.
Taux d'apprentissage maximal	2×10^{-5}	Valeur prudente pour adapter le modèle préentraîné sans modifier brutalement ses représentations initiales.
Calendrier du taux	Cosinus avec warmup	Augmentation progressive au démarrage, puis diminution contrôlée afin de stabiliser la fin de l'entraînement.
Warmup	100 étapes au maximum	Limite les mises à jour instables au début ; le nombre effectif reste borné par les étapes disponibles.
Fonction de perte	<i>MultipleNegat</i> <i>RankingLoss</i>	Adaptée au retrieval : elle rapproche chaque paire correcte et éloigne les autres éléments du lot selon la similarité cosinus.

Source : Nos travaux

Ces choix répondent aux ressources disponibles. L'entraînement a duré 15 160 secondes sur CPU. Cinq époques ne signifient pas que la cinquième est la meilleure : elles constituent une borne d'observation, tandis que le checkpoint retenu est celui qui obtient la meilleure validation autour de l'époque 4.

La perte d'entraînement diminue de 2,445 à 0,132, tandis que la perte de test reste plus élevée (0,433) que celle d'entraînement (0,171). Cet écart signale une généralisation moins bonne sur les paires non vues, sans divergence du modèle. Le panneau des métriques montre un NDCG@10 maximal à l'époque 4, puis un léger recul ; le panneau du learning rate confirme une montée puis une décroissance contrôlée du pas d'apprentissage. Prolonger l'entraînement n'était donc pas justifié.

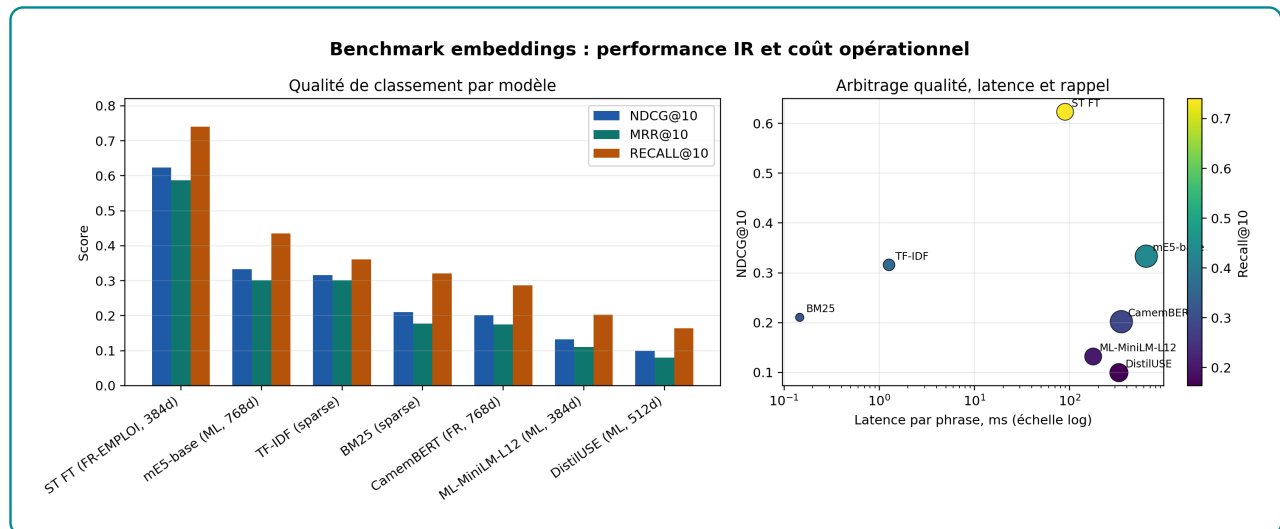
Figure 3.3 – perte, généralisation, métriques de validation et calendrier du learning rate

Source : Nos travaux, traitements Python et SentenceTransformer.

3.2.2.2 Preuve de gain sur le ranking sémantique

Sur 953 paires de test, le fine-tuning multiplie le NDCG@10 par 3,7 et le MRR@10 par 4,2 par rapport au modèle de départ. La notation @10 signifie que le calcul examine les dix premiers résultats retournés pour chaque requête. Le NDCG vérifie leur ordre, tandis que le MRR dépend de la position du premier résultat pertinent. Le détail figure au tableau A.6 en annexe ; l'évaluation complète est présentée au chapitre 4.

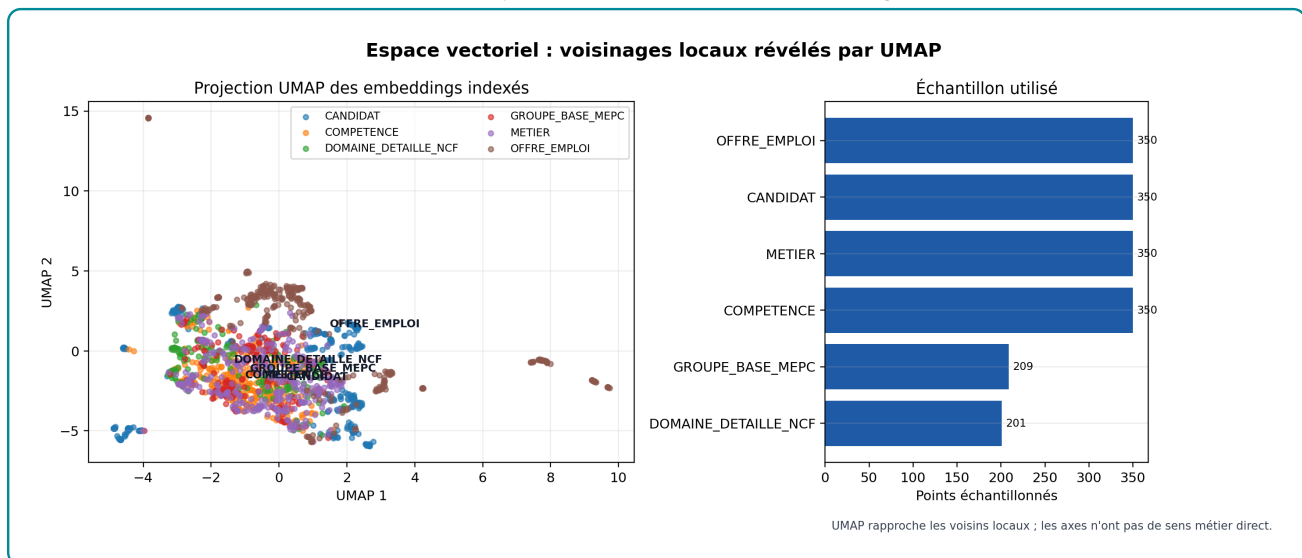
Le modèle fine-tuné obtient un NDCG@10 de 0,6232, un MRR@10 de 0,5874 et un Recall@10 de 0,7398. Les barres de qualité de classement et de rappel justifient son choix pour pgvector ; la latence rappelle toutefois que le meilleur modèle statistique doit rester compatible avec une recherche interactive. TF-IDF reste un complément rapide pour les correspondances lexicales.

Figure 3.4 – Comparaison des modèles d'embeddings : qualité de classement, rappel et latence

Source : Nos travaux, traitements Python et SentenceTransformer

3.2.3 Structure de l'espace vectoriel après fine-tuning

La projection porte sur 1 810 embeddings : 350 offres, 350 candidats, 350 métiers, 350 compétences, 209 groupes MEPC et 201 domaines NCF. Le noyau central mélange plusieurs catégories. Cette absence de séparation stricte est utile au matching : le modèle organise principalement les textes selon leur proximité de sens plutôt que selon leur type administratif. Un candidat peut ainsi se retrouver près d'une offre, d'un métier ou d'une compétence décrivant le même domaine professionnel.

Figure 3.5 – Projection UMAP des embeddings indexés

Source : Nos travaux, traitements Python et SentenceTransformer

Les offres sont toutefois plus dispersées et forment plusieurs prolongements autour du noyau. Leurs textes combinent davantage d'informations – intitulé, missions, contraintes, secteur et localisation – ce qui produit des représentations plus variées que les libellés généralement courts des métiers ou des compétences. Les petits groupes éloignés peuvent correspondre à des domaines spécialisés, à des formulations rares ou à des descriptions peu renseignées. Ils doivent être contrôlés, mais leur éloignement ne prouve pas à lui seul une erreur d'encodage.

La figure montre donc que l'espace permet des voisinages entre objets de nature différente, condition nécessaire au retrieval emploi-compétences. Elle ne démontre cependant pas que chaque voisin est une bonne recommandation. UMAP réduit les vecteurs de 384 dimensions à deux dimensions et privilégie les voisinages locaux ; les axes UMAP 1 et UMAP 2 n'ont pas de signification métier directe. La qualité finale est vérifiée par les métriques de classement, puis par les relations Neo4j utilisées pour confirmer ou corriger la proximité sémantique. Une projection t-SNE pourrait compléter ce diagnostic : elle met souvent davantage en évidence les groupes locaux, mais dépend fortement de la perplexité, du taux d'apprentissage, du nombre d'itérations et de la graine aléatoire. UMAP et t-SNE doivent donc être lus comme des visualisations exploratoires convergentes, jamais comme une mesure de performance ou une preuve de séparation métier.

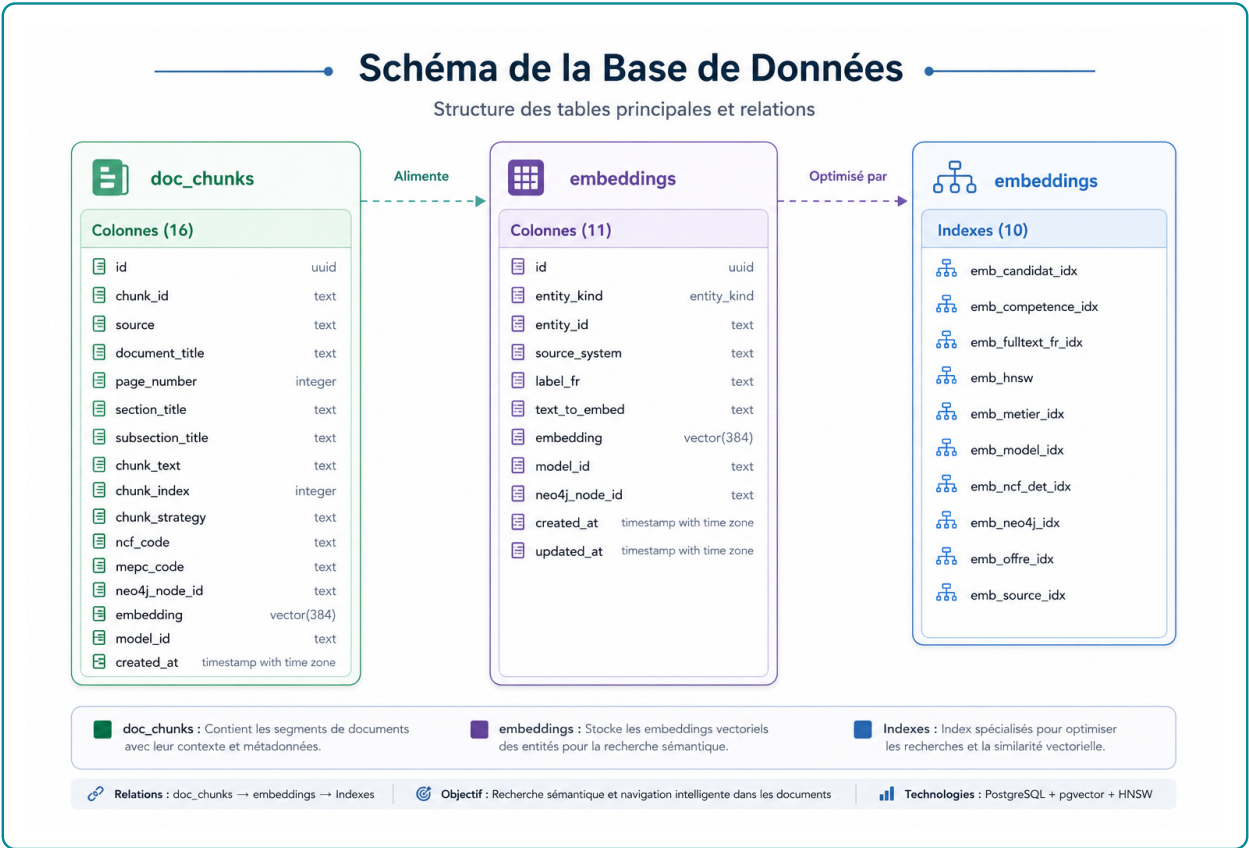
3.3 Mémoire vectorielle pgvector pour le retrieval

Après l'encodage, pgvector récupère un top- K d'objets sémantiquement proches. Ce vivier est ensuite contrôlé et rerangé par Neo4j.

3.3.1 Schéma physique, tables et volumes indexés

La table `embeddings` stocke les entités structurées ; `doc_chunks` contient les fragments des référentiels. Le schéma montre aussi le rôle des index : accélérer la similarité vectorielle, la recherche textuelle et les filtres par type d'entité. L'identifiant commun assure ensuite le passage vers Neo4j.

Figure 3.6 – Schéma physique de la base pgvector



Source : Nos travaux, diagnostics PostgreSQL-pgvector

L'instance contient 72 643 embeddings d'entités et 142 embeddings documentaires, tous en 384 dimensions. Les identifiants communs permettent de replacer les résultats vectoriels dans le graphe afin de récupérer leurs relations métier. Cette continuité est un lien technique ; elle ne constitue pas, à elle seule, une preuve de qualité sémantique.

Table 3.3 – Distribution des embeddings par type d'entité

Type d'entité	Vecteurs	Part
Candidats	41 298	56,85 %
Offres d'emploi	13 957	19,22 %
Compétences	13 939	19,19 %
Métiers	3 039	4,18 %
Groupes de base MEPC	209	0,29 %
Domaines détaillés NCF	201	0,28 %
Total entités structurées	72 643	100 %
Chunks documentaires	142	–

Source : Nos travaux, diagnostics PostgreSQL-pgvector.

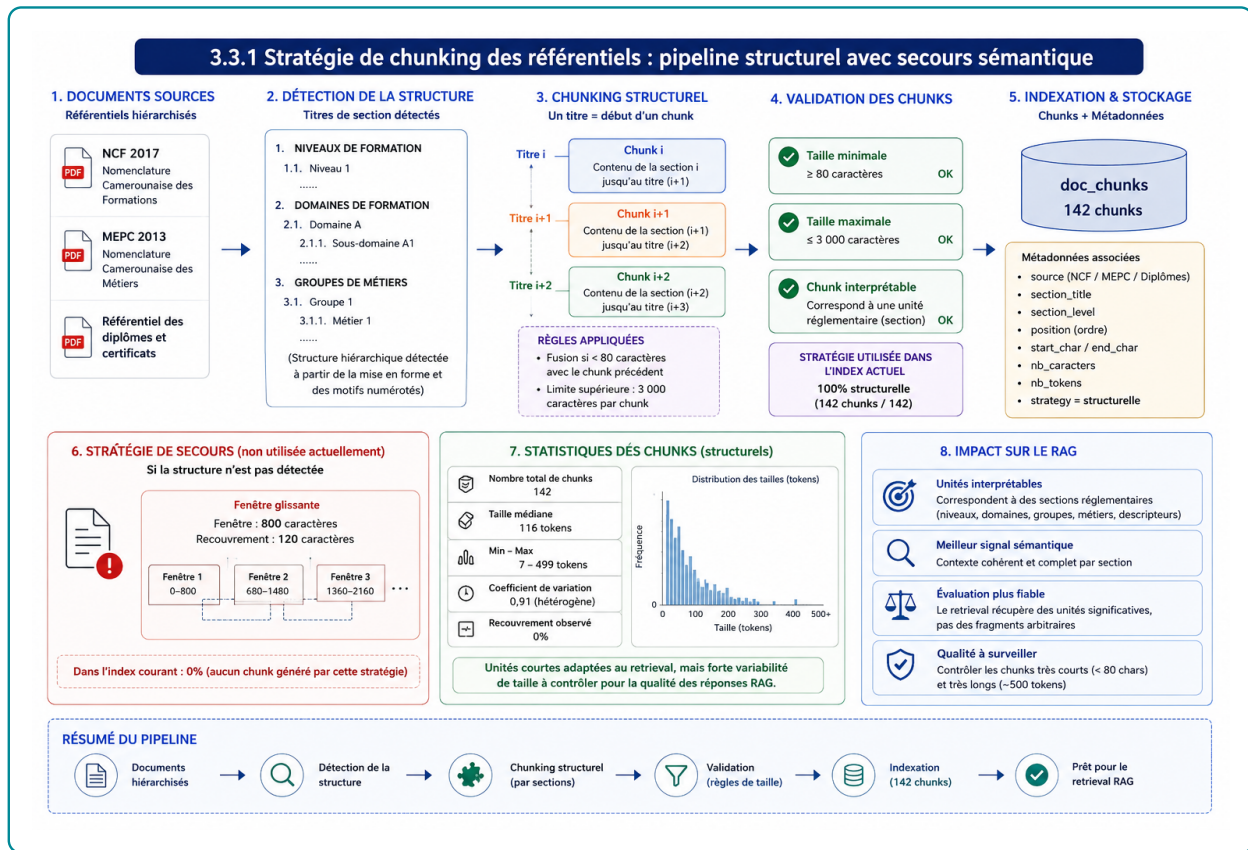
3.3.2 Référentiels découpés en chunks interrogeables

Les référentiels produisent 142 chunks : 43 pour les diplômes, 61 pour le MEPC 2013 et 38 pour la NCF 2017. Le traitement suit cinq étapes :

- identifier le document source et son référentiel ;
- détecter les titres et la hiérarchie des sections ;
- créer un chunk par unité documentaire cohérente ;
- vérifier la taille, puis fusionner les fragments de moins de 80 caractères et limiter les chunks à 3 000 caractères ;
- stocker le texte et ses métadonnées dans pgvector pour la recherche.

La stratégie de secours par fenêtres glissantes n'a pas été utilisée dans l'index courant. La figure 3.7 doit être lue comme un contrôle de qualité du contexte injecté dans le RAG. Le découpage ne part pas d'une fenêtre arbitraire, mais de la structure réelle des référentiels : niveaux, domaines, groupes de métiers, diplômes et sections. Cette stratégie limite deux risques. D'abord, elle évite de couper une règle au milieu d'une phrase ou d'une section. Ensuite, elle évite de mélanger dans un même fragment plusieurs règles qui ne répondent pas à la même question.

Figure 3.7 – Pipeline de chunking structurel des référentiels documentaires



Source : Nos travaux, schématisation du pipeline de chunking documentaire et d'indexation pgvector.

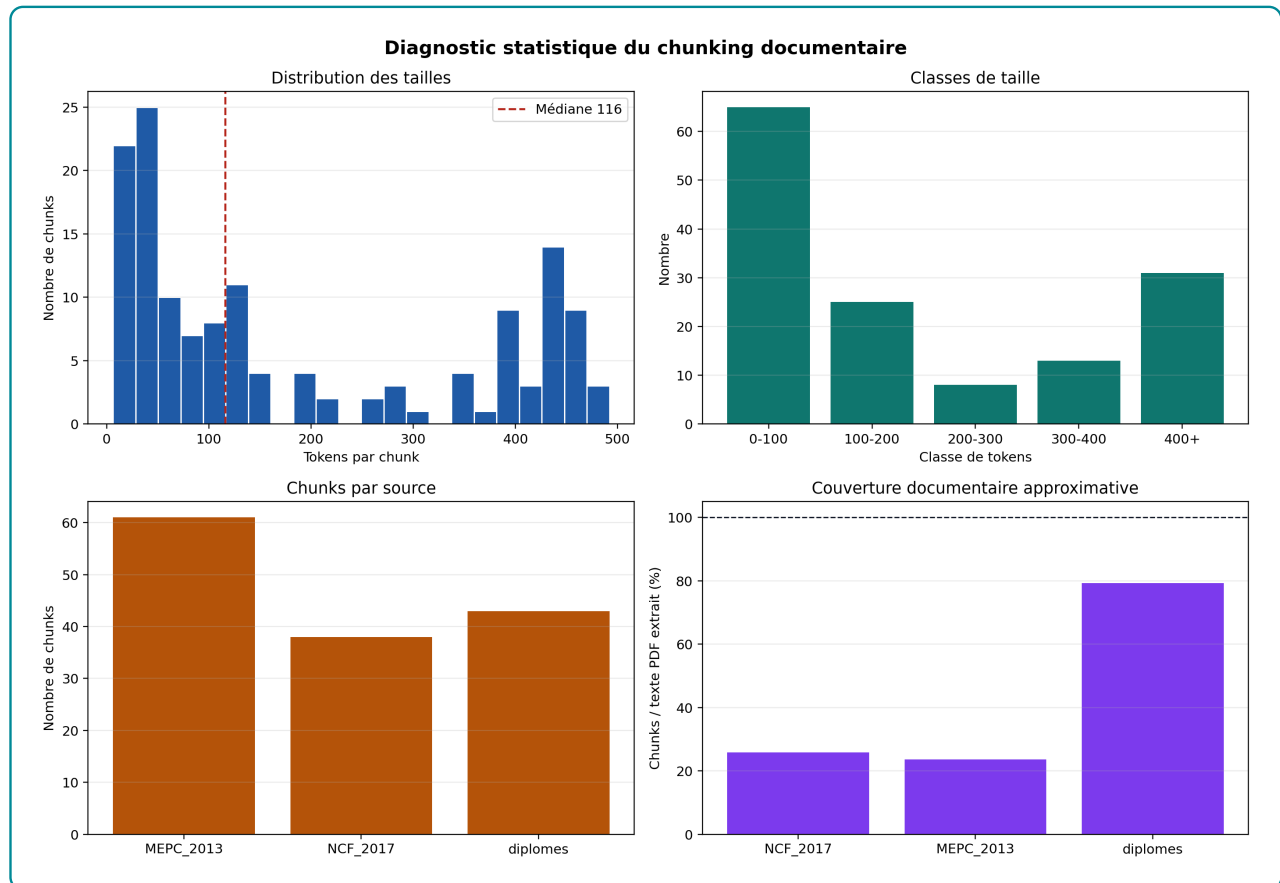
Concrètement, un chunk correspond à une unité interprétable : une partie de nomenclature, un niveau de formation, un groupe métier ou une description réglementaire. Lorsqu'une question porte sur un métier, une compétence ou un niveau NCF, pgvector peut donc récupérer un passage qui a encore un sens seul. C'est ce qui rend le contexte utilisable par l'agent : le LLM ne reçoit pas seulement du texte proche, mais un fragment rattaché à une source, une section et une position documentaire.

La figure 3.8 complète cette lecture par trois diagnostics. Premièrement, la taille médiane de 116 tokens indique que les fragments sont assez courts pour être retrouvés et cités sans saturer le contexte. Deuxièmement, la présence de quelques chunks supérieurs à 400 tokens montre que certaines sections des référentiels restent denses ; ces fragments sont acceptables pour préserver le sens, mais ils doivent être surveillés car ils peuvent contenir plusieurs informations. Troisièmement, la couverture moyenne de 42,9 % renseigne la part du texte utile conservée après nettoyage et structuration ; elle ne doit pas être interprétée comme une mesure directe de qualité, mais comme un indicateur de densité documentaire.

La comparaison par source montre aussi que les référentiels n'ont pas la même régula-

rité. La NCF présente une cohérence moyenne plus élevée (0,336) que les diplômes (0,268), ce qui signifie que ses sections sont plus homogènes après découpage. Les diplômes demandent donc davantage de vigilance : leurs libellés peuvent être courts, hétérogènes ou très administratifs. Pour le système, cela implique que le retrieval documentaire est plus fiable lorsqu'il interroge des sections bien structurées, et plus fragile lorsqu'il dépend de fragments courts ou descriptivement pauvres. Les mesures complètes figurent en annexe.

Figure 3.8 – Distribution des tailles, sources et couverture du chunking documentaire



Source : Nos travaux, diagnostics Python du chunking documentaire.

3.4 Neo4j : mémoire relationnelle du matching

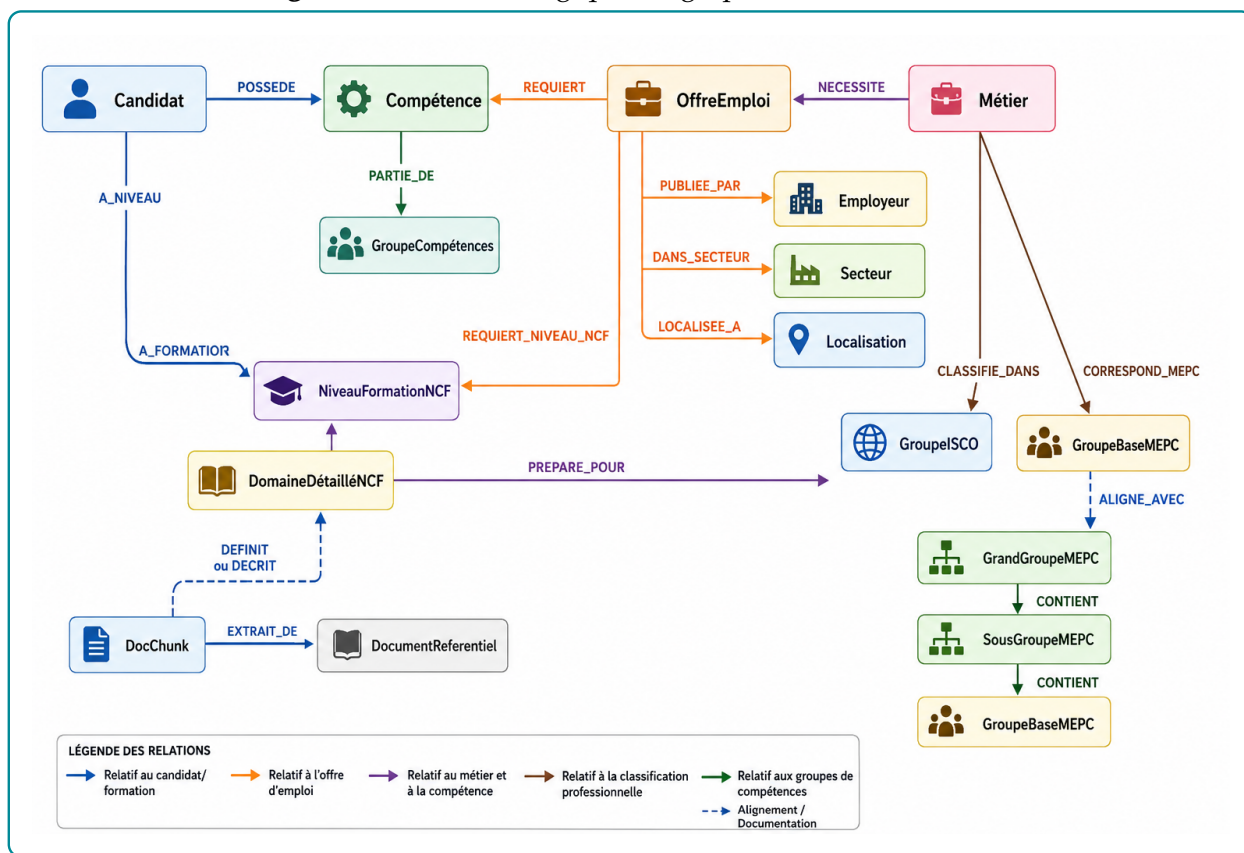
Dans le workflow méthodologique, Neo4j intervient après le rappel pgvector et avant le score hybride. Pgvector propose des objets proches dans l'espace sémantique ; Neo4j vérifie ensuite si cette proximité repose sur des relations métier exploitables : compétences possédées, compétences requises, métiers, niveaux NCF, secteurs et localisations. Cette section présente donc la structure du graphe et les précautions nécessaires avant son usage dans le reranking.

3.4.1 Schéma, noeuds et relations mobilisés par le matching

Cette sous-section décrit la partie du graphe réellement utile au matching. L'objectif n'est pas de présenter tous les objets stockés, mais de montrer comment une offre, un candidat et des référentiels deviennent interrogeables par relations.

La figure 3.9 montre deux fonctions du graphe : relier les candidats aux offres par les compétences et rattacher ces objets aux référentiels métier et formation. pgvector repère une proximité textuelle ; Neo4j indique par quelles relations cette proximité peut être expliquée. Le chemin principal est $\text{Candidat} \rightarrow \text{POSSEDE} \rightarrow \text{Compétence} \leftarrow \text{REQUIERT} \leftarrow \text{OffreEmploi}$. Il permet de distinguer les compétences couvertes des compétences manquantes. Les liens vers le métier, le niveau NCF, le secteur et la localisation complètent l'explication. Leur fiabilité reste conditionnée par la qualité de l'extraction et de la normalisation.

Figure 3.9 – Schéma logique du graphe de connaissances



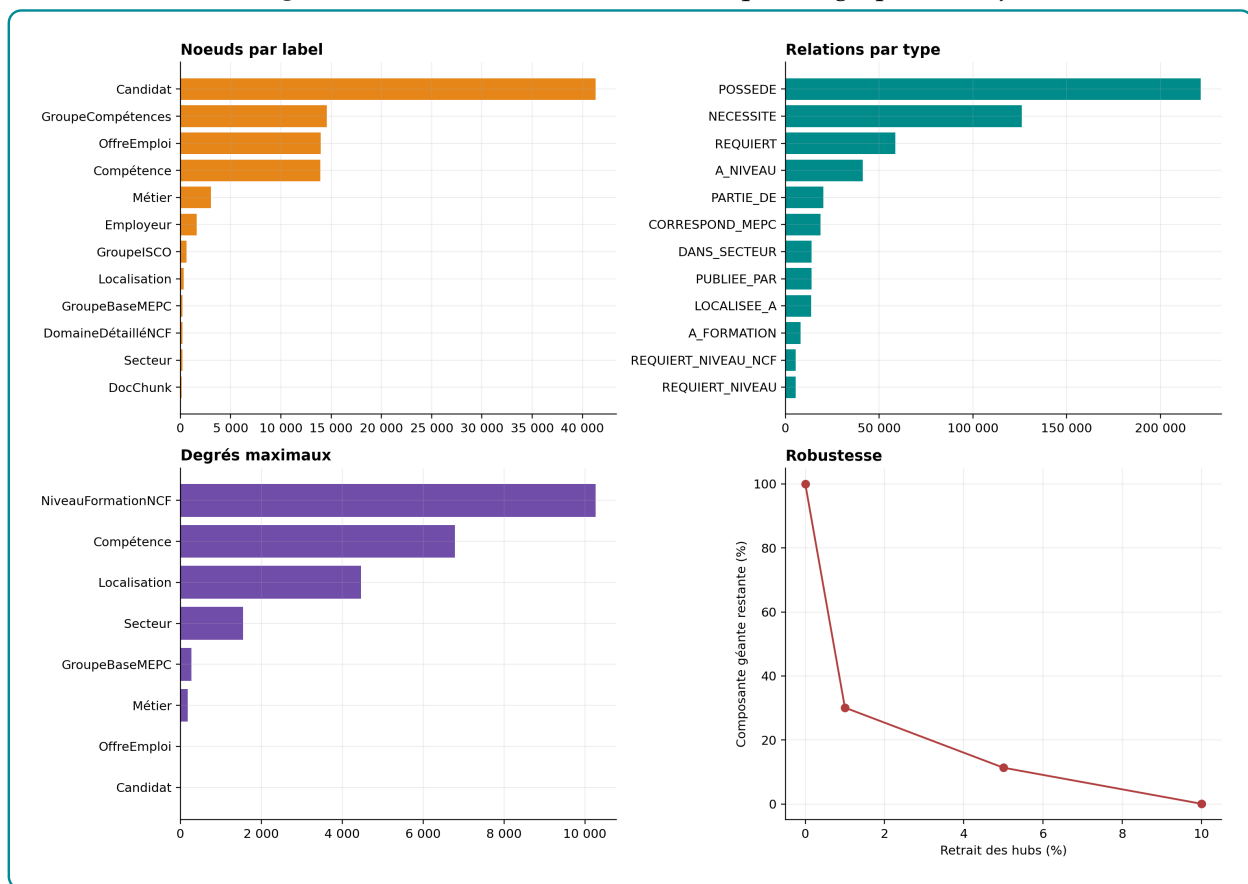
Source : Nos travaux, diagnostics Neo4j

3.4.2 Structure du graphe et dépendance aux hubs

Cette sous-section contrôle si le graphe est exploitable pour le GraphRAG. Dans le workflow, ce diagnostic précède le score hybride : si le graphe est trop fragmenté, Neo4j ne peut

pas expliquer les recommandations ; s'il dépend trop de quelques noeuds génériques, il peut au contraire produire des correspondances trop larges. La figure 3.10 regroupe quatre diagnostics complémentaires. Le panneau des labels montre que le graphe est dominé par les candidats (41 298 noeuds), puis par les groupes de compétences, les offres (13 957) et les compétences (13 939). Cette composition est cohérente avec le workflow : relier des profils nombreux à un vivier d'offres à travers des compétences normalisées. Le deuxième panneau, « Relations par type », indique que les relations POSSEDE dominent avec 221 413 liens, suivies par NECESSITE (126 051) et REQUIERT (58 519). Le graphe est donc bien centré sur la chaîne candidat-compétence-offre, avec un enrichissement par les métiers de référence. C'est utile pour expliquer le *skill gap*, mais cela rend aussi le système sensible à la qualité d'extraction des compétences : une compétence mal extraite crée directement un lien erroné.

Figure 3.10 – Tableau de bord statistique du graphe Neo4j



Source : Nos travaux, diagnostics Neo4j

Le troisième panneau, « Degrés maximaux », révèle la présence de hubs. Les niveaux NCF atteignent 10 261 connexions et certaines compétences 6 787, alors que le degré médian d'une compétence est de 4. Ces hubs sont utiles pour connecter le graphe, mais ils sont peu discri-

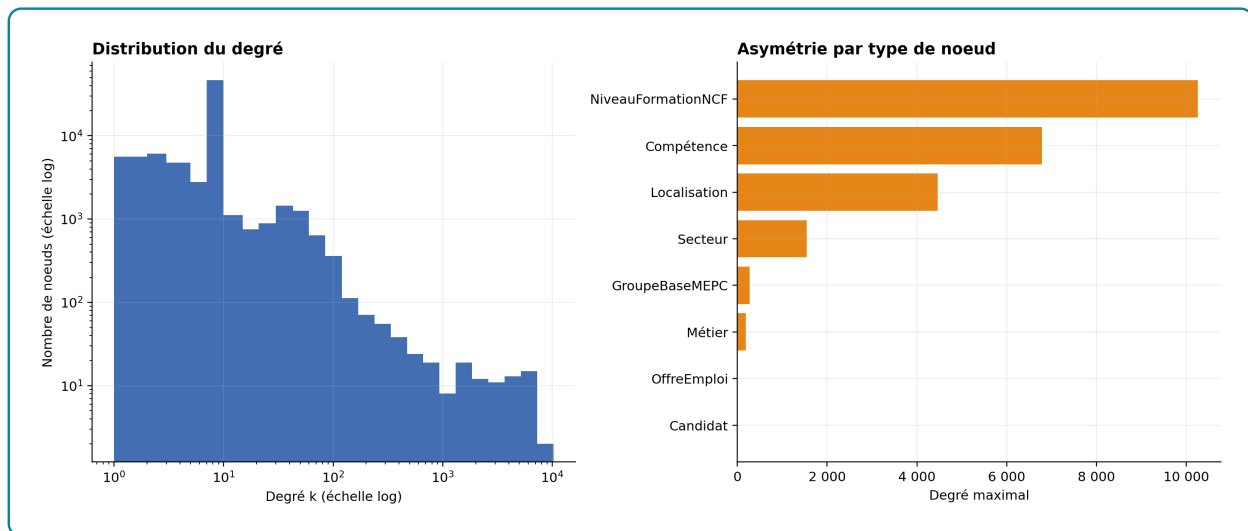
minants pour recommander une offre précise. Ils ne doivent donc pas être interprétés comme des preuves fortes de compatibilité.

Le quatrième panneau, « Robustesse », teste la dépendance du graphe à ces hubs. Lorsque les 1 % de noeuds les plus connectés sont retirés, la composante géante ne conserve plus que 30,1 % de sa taille initiale ; après retrait des 5 %, elle tombe à 11,3 % ; après 10 %, elle s'effondre presque totalement. Le résultat est clair : la connectivité du graphe repose fortement sur quelques noeuds génériques. Cette faiblesse ne bloque pas l'usage de Neo4j, mais elle impose de ne pas utiliser le degré brut comme score de pertinence.

Le graphe contient au total 90 230 noeuds et 550 542 relations. Sa faible densité ($6,76 \times 10^{-5}$) n'indique donc pas un manque d'information ; elle traduit un schéma spécialisé où une offre est reliée aux entités qui l'expliquent, et non à toutes les entités disponibles. La projection de matching regroupe 72 516 noeuds et 499 080 arêtes dans une composante connexe : les chemins candidat-compétence-offre existent, mais leur pertinence dépend de la qualité des relations POSSEDE et REQUIERT. La composition détaillée est reportée en annexe.

La distribution est asymétrique : les niveaux NCF atteignent 10 261 connexions et certaines compétences 6 787, alors que le degré médian d'une compétence est de 4. Quelques libellés généraux concentrent donc une grande partie des chemins et sont peu discriminants.

Figure 3.11 – Distribution des degrés dans la projection matching



Source : Nos travaux, diagnostics Neo4j sur la projection des relations utilisées pour le matching.

Le score Neo4j utilise donc la proportion de compétences requises couvertes, et non le degré brut des noeuds. Une compétence fréquente n'améliore pas automatiquement une recommandation ; elle ne compte que si elle est exigée par l'offre et associée au candidat. Cette règle protège le reranking contre le bruit de popularité et prépare la section suivante, où Neo4j

devient un signal pondéré du score hybride.

3.5 Score hybride et reranking des recommandations

Le moteur combine le rappel sémantique de pgvector et la couverture de compétences calculée dans Neo4j. Il réordonne les offres avant leur transmission au module de génération.

3.5.1 Formalisation du score hybride

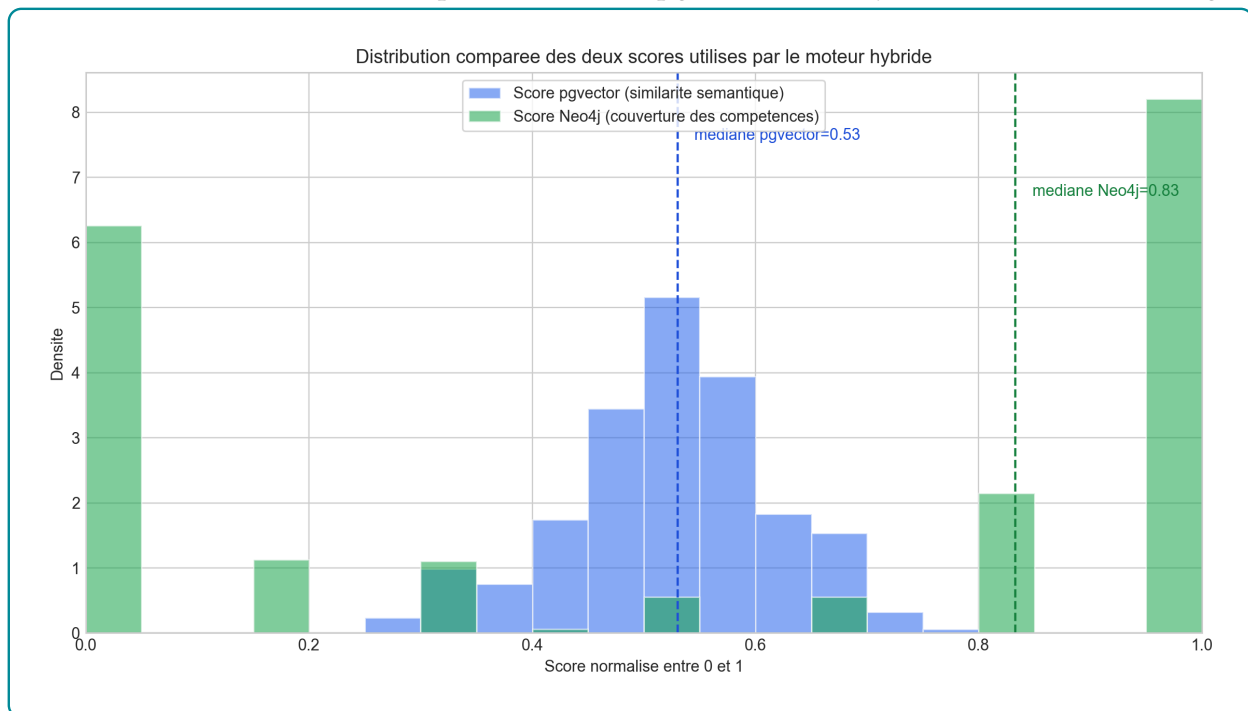
Dans la version calibrée, le score combine deux signaux :

$$S_{\text{hybride}} = \alpha S_{\text{pgvector}} + \beta S_{\text{Neo4j}}, \quad \alpha + \beta = 1$$

où `score_sem` mesure la similarité pgvector et `taux_match` la part des compétences requises couvertes dans Neo4j. Le niveau NCF, le secteur et la localisation restent disponibles pour l'explication, mais ne reçoivent pas de poids autonome dans cette calibration.

Sur 690 couples candidat-offre, le score pgvector est continu et centré autour de 0,52, tandis que 31,30 % des scores Neo4j sont nuls. Le premier panneau montre donc une proximité textuelle graduelle ; le second montre un signal relationnel plus sélectif. pgvector distingue des degrés de ressemblance entre presque tous les couples ; Neo4j écarte plus nettement ceux pour lesquels aucune compétence requise n'est couverte.

Figure 3.12 – Distribution comparée des scores pgvector et Neo4j utilisés dans le reranking



Source : Nos travaux, diagnostics PostgreSQL–pgvector, à partir des embeddings

Avec 500 essais Optuna, l'objectif NDCG@10 retient $\alpha = 0$ et $\beta = 1$: ce résultat de bord est inhabituel et ne doit pas être interprété comme l'inutilité de pgvector. Le vivier a déjà été présélectionné par pgvector ; l'optimisation ne compare donc que des offres sémantiquement plausibles. Dans ce sous-ensemble, la couverture relationnelle peut suffire à ordonner le top 10 selon le proxy retenu. Le résultat peut aussi refléter un alignement partiel entre ce proxy et les signaux de couverture utilisés par Neo4j. L'objectif MRR@10–Precision@10 retient plutôt 0,6615 pour pgvector et 0,3385 pour Neo4j, car il valorise davantage le premier résultat pertinent. Les poids sont donc propres à l'objectif et à l'échantillon de calibration ; ils doivent être confrontés à une analyse de sensibilité et à des jugements indépendants avant d'être fixés en production.

Table 3.4 – Résultats de calibration des poids du score hybride

Objectif optimisé	Outil	α Sem	β Neo4j	NDCG@10
NDCG@10	Neo4j seul	0,0000	1,0000	0,9697
(NDCG@10 + Recall@10 + Precision@10) / 3	Neo4j seul	0,0000	1,0000	0,9697
(MRR@10 + Precision@10) / 2	Mixte Optuna	0,6615	0,3385	0,9674

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

3.5.2 Effet du reranking hybride

Neo4j ne relance pas la recherche : il réordonne le vivier pgvector selon la couverture de compétences. Les déplacements de rang et l'ablation avec ou sans graphe sont analysés au chapitre 4. Le tableau A.12 en annexe conserve le résumé numérique.

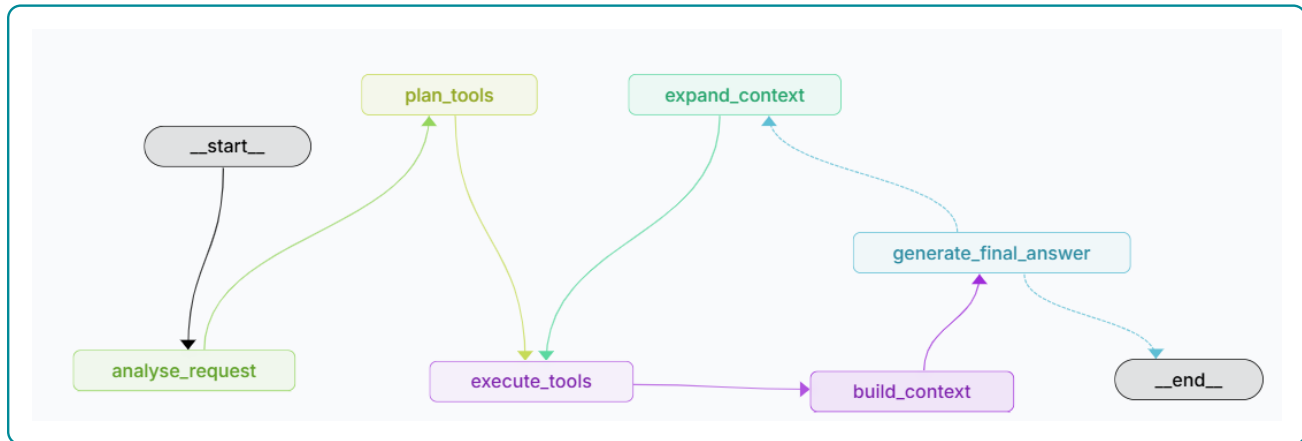
3.6 Orchestration agentique du système par LangGraph

Cette dernière brique transforme les composants précédents en agent IA outillé. Dans ce mémoire, l'agent n'est pas un simple chatbot qui répond directement à partir d'un prompt. Il suit une boucle contrôlée : comprendre la demande, choisir les outils utiles, exécuter les recherches, observer les résultats, construire un contexte, générer une réponse, puis vérifier si cette réponse est suffisamment appuyée par les preuves.

LangGraph sert à représenter cette boucle sous forme de graphe d'états. L'état partagé contient la requête, le cas d'usage, les outils planifiés, leurs résultats, le contexte construit, la réponse, la décision du critic et les traces d'exécution. L'agent ne remplace donc pas pgvector,

Neo4j ou le score hybride ; il décide quand les mobiliser et comment combiner leurs sorties pour répondre.

Figure 3.13 – Workflow agentique orchestré dans LangGraph Studio



Source : Nos travaux, visualisation LangGraph Studio du workflow implémenté.

La figure 3.13 décrit le comportement de cet agent IA. Le noeud `analyse_request` joue le rôle de perception : il transforme une question libre en intention exploitable. `plan_tools` correspond à la planification : l’agent choisit les outils nécessaires au lieu d’appeler toutes les bases sans distinction. `execute_tools` représente l’action : l’agent interroge pgvector, Neo4j ou le moteur hybride. `build_context` correspond à l’observation organisée : les résultats bruts sont regroupés en preuves utilisables. Enfin, `generate_final_answer` produit la réponse à partir du contexte construit.

Les flèches de retour vers `expand_context` sont importantes. Elles montrent que l’agent peut réviser sa trajectoire lorsque les preuves sont insuffisantes : il élargit alors la recherche, reconstruit le contexte et régénère la réponse. L’agent reste cependant borné par le workflow ; il ne crée pas librement de nouvelles actions et ne modifie pas les bases. Cette contrainte est nécessaire pour garder un système défendable.

3.6.1 Perception de la requête et planification des outils

Le noeud `analyse_request` extrait la question, recherche un identifiant candidat et classe la demande dans un cas d’usage : diagnostic, recommandation, analyse de *skill gap*, consultation d’un référentiel, interrogation du graphe, orientation métier, synthèse globale ou recherche générale. Le noeud `plan_tools` associe ensuite ce cas aux outils nécessaires.

Cette étape correspond à la partie décisionnelle de l’agent. Une question sur un référentiel mobilise les chunks pgvector ; une question relationnelle mobilise Neo4j ; une recommandation candidat–offre appelle le moteur hybride ; une demande de diagnostic vérifie plutôt l’état

des services. Le registre contient six outils principaux : diagnostic, recherche sémantique, recherche documentaire, interrogation Neo4j, recommandation hybride et synthèse globale du graphe.

Cette planification évite deux erreurs fréquentes dans les applications LLM. La première consiste à demander au modèle de répondre sans preuve. La seconde consiste à appeler tous les outils à chaque question, ce qui alourdit le système et produit du bruit. Ici, l'agent choisit le chemin le plus pertinent selon l'intention détectée.

3.6.2 Action, observation et construction du contexte

`execute_tools` appelle les outils planifiés et conserve chaque résultat ou erreur dans l'état. `build_context` rassemble ensuite les sorties hétérogènes dans une structure commune : offres candidates, compétences couvertes ou manquantes, lignes Neo4j, résultats sémantiques, fragments documentaires et état des services.

Dans une logique d'agent IA, cette étape correspond au couple action–observation. L'action est l'appel aux outils ; l'observation est la transformation des sorties en éléments interprétables. Par exemple, un résultat pgvector donne une proximité sémantique, tandis qu'un résultat Neo4j donne un chemin relationnel ou une couverture de compétences. Ces deux observations ne sont pas équivalentes ; elles doivent être rapprochées avant la génération.

Cette normalisation sépare la récupération de la rédaction. Le LLM ne consulte pas directement les bases et ne choisit pas librement les données à utiliser ; il reçoit un contexte déjà récupéré et structuré. Les erreurs d'un outil restent également traçables sans supprimer les résultats valides produits par les autres outils. C'est ce qui permet de justifier une réponse : l'agent peut indiquer les offres trouvées, les compétences couvertes, les compétences manquantes et les sources documentaires mobilisées.

3.6.3 Génération contrôlée et boucle de révision

`generate_final_answer` produit une réponse française à partir du contexte et impose la citation des identifiants ou sources disponibles. Si OpenRouter est indisponible, le système restitue les données récupérées sous une forme structurée plutôt que d'inventer une réponse.

Le `critic` joue le rôle de contrôle interne de l'agent. Il ne vérifie pas toute la vérité métier de la réponse ; il vérifie si la réponse est suffisamment ancrée dans le contexte récupéré. Une décision `accept` termine le workflow. Une décision `revise` active `expand_context`, augmente le top-K dans la limite de 10 et ajoute les recherches pgvector ou Neo4j qui n'avaient pas encore été exécutées. Le contexte est alors reconstruit et la réponse régénérée.

Cette boucle donne au système un comportement agentique : il ne répond pas seulement, il peut constater que son contexte est insuffisant et demander plus de preuves avant de finaliser. La boucle reste limitée à une seule révision afin d'éviter une exécution indéfinie. Le chapitre 4 présente la calibration du seuil utilisé par ce contrôle.

Conclusion

Le chapitre a matérialisé la chaîne suivante : données normalisées, embeddings spécialisés, rappel pgvector, contrôle Neo4j, reranking hybride et orchestration LangGraph. Chaque brique remplit une fonction distincte, tandis que l'orchestrateur assure leur enchaînement et la traçabilité des décisions. Le résultat reste dépendant de la qualité des données, des relations de compétences et du contexte récupéré ; le chapitre 4 mesure l'effet de ces choix sur le classement et la génération.

ÉVALUATION DE LA PERFORMANCE INTERNE DU SYSTÈME

Ce chapitre évalue le système implémenté au chapitre précédent. Il ne répète pas la description technique des briques ; il vérifie ce que ces briques apportent effectivement au fonctionnement du système. L'évaluation suit donc une logique en cinq niveaux : qualité du retrieval sémantique, effet du graphe sur le classement, validité fonctionnelle des requêtes GraphRAG, calibration du score hybride et contrôle de la génération finale par le critic.

4.1 Protocole expérimental et métriques retenues

L'évaluation mobilise trois familles d'artefacts. Le benchmark du modèle d'embeddings utilise le jeu de test issu des paires contrastives offre–profil, soit 473 requêtes. L'ablation pgvector contre pgvector enrichi par Neo4j utilise un tirage de 80 candidats avec graine fixée à 42 ; 69 candidats sont effectivement évalués après contrôles d'exécution. Enfin, la calibration du critic utilise un jeu contrastif de 102 évaluations construit à partir de 51 cas réels et de leurs contextes mélangés.

Les métriques de retrieval sont calculées sur les premières offres du classement. La notation « @10 » signifie que l'évaluation porte sur les dix offres considérées comme les plus compatibles avec chaque candidat. Precision@10 mesure la part de bonnes offres dans cette liste de dix ; Recall@10 mesure la part des offres pertinentes du vivier effectivement retrouvées ; NDCG@10 vérifie si les meilleures offres sont placées en tête ; MRR@10 dépend du rang de la première bonne offre ; HitRate@10 indique seulement si le top 10 en contient au moins une. Ces métriques sont moyennées sur les candidats évalués et varient entre 0 et 1 : plus elles sont proches de 1, meilleur est le classement.

Pour l'ablation, les deux configurations reçoivent le même pool initial de 30 offres produit par pgvector. La configuration `pgvector seul` conserve l'ordre vectoriel. La configuration `pgvector + graphe` enrichit ce même pool avec Neo4j, puis le rerange à partir des relations candidat–offre–compétence. Cette construction isole l'effet du graphe : la différence observée ne vient pas d'un corpus différent, mais de l'information relationnelle ajoutée au classement.

4.2 Retrieval sémantique et apport mesuré du graphe

4.2.1 Modèle d'embeddings spécialisé

Le premier test vérifie si le fine-tuning du SentenceTransformer améliore réellement le retrieval. Le tableau 4.1 compare le modèle spécialisé à plusieurs modèles généralistes ou multilingues sur les mêmes 473 requêtes de test.

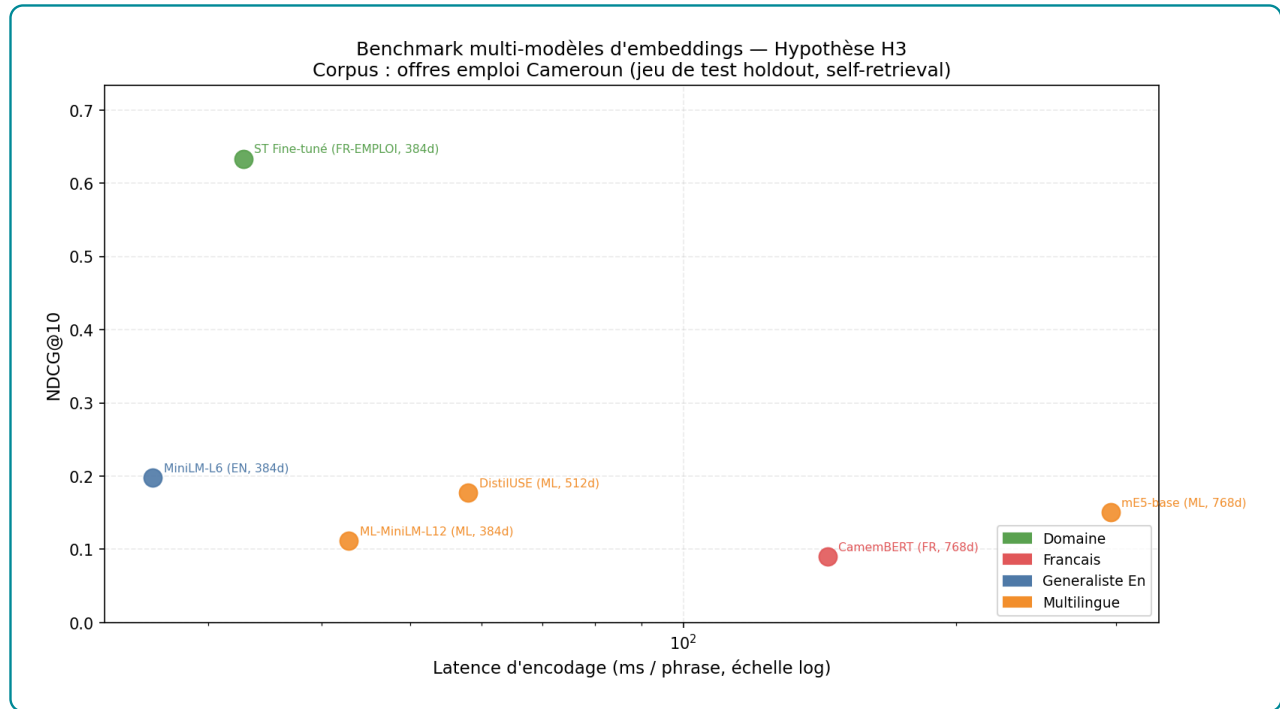
Table 4.1 – Benchmark des modèles d'embeddings sur 473 requêtes de test

Modèle	NDCG@10	MRR@10	Recall@10	Latence ms/phrased
ST fine-tuné emploi-compétences	0.6336	0.5653	0.8520	32.77
MiniLM-L6 généraliste	0.1980	0.1603	0.3192	26.03
DistilUSE multilingue	0.1773	0.1434	0.2896	57.96
mE5-base multilingue	0.1505	0.1274	0.2262	295.76
ML-MiniLM-L12	0.1115	0.0922	0.1734	42.82
CamemBERT sentence	0.0904	0.0771	0.1332	144.15

Source : Nos travaux, évaluation SentenceTransformer sur le jeu de test offre-profil.

Le modèle fine-tuné atteint un Recall@10 de 0,8520, contre 0,3192 pour MiniLM-L6 généraliste. Cela signifie que, sur l'ensemble des requêtes de test, environ 85,2 % des éléments attendus sont retrouvés dans les dix premiers résultats. Le NDCG@10 passe de 0,1980 à 0,6336 : l'ordre obtenu représente 63,4 % de la qualité du classement idéal sur ce test. Cette mesure porte sur la position des résultats, pas sur une probabilité de recrutement. La latence moyenne reste contenue à 32,77 ms par phrase, ce qui reste compatible avec une recherche interactive locale.

La figure 4.1 confirme que le gain ne vient pas seulement d'une métrique isolée. Le modèle fine-tuné domine sur la qualité de classement, le rappel et le compromis performance-latence. Ce résultat justifie son utilisation comme encodeur principal de pgvector. La barre de rappel indique la capacité à retrouver l'offre attendue dans le top 10 ; la barre de NDCG mesure si cette offre est bien placée ; la latence vérifie que le modèle reste utilisable dans le workflow. Le bon résultat n'est donc pas seulement statistique : il est aussi opérationnel pour alimenter rapidement pgvector.

Figure 4.1 – Comparaison des modèles d’embeddings sur le jeu de test

Source : Nos travaux, évaluation SentenceTransformer sur le jeu de test offre-profil.

4.2.2 Ablation pgvector seul contre pgvector enrichi par Neo4j

Le deuxième test mesure l’apport du graphe. Le même pool initial de 30 offres est utilisé dans les deux configurations ; seul le reranking par les relations Neo4j change. Le tableau 4.2 présente les résultats obtenus sur les 69 candidats effectivement évalués.

Table 4.2 – Comparaison retrieval : pgvector seul versus pgvector + graphe

Configuration	Precision@10	Recall@10	NDCG@10	MRR@10	HitRate@10
pgvector seul	0.4623	0.4491	0.5575	0.6908	0.9420
pgvector + graphe	0.6087	0.6147	0.7392	0.8093	0.9420
Différence	+0.1464	+0.1656	+0.1817	+0.1185	0.0000

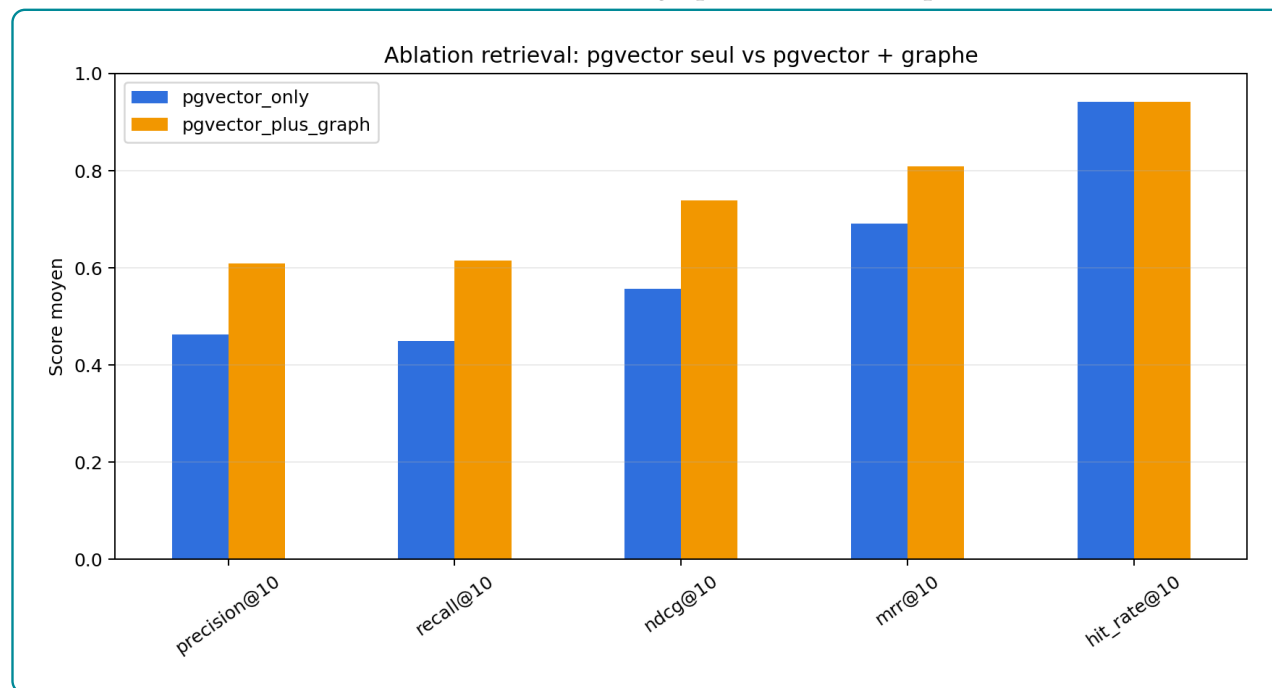
Source : Nos travaux, ablation du moteur de recommandation sur 69 candidats évalués.

L’ajout de Neo4j améliore toutes les mesures du top 10 sauf le HitRate, qui reste stable. Une Precision@10 de 0,6087 signifie qu’en moyenne environ 6 des 10 offres affichées après reranking sont jugées pertinentes par le proxy, contre environ 4,6 avec pgvector seul. Le Recall@10 de 0,6147 signifie que le top 10 récupère près de 61,5 % des offres pertinentes présentes dans le vivier évalué. Le NDCG@10 de 0,7392 indique que l’ordre obtenu atteint 73,9 % de la qualité du classement idéal défini par ce proxy. Enfin, un MRR@10 de 0,8093 montre que la

première offre pertinente apparaît généralement très près du début de la liste. Ces valeurs décrivent une cohérence interne avec le proxy de pertinence ; elles ne signifient pas que 73,9 % des recommandations seraient validées par des recruteurs.

La lecture correcte est séquentielle. pgvector constitue d’abord un vivier de 30 offres plausibles. Neo4j départage ensuite ces offres à partir des relations `POSSEDE`, `REQUIERT`, `A_NIVEAU` et `REQUIERT_NIVEAU`. L’amélioration mesurée vient donc du nouvel ordre appliqué au même vivier, et non d’un changement des offres disponibles.

Figure 4.2 – Effet de l’enrichissement graphe sur les métriques de retrieval



Source : Nos travaux, ablation du moteur de recommandation sur 69 candidats évalués.

4.3 Validation fonctionnelle du GraphRAG

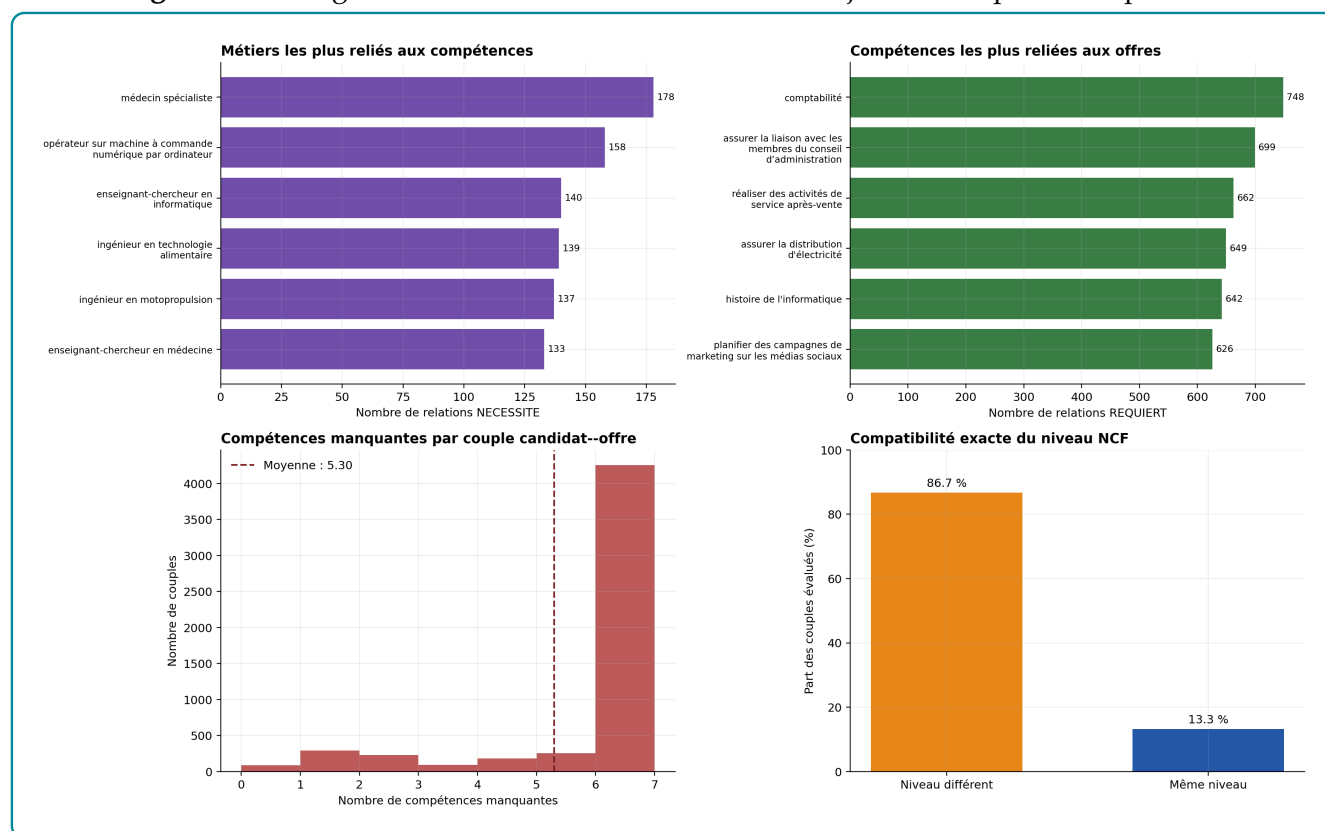
L’ablation précédente mesure l’effet du graphe sur le classement, mais elle ne suffit pas à vérifier que le GraphRAG fonctionne correctement dans les usages métier. Cette section évalue donc la chaîne fonctionnelle : partir d’une demande utilisateur, activer les chemins Neo4j utiles, récupérer les entités reliées, puis transformer ces relations en contexte exploitable par la génération. La figure 4.3 synthétise ces contrôles selon cinq angles complémentaires : recherche métier vers compétences, recherche compétence vers offres, calcul de *skill gap*, compatibilité NCF et comparaison RAG contre GraphRAG.

Le rôle de cette figure n’est pas seulement illustratif. Elle sert de tableau de bord fonctionnel : chaque barre correspond à une capacité attendue du système. Si une barre est faible, le

problème ne vient pas nécessairement du LLM ; il peut venir d'une relation manquante, d'un référentiel incomplet ou d'un chemin Neo4j trop pauvre. La lecture des sous-sections suivantes reprend donc les mêmes familles de tests que la figure afin de relier les résultats chiffrés au fonctionnement concret du GraphRAG.

La figure doit être lue comme quatre diagnostics distincts, et non comme un score global. En haut, elle vérifie que le graphe permet de passer d'un métier à ses compétences et d'une compétence aux offres associées. En bas à gauche, elle montre la distribution des compétences manquantes par couple candidat-offre. En bas à droite, elle mesure la part des couples dont les niveaux NCF correspondent exactement. Elle contrôle donc le fonctionnement des chemins Neo4j ; elle ne prouve pas à elle seule la pertinence finale des recommandations.

Figure 4.3 – Diagnostics fonctionnels des chemins Neo4j mobilisés par le GraphRAG



Source : Nos travaux, diagnostics Neo4j et GraphRAG sur les scénarios fonctionnels du système.

4.3.1 Requêtes relationnelles métier, compétence et offre

La première partie de la figure 4.3 vérifie la navigation depuis un métier vers les compétences attendues. Le graphe part d'un noeud **Métier** et suit la relation **NECESSITE** pour retrouver les compétences associées. Les métiers les plus riches sont très connectés : « médecin spécialiste » est relié à 178 compétences, « opérateur sur machine à commande numérique » à

158 et « enseignant-chercheur en informatique » à 140. Cette richesse est utile pour l'orientation, car elle donne au GraphRAG un contexte métier large. Elle ne doit cependant pas être lue comme une liste minimale d'exigences pour chaque offre : une offre précise ne mobilise qu'une partie de ce contexte.

La deuxième partie de la même figure contrôle le sens inverse : à partir d'une compétence, le graphe retrouve les offres qui la requièrent. La compétence « comptabilité » est reliée à 748 offres, « réaliser des activités de service après-vente » à 662 offres et « planifier des campagnes de marketing sur les médias sociaux » à 626 offres. Ce résultat confirme que Neo4j sert bien de moteur de recherche relationnelle pour compléter pgvector. Il révèle aussi une limite importante : certaines compétences fréquentes deviennent des hubs peu discriminants. Dans le score final, elles doivent donc être pondérées avec prudence, sinon le système risque de favoriser des correspondances trop générales.

4.3.2 Skill gap, niveau NCF et comparaison GraphRAG

La troisième lecture de la figure 4.3 porte sur la *skill gap*. Le calcul compare les compétences **REQUIERT** par l'offre aux compétences **POSSEDE** par le candidat. Sur l'échantillon fonctionnel, le déficit moyen est de 5,30 compétences et la médiane vaut 6. Ce résultat est utile parce qu'il montre que le GraphRAG ne se limite pas à dire qu'un candidat correspond ou ne correspond pas. Il identifie les compétences manquantes et peut transformer un mauvais appariement brut en recommandation explicable de montée en compétences.

La quatrième lecture concerne la compatibilité NCF. Elle vérifie si le niveau du candidat correspond au niveau requis par l'offre. Dans l'échantillon évalué, 13,29 % des couples partagent exactement le même niveau. Ce taux confirme que le niveau de formation apporte un signal structurant, mais il ne doit pas devenir un couperet automatique. Selon l'expérience et les compétences détenues, un écart de niveau peut rester acceptable ; le graphe doit donc éclairer la décision, pas la remplacer.

Enfin, la dernière partie de la figure 4.3 met en relation ces contrôles fonctionnels avec la comparaison RAG contre GraphRAG. Le RAG vectoriel produit le pool initial, tandis que le GraphRAG ajoute les relations Neo4j pour reranger les offres et fournir des explications. Les résultats sont nets : le NDCG@10 passe de 0.5575 à 0.7392, le Recall@10 de 0.4491 à 0.6147 et le MRR@10 de 0.6908 à 0.8093. Le GraphRAG améliore donc l'ordre des recommandations et pas seulement leur justification textuelle.

4.4 Reranking hybride et calibration du score

4.4.1 Déplacements de rang après reranking par le graphe

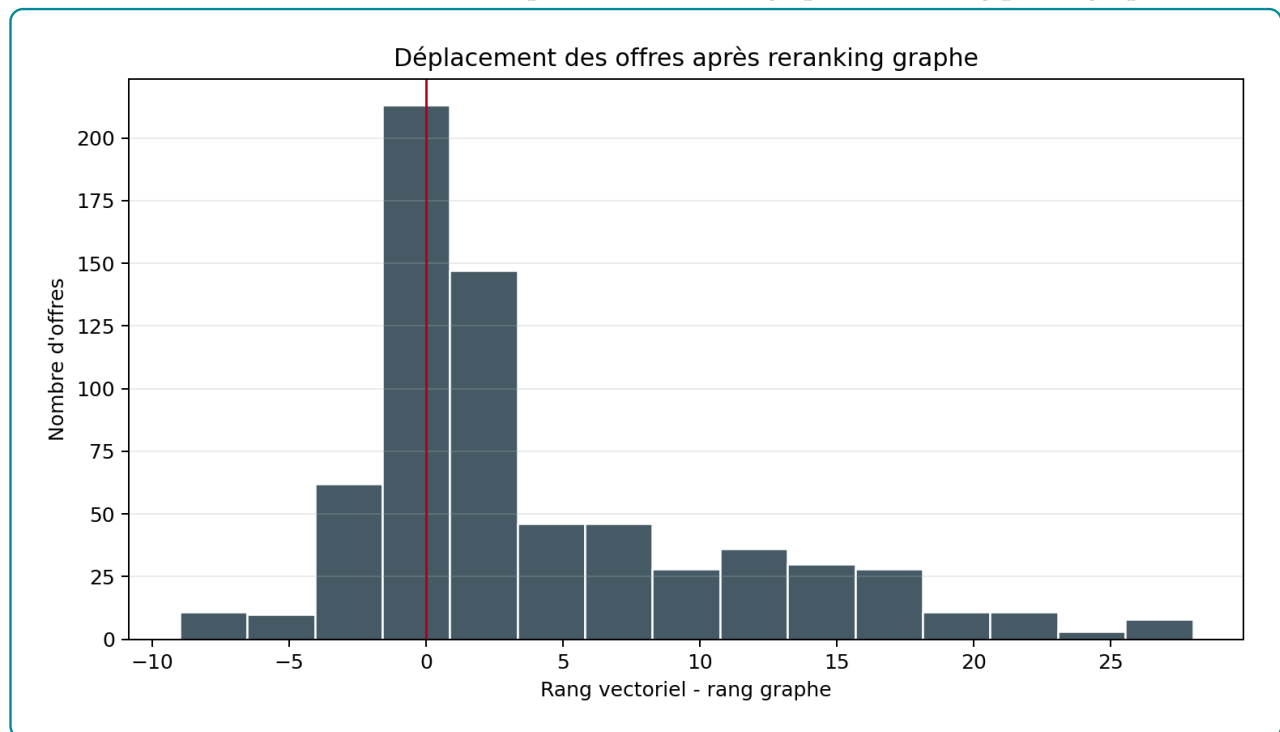
Le déplacement de rang est calculé par la différence

$$\Delta r = r_{\text{pgvector}} - r_{\text{hybride}}.$$

Une valeur positive indique qu'une offre remonte après l'ajout du score Neo4j ; une valeur négative indique une rétrogradation ; une valeur nulle signifie que son rang ne change pas. Dans cette expérience, le score Neo4j correspond au taux de couverture des compétences requises par l'offre. Le graphique ne mesure donc pas directement l'effet du secteur ou du niveau NCF.

Sur les 690 couples candidat-offre associés aux 69 candidats évalués, 59,86 % des offres remontent, 13,77 % reculent et 26,38 % conservent leur position. Le rang moyen passe de 9,51 avec pgvector seul à 5,50 après reranking, soit un gain moyen de 4,01 positions. Le déplacement médian n'est toutefois que de +1 : l'amélioration moyenne est donc influencée par un nombre plus limité de remontées importantes et ne signifie pas que chaque offre gagne quatre places.

Figure 4.4 – Distribution des déplacements de rang après reranking par le graphe



Source : Nos travaux, ablation du moteur hybride et analyse des déplacements de rang.

La figure montre ainsi que Neo4j modifie réellement l'ordre du vivier initial, principalement en favorisant les offres dont les compétences requises sont mieux couvertes. Une rétrogradation n'est pas nécessairement une erreur : elle peut signaler qu'une proximité textuelle élevée n'est pas confirmée par les relations de compétences. Inversement, le résultat dépend de la qualité des liens POSSEDE et REQUIERT. Cette analyse démontre un effet mesurable du graphe sur le classement, mais pas encore une pertinence métier définitive sans annotation humaine.

4.4.2 Pondérations optimales avec Optuna (optimisation bayésienne)

Afin de ne pas conserver des pondérations arbitraires, les poids du score hybride ont été optimisés avec Optuna sur 690 couples candidat-offre issus des 69 candidats évalués. Trois fonctions objectif sont comparées : NDCG@10, moyenne NDCG@10-Recall@10-Precision@10 et moyenne MRR@10-Precision@10. La pertinence utilisée est `proxy_relevance`, un label faible construit à partir des métadonnées normalisées.

La calibration explore un poids sémantique $w_{sem} \in [0, 1]$ et pose $w_{Neo4j} = 1 - w_{sem}$; 500 essais Optuna recherchent ainsi un compromis sur une seule dimension. L'unité pertinente est le candidat et son top 10, non un couple candidat-offre isolé : les mesures doivent être agrégées par candidat pour éviter qu'un candidat ayant davantage de couples ne pèse indûment dans l'objectif. Cette étape est une calibration interne, pas un test externe : le même protocole doit ensuite être rejoué sur des candidats tenus à l'écart et, idéalement, sur des annotations métier indépendantes.

Table 4.3 – Pondérations retenues selon la fonction objectif du score hybride

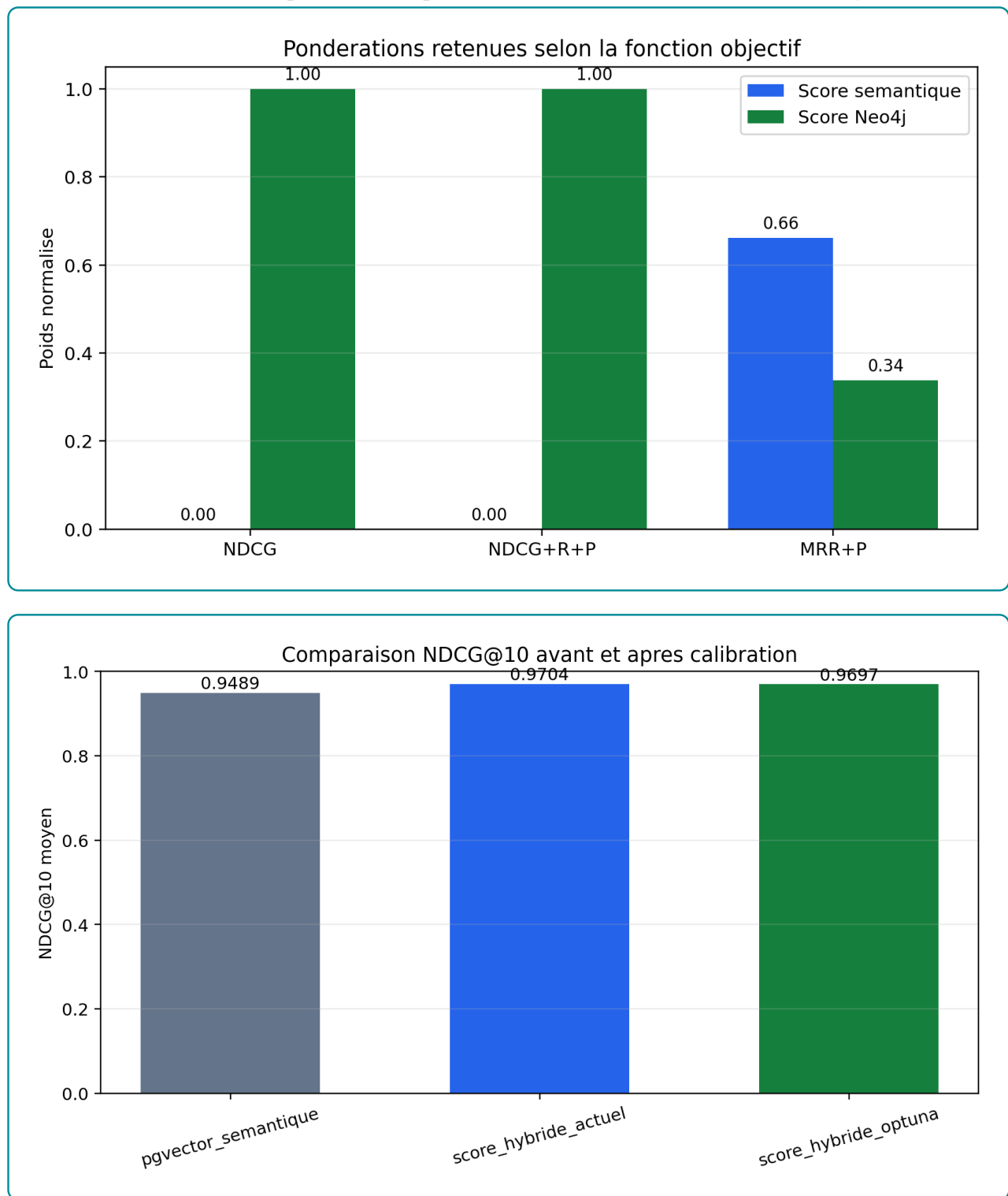
Fonction objectif	w_{sem}	w_{Neo4j}	NDCG@10	Precision@10	Recall@10	MRR@10
NDCG@10	0.0000	1.0000	0.9697	0.5000	1.0000	0.8109
NDCG@10 + Recall@10 + Precision@10	0.0000	1.0000	0.9697	0.5000	1.0000	0.8109
MRR@10 + Precision@10	0.6615	0.3385	0.9674	0.5000	1.0000	0.8183

Source : Nos travaux, optimisation Optuna des poids du score hybride sur les sorties d'ablation.

Le résultat est clair mais doit être bien défendu. Un NDCG@10 de 0,9697 signifie que, selon le proxy utilisé pour cette calibration, l'ordre du top 10 atteint 96,97 % de l'ordre idéal. Ce n'est ni une précision de 96,97 %, ni un taux de recrutement réussi. Lorsque l'objectif privilégie l'ordre global du top 10, Optuna retient un score Neo4j pur : $w_{sem} = 0$ et $w_{Neo4j} = 1$. Cela ne signifie pas que pgvector est inutile : pgvector a déjà construit le pool de candidats à reclasser, de sorte que l'optimum ne porte que sur la seconde étape. Il peut aussi refléter un

alignement partiel entre le proxy de pertinence et les métadonnées de couverture exploitées par Neo4j. Lorsque l'objectif privilégie davantage la première bonne recommandation, le compromis devient hybride avec $w_{sem} = 0,6615$ et $w_{Neo4j} = 0,3385$. La solution de bord doit donc être analysée par stabilité des poids, par bootstrap ou validation croisée au niveau candidat, puis sur un jeu séparé ; elle n'est pas un paramètre universel du pipeline.

La figure compare deux lectures. Le panneau multi-objectifs montre que les poids changent selon la priorité retenue : qualité globale du top 10 ou première bonne recommandation. Le panneau centré sur NDCG@10 montre au contraire que, pour ordonner l'ensemble du top 10 dans ce protocole, le signal Neo4j domine. Cette différence justifie de présenter les poids comme dépendants de l'objectif, et non comme des coefficients universels.

Figure 4.5 – Comparaison des pondérations retenues selon la fonction objectif

Source : Nos travaux, optimisation Optuna des poids du score hybride.

La conclusion opérationnelle est la suivante : pgvector doit être conservé pour le rappel initial, tandis que Neo4j doit peser fortement dans le classement final lorsque les relations **POSSEDE** et **REQUIERT** sont disponibles. La calibration reste cependant dépendante du proxy de

pertinence ; une validation humaine est nécessaire avant d'en faire des poids définitifs.

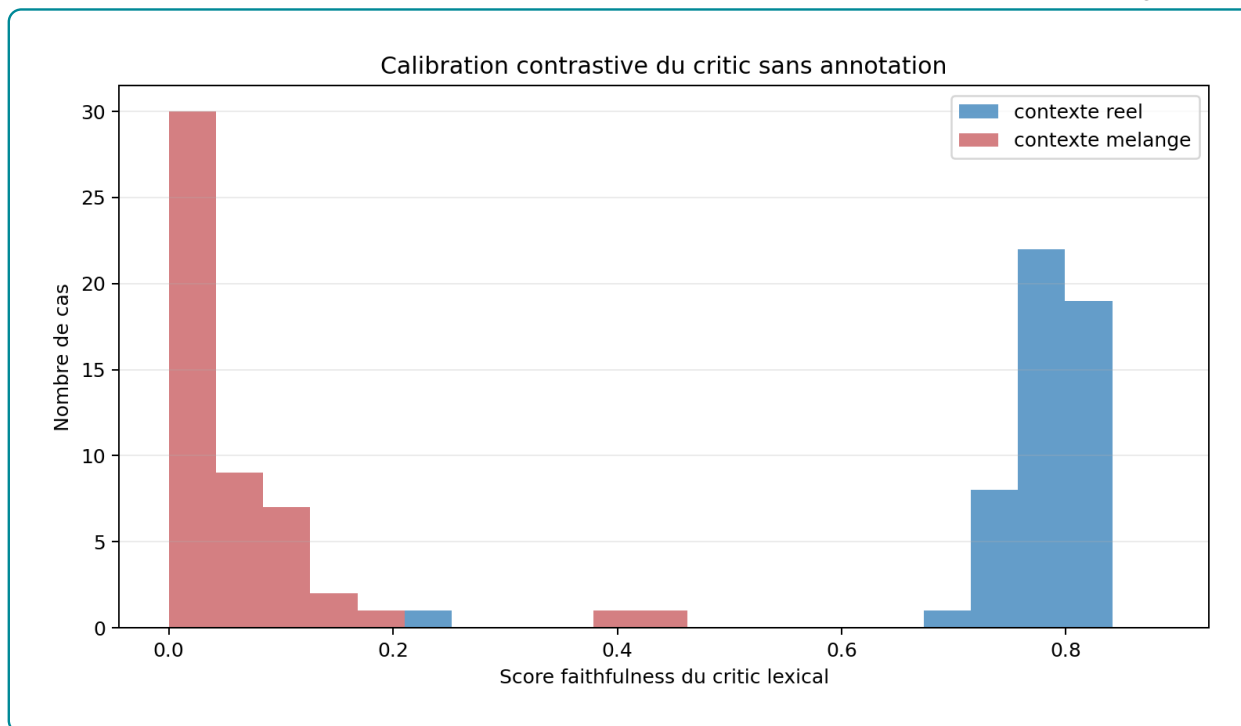
4.5 Contrôle de génération par critic

4.5.1 Calibration contrastive du seuil de fidélité

Après le retrieval et le reranking, le LLM OpenRouter formule la réponse finale. Le critic vérifie ensuite si cette réponse reste suffisamment appuyée par le contexte récupéré. Le score utilisé est un score de fidélité lexicale : il compare les mots significatifs de la réponse aux mots présents dans les preuves. Il ne juge pas la qualité métier complète de la réponse ; il vérifie seulement son ancrage dans le contexte.

Aucun jeu d'annotations humaines n'étant disponible, le seuil est calibré par contraste. Une réponse comparée à son vrai contexte doit obtenir un score plus élevé que la même réponse comparée à un contexte mélangé provenant d'une autre requête. Le protocole contient 51 cas réels évalués deux fois, soit 102 évaluations. Le score moyen vaut 0.7725 avec contexte réel contre 0.0532 avec contexte mélangé ; la médiane passe de 0.7857 à 0.

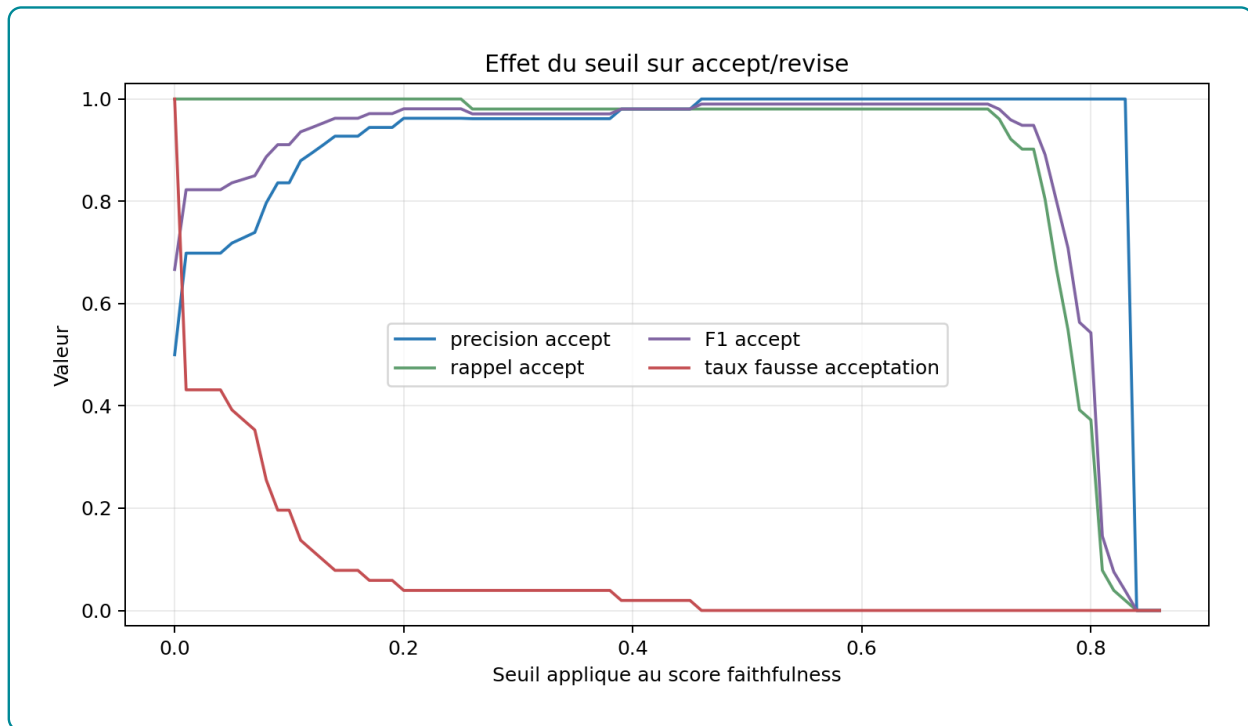
La figure oppose deux distributions. Les scores avec vrai contexte sont concentrés vers des valeurs élevées, tandis que les scores avec contexte mélangé sont proches de zéro. Cette séparation montre que le critic détecte bien l'ancrage lexical dans les preuves. La limite est importante : il mesure la fidélité au contexte, pas la vérité métier complète de la réponse.

Figure 4.6 – Distribution des scores du critic avec contexte réel et contexte mélangé

Source : Nos travaux, évaluation du critic à partir des réponses générées et des contextes récupérés.

4.5.2 Interprétation des décisions accept/revise

Le seuil opérationnel retenu est 0,46. Il a été obtenu par une calibration empirique sur une grille de seuils appliquée aux 102 évaluations contrastives : 51 réponses comparées à leur vrai contexte et les mêmes 51 réponses comparées à un contexte mélangé. Pour chaque seuil candidat, le critic transforme le score continu en décision binaire : **accept** si le score est supérieur ou égal au seuil, **revise** sinon. La grille permet ensuite de compter deux erreurs : les fausses acceptations, c'est-à-dire les contextes mélangés acceptés à tort, et les fausses révisions, c'est-à-dire les vrais contextes renvoyés inutilement en révision.

Figure 4.7 – Effet du seuil du critic sur les décisions accept/revise

Source : Nos travaux, évaluation du critic à partir des réponses générées et des contextes récupérés.

La valeur 0,46 est retenue parce qu'elle maximise le compromis F1 dans ce test tout en respectant une contrainte de prudence : ne laisser passer aucun contexte mélangé. À ce niveau, la matrice de décision est $TP = 50$, $FN = 1$, $TN = 51$ et $FP = 0$, où la classe positive désigne une réponse associée à son vrai contexte. La sensibilité vaut donc $50/51 = 98,04\%$, la spécificité $51/51 = 100\%$, la précision 100% et $F1 = 100/101 = 99,01\%$. Ces valeurs décrivent uniquement la capacité à distinguer le vrai contexte du contexte mélangé dans cet échantillon.

La figure montre l'arbitrage du seuil. Un seuil trop bas accepte trop facilement des réponses peu ancrées ; un seuil trop haut force des révisions inutiles. Le seuil 0,46 se situe dans la zone où les contextes mélangés sont rejetés tout en conservant presque toutes les réponses associées à leur vrai contexte. Le choix est donc une règle de prudence opérationnelle, mais l'absence observée de fausse acceptation sur 51 cas ne prouve pas que son taux réel est nul.

Cette calibration ne transforme pas le critic en juge parfait. Une réponse acceptée n'est pas automatiquement correcte au sens métier ; elle est seulement suffisamment ancrée lexicalement dans les preuves récupérées. En outre, le même échantillon a servi à sélectionner puis à décrire le seuil : la performance rapportée est donc optimiste. Une calibration expérimentale complète doit séparer les requêtes de développement, de validation et de test, conserver les paires réponse–contexte ensemble lors du découpage, puis fournir des intervalles d'incer-

titude par bootstrap et une annotation humaine de la factualité. Pour défendre le système devant un jury, la formulation correcte est donc : le seuil 0,46 est un seuil opérationnel calibré empiriquement par contraste, en attente d'une validation indépendante de la qualité réelle des recommandations générées.

Conclusion

L'évaluation montre trois résultats principaux. Premièrement, le fine-tuning du modèle d'embeddings améliore fortement le retrieval sémantique, avec un Recall@10 de 0.8520 et un NDCG@10 de 0.6336. Deuxièmement, l'ajout de Neo4j améliore le classement des offres dans le pool pgvector : le NDCG@10 passe de 0.5575 à 0.7392 et le MRR@10 de 0.6908 à 0.8093. Troisièmement, le critic distingue correctement les réponses ancrées dans leur contexte des réponses confrontées à un contexte mélangé, ce qui justifie le seuil opérationnel de 0,46.

La limite principale reste l'absence d'annotation humaine indépendante. Les résultats démontrent la cohérence interne du système et l'intérêt mesurable du graphe dans le protocole actuel. Ils ne prouvent pas encore la satisfaction d'un recruteur, d'un candidat ou d'un conseiller d'orientation. Cette limite doit être assumée : le système est techniquement validé sur ses sorties internes, mais son évaluation métier finale devra passer par un jeu annoté et des tests utilisateurs.

SYNTHÈSE ET DISCUSSION DES RÉSULTATS

Ce chapitre relie les résultats techniques aux hypothèses de recherche formulées en introduction. Il ne reprend pas tous les tableaux d'évaluation ; il met en évidence les enseignements majeurs du mémoire, les confronte à la littérature et précise la portée réelle des résultats obtenus.

5.1 Résultats majeurs de l'étude

Le premier résultat important concerne la qualité de la représentation sémantique. Le modèle SentenceTransformer adapté au domaine emploi-compétences atteint un Recall@10 de 0.8520 et un NDCG@10 de 0.6336. Cette progression confirme que le vocabulaire des offres, des profils et des compétences ne peut pas être correctement traité par une représentation générique seule. Les intitulés de poste, les compétences opérationnelles et les descriptions de formation possèdent des régularités spécifiques ; le fine-tuning permet de rapprocher les objets qui se correspondent réellement dans ce domaine.

Le deuxième résultat porte sur l'apport de Neo4j. L'ablation montre que le passage de pgvector seul à pgvector enrichi par le graphe améliore le NDCG@10 de 0.5575 à 0.7392 et le MRR@10 de 0.6908 à 0.8093. Ce gain est central : pgvector fournit un vivier sémantiquement plausible, mais Neo4j permet de départager les offres à partir de relations métier observables, notamment les compétences possédées, les compétences requises et les niveaux de formation.

Le troisième résultat concerne l'explicabilité. Le système ne produit pas uniquement une liste d'offres ; il identifie les compétences couvertes, les compétences manquantes et les relations qui relient un candidat à une offre. Cette capacité est importante pour un outil de décision : une recommandation emploi-compétences ne doit pas seulement répondre à la question « quelle offre proposer ? », mais aussi à « pourquoi cette offre ? » et « que manque-t-il pour y accéder ? ».

Le quatrième résultat concerne la génération contrôlée. Le critic calibré à 0,46 sépare correctement les réponses ancrées dans leur vrai contexte des réponses associées à un contexte mélangé. Ce résultat ne prouve pas que toutes les réponses sont parfaites au sens métier, mais

il montre que le système dispose d'un mécanisme de prudence avant de présenter une réponse finale.

5.2 Confrontation avec la littérature

Les résultats confirment d'abord les limites des systèmes de recommandation fondés uniquement sur les interactions passées. Les méthodes de filtrage collaboratif et de factorisation matricielle sont efficaces lorsque les historiques d'interaction sont riches [22, 17]. Dans le cas étudié, cette condition n'est pas pleinement remplie : les profils et offres disponibles ne forment pas une matrice dense de clics, candidatures ou recrutements validés. Le recours à une recommandation hybride est donc cohérent avec la littérature qui recommande de combiner plusieurs signaux lorsque les données sont hétérogènes [12, 37].

Les résultats rejoignent également les travaux sur les embeddings de phrases. Sentence-BERT montre l'intérêt de représentations adaptées aux tâches de similarité sémantique [36]. Dans ce mémoire, le fine-tuning confirme cette idée sur un cas appliqué : l'adaptation au vocabulaire emploi-compétences améliore le retrieval et rend pgvector plus utile comme mémoire de rappel.

L'apport du graphe s'inscrit dans la logique des graphes de connaissances, qui structurent les entités et leurs relations [19]. La littérature RAG montre que la génération gagne en fiabilité lorsqu'elle s'appuie sur des sources récupérées plutôt que sur la seule mémoire paramétrique du modèle [24, 15]. Le GraphRAG utilisé ici prolonge cette logique : les preuves ne sont pas seulement des textes proches, mais aussi des relations métier-compétence-formation exploitables.

Enfin, la présence du critic rejoint les approches récentes qui cherchent à contrôler l'ancrage des réponses générées [4, 14]. Le résultat obtenu doit toutefois rester correctement interprété : le critic mesure surtout la cohérence entre la réponse et le contexte récupéré ; il ne remplace pas l'expertise humaine sur la pertinence finale d'une recommandation.

5.3 Discussion des hypothèses de recherche

Le tableau ci-dessous résume la lecture des hypothèses au regard des résultats obtenus. Le statut retenu reste prudent : une hypothèse peut être soutenue par les résultats internes sans être définitivement validée au sens métier.

Table 5.1 – Lecture des hypothèses de recherche au regard des résultats

Hyp.	Idée testée	Statut	Justification synthétique
H1	Données structurées et modèle adapté au vocabulaire de l'emploi	Soutenue partiellement	Le modèle adapté améliore nettement le classement sémantique et les référentiels harmonisent les métiers, compétences et niveaux. L'effet propre de chaque référentiel n'a toutefois pas été isolé.
H2	Apport des relations entre candidats, compétences et offres	Soutenue	À dix résultats, le NDCG passe de 0,5575 à 0,7392 et le MRR de 0,6908 à 0,8093 après l'ajout du graphe.
H3	Pertinence et compréhension de la recommandation complète	Soutenue avec prudence	Le workflow combine recherche, relations et contrôle de la réponse, puis indique les compétences couvertes et manquantes. Une validation par des experts métier reste nécessaire.

Source : Nos travaux, synthèse des résultats des chapitres 3 et 4.

La lecture globale est favorable au système hybride, sans permettre une conclusion définitive sur son usage en recrutement. H2 est la mieux étayée, car elle repose sur une comparaison directe entre deux classements appliqués au même vivier d'offres. H1 combine un gain quantitatif du modèle adapté et un apport structurel des référentiels, dont l'effet séparé reste à mesurer. H3 est cohérente avec les contrôles réalisés, mais dépend encore d'une validation par des utilisateurs et des experts métier.

5.4 Portée, limites et conditions d'usage

La première limite concerne la validation humaine. Les métriques de ranking, le proxy de pertinence et le critic donnent une évaluation interne utile, mais ils ne remplacent pas une annotation indépendante par des recruteurs, des conseillers d'orientation ou des experts métier. Cette limite ne rend pas l'étude inutile ; elle précise simplement son niveau de maturité. Le mémoire valide une architecture et ses effets internes, pas encore un déploiement métier définitif.

La deuxième limite concerne les données. Les offres proviennent de sources scrapées telles que les plateformes d'emploi disponibles, avec des niveaux de complétude variables selon les champs. Le pipeline ETL réduit ce bruit par normalisation, mais il ne peut pas créer une

information absente dans la source initiale. Les recommandations restent donc dépendantes de la qualité des compétences, niveaux, secteurs et descriptions collectés.

La troisième limite concerne le graphe. Les diagnostics montrent une dépendance à certains hubs de compétences très fréquents. Le score Neo4j a donc été construit à partir de la couverture des compétences requises plutôt qu'à partir du degré brut. Ce choix limite le bruit de popularité, mais il suppose que les relations **POSSEDE** et **REQUIERT** soient correctement extraites et maintenues.

La quatrième limite concerne la génération. Le critic réduit le risque de réponses mal ancrées, mais il ne juge pas toute la vérité métier. Une réponse peut être fidèle au contexte récupéré tout en restant discutable si le contexte lui-même est incomplet. L'usage professionnel du système doit donc conserver une logique d'aide à la décision, et non de décision automatique.

En définitive, l'étude montre qu'un système hybride est plus défendable qu'une approche uniquement textuelle ou uniquement générative. Sa valeur tient à l'articulation entre trois couches : une mémoire vectorielle pour retrouver rapidement des candidats plausibles, une mémoire relationnelle pour expliquer les correspondances, et une orchestration agentique pour mobiliser les bons outils selon la requête.

CONCLUSION GÉNÉRALE ET PERSPECTIVES

Ce mémoire avait pour objectif de proposer et de formaliser un pipeline hybride de recherche sémantique et de recommandation emploi-compétences, capable de transformer des profils, des offres et des référentiels hétérogènes en recommandations explicables. Cette proposition est structurée autour de rôles séparés : la normalisation rend les données comparables ; la mémoire vectorielle fournit le rappel sémantique ; le graphe de connaissances vérifie les relations métier-compétence-formation ; le reranking et le *skill gap* explicitent le classement ; enfin, le LLM orchestre ou formule une réponse à partir de preuves tracées.

Les chapitres d'implémentation et d'évaluation conservent une opérationnalisation pilote de ce pipeline. Ils montrent que les briques peuvent être exécutées ensemble et que leurs sorties internes peuvent être mesurées. Le fine-tuning, le reranking relationnel et le critic constituent ainsi des diagnostics techniques utiles, mais ils ne démontrent ni un taux de recrutement réussi ni une supériorité métier généralisable. Cette distinction est essentielle : une bonne valeur de NDCG sur un proxy renseigne sur l'ordre induit par ce proxy ; elle ne remplace pas le jugement d'un recruteur ou d'un conseiller.

La contribution du mémoire est donc une architecture modulaire et auditable plutôt qu'un modèle prétendument autonome. Elle défend une règle simple : une similarité élevée ne suffit pas à recommander ; la compatibilité doit être contrôlée par des relations explicites, et toute explication générée doit pouvoir être reliée à ses sources. La plateforme KmerAI constitue un contexte de démonstration de cette proposition, non une base suffisante pour généraliser des conclusions à l'ensemble du marché camerounais.

Les limites identifiées sont méthodologiquement décisives. Le proxy de pertinence partage des signaux avec le système évalué ; la calibration du critic repose sur des contextes contrastés ; les données collectées peuvent être incomplètes ou déséquilibrées. Avant tout usage métier, les poids du reranking, les seuils et les prompts doivent être évalués hors échantillon, avec des annotations indépendantes et une analyse d'incertitude.

Les perspectives suivent donc un ordre exigeant : constituer un jeu de test annoté par des recruteurs, conseillers d'orientation et experts formation ; séparer calibration et évaluation finale ; mesurer la pertinence, l'explicabilité et les erreurs par sous-groupes ; puis réaliser des

tests utilisateurs encadrés. L'intégration ultérieure dans le site de KmerAI, par une interface de programmation et un chatbot, ne devra intervenir qu'après ces validations. Le graphe, les modèles d'embeddings et les artefacts de test pourront alors être versionnés et documentés afin de rendre le pipeline reproductible et maintenable.

RÉFÉRENCES BIBLIOGRAPHIQUES

- [1] Gediminas ADOMAVICIUS et Alexander TUZHILIN. "Toward the Next Generation of Recommender Systems: A Survey of the State-of-the-Art and Possible Extensions". In : *IEEE Transactions on Knowledge and Data Engineering* 17.6 (2005), p. 734-749. DOI : [10.1109/TKDE.2005.99](https://doi.org/10.1109/TKDE.2005.99).
- [2] George A. AKERLOF. "The Market for "Lemons": Quality Uncertainty and the Market Mechanism". In : *The Quarterly Journal of Economics* 84.3 (1970), p. 488-500. DOI : [10.2307/1879431](https://doi.org/10.2307/1879431).
- [3] Takuya AKIBA et al. "Optuna: A Next-generation Hyperparameter Optimization Framework". In : *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. 2019, p. 2623-2631. DOI : [10.1145/3292500.3330701](https://doi.org/10.1145/3292500.3330701).
- [4] Akari ASAI et al. "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection". In : *International Conference on Learning Representations*. 2024. URL : <https://openreview.net/forum?id=hSyW5go0v8>.
- [5] David H. AUTOR. "The "Task Approach" to Labor Markets: An Overview". In : *Journal for Labour Market Research* 46.3 (2013), p. 185-199. DOI : [10.1007/s12651-013-0128-z](https://doi.org/10.1007/s12651-013-0128-z).
- [6] Dzmitry BAHDANAU, Kyunghyun CHO et Yoshua BENGIO. "Neural Machine Translation by Jointly Learning to Align and Translate". In : *International Conference on Learning Representations*. 2015. URL : <https://arxiv.org/abs/1409.0473>.
- [7] BANQUE MONDIALE. *Chômage des jeunes de 15 à 24 ans au Cameroun, estimations modélisées de l'OIT*. Indicateur SL.UEM.1524.ZS, base des Indicateurs du développement dans le monde. 2026. URL : <https://api.worldbank.org/v2/country/CMR/indicator/SL.UEM.1524.ZS?format=json> (visité le 22/06/2026).
- [8] BANQUE MONDIALE. *Emploi vulnérable au Cameroun, estimations modélisées de l'OIT*. Indicateur SL.EMP.VULN.ZS, base des Indicateurs du développement dans le monde. 2026. URL : <https://api.worldbank.org/v2/country/CMR/indicator/SL.EMP.VULN.ZS?format=json> (visité le 22/06/2026).

- [9] BANQUE MONDIALE. *Part des jeunes sans emploi et ne suivant ni études ni formation au Cameroun*. Indicateur SL.UEM.NEET.ZS, base des Indicateurs du développement dans le monde. 2026. URL : <https://api.worldbank.org/v2/country/CMR/indicator/SL.UEM.NEET.ZS?format=json> (visité le 22/06/2026).
- [10] Gary S. BECKER. *Human Capital: A Theoretical and Empirical Analysis, with Special Reference to Education*. New York : Columbia University Press, 1964.
- [11] Tom B. BROWN et al. "Language Models are Few-Shot Learners". In : *Advances in Neural Information Processing Systems* 33 (2020), p. 1877-1901. URL : <https://proceedings.neurips.cc/paper/2020/hash/1457c0d6bfc4967418bfb8ac142f64a-Abstract.html>.
- [12] Robin BURKE. "Hybrid Recommender Systems: Survey and Experiments". In : *User Modeling and User-Adapted Interaction* 12 (2002), p. 331-370. DOI : [10.1023/A:1021240730564](https://doi.org/10.1023/A:1021240730564).
- [13] Jacob DEVLIN et al. "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding". In : *Proceedings of NAACL-HLT*. 2019, p. 4171-4186. URL : <https://aclanthology.org/N19-1423/>.
- [14] Shahul Es et al. "RAGAS: Automated Evaluation of Retrieval Augmented Generation". In : *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics: System Demonstrations*. 2024, p. 150-158. URL : <https://aclanthology.org/2024.eacl-demo.16/>.
- [15] Yunfan GAO et al. *Retrieval-Augmented Generation for Large Language Models: A Survey*. 2023. arXiv : [2312.10997](https://arxiv.org/abs/2312.10997) [cs.CL]. URL : <https://arxiv.org/abs/2312.10997>.
- [16] William L. HAMILTON. *Graph Representation Learning*. Morgan & Claypool Publishers, 2020. DOI : [10.2200/S01045ED1V01Y202009AIM046](https://doi.org/10.2200/S01045ED1V01Y202009AIM046).
- [17] Xiangnan HE et al. "Neural Collaborative Filtering". In : *Proceedings of the 26th International Conference on World Wide Web*. 2017, p. 173-182.
- [18] Xiangnan HE et al. "LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation". In : *Proceedings of SIGIR*. 2020, p. 639-648.
- [19] Aidan HOGAN et al. "Knowledge Graphs". In : *ACM Computing Surveys* 54.4 (2021), p. 1-37. DOI : [10.1145/3447772](https://doi.org/10.1145/3447772).
- [20] Jeff JOHNSON, Matthijs DOUZE et Hervé JÉGOU. "Billion-scale Similarity Search with GPUs". In : *IEEE Transactions on Big Data* 7.3 (2019), p. 535-547. DOI : [10.1109/TBDATA.2019.2921572](https://doi.org/10.1109/TBDATA.2019.2921572).

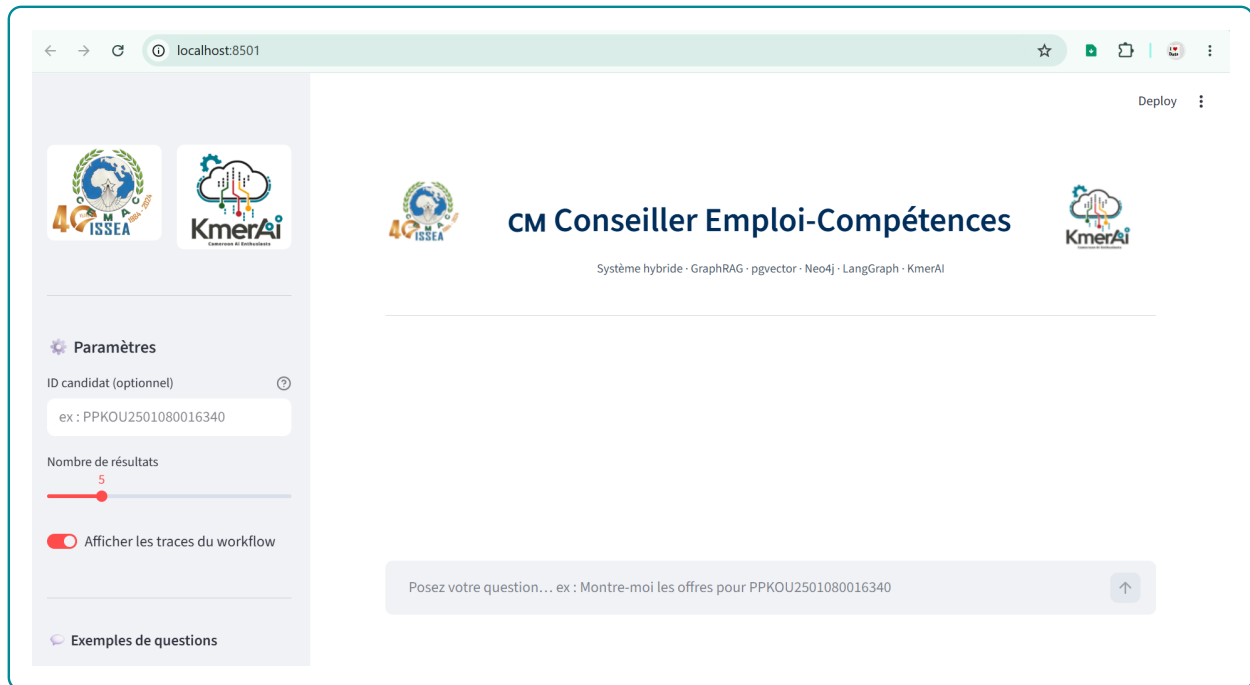
- [21] Thomas N. KIPF et Max WELLING. “Semi-Supervised Classification with Graph Convolutional Networks”. In : *International Conference on Learning Representations*. 2017.
- [22] Yehuda KOREN, Robert BELL et Chris VOLINSKY. “Matrix Factorization Techniques for Recommender Systems”. In : *Computer* 42.8 (2009), p. 30-37. DOI : [10.1109/MC.2009.263](https://doi.org/10.1109/MC.2009.263).
- [23] LANGCHAIN. *LangGraph Documentation*. Documentation officielle du framework LangGraph. 2026. URL : <https://langchain-ai.github.io/langgraph/> (visité le 06/06/2026).
- [24] Patrick LEWIS et al. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. In : *Advances in Neural Information Processing Systems* 33 (2020), p. 9459-9474. URL : <https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html>.
- [25] Yang LIU et al. “G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment”. In : *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. 2023, p. 2511-2522. URL : <https://aclanthology.org/2023.emnlp-main.153/>.
- [26] Yu A. MALKOV et D. A. YASHUNIN. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4 (2020), p. 824-836. DOI : [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- [27] Tomas MIKOLOV et al. “Distributed Representations of Words and Phrases and their Compositionality”. In : *Advances in Neural Information Processing Systems*. T. 26. 2013. URL : <https://proceedings.neurips.cc/paper/5021-distributed-representations-of-words-and-phrases-and-their-compositionality>.
- [28] Dale T. MORTENSEN et Christopher A. PISSARIDES. “Job Creation and Job Destruction in the Theory of Unemployment”. In : *The Review of Economic Studies* 61.3 (1994), p. 397-415. DOI : [10.2307/2297896](https://doi.org/10.2307/2297896).
- [29] Niklas MUENNIGHOFF et al. “MTEB: Massive Text Embedding Benchmark”. In : *Proceedings of the 17th Conference of the European Chapter of the Association for Computational Linguistics*. 2023, p. 2014-2037. URL : <https://aclanthology.org/2023.eacl-main.148/>.
- [30] ORGANISATION INTERNATIONALE DU TRAVAIL. *The Jobs Gap: Measuring Labour Underutilisation Beyond Unemployment*. 13. Organisation Internationale du Travail, fév. 2024.

- [31] Long OUYANG et al. "Training Language Models to Follow Instructions with Human Feedback". In : *Advances in Neural Information Processing Systems* 35 (2022), p. 27730-27744. URL : https://proceedings.neurips.cc/paper_files/paper/2022/hash/b1efde53be364a73914f58805a001731-Abstract-Conference.html.
- [32] Michael J. PAZZANI et Daniel BILLSUS. "Content-Based Recommendation Systems". In : *The Adaptive Web* (2007), p. 325-341. DOI : [10.1007/978-3-540-72079-9_10](https://doi.org/10.1007/978-3-540-72079-9_10).
- [33] PGVECTOR CONTRIBUTORS. *pgvector: Open-source Vector Similarity Search for Postgres*. 2024. URL : <https://github.com/pgvector/pgvector> (visité le 04/05/2026).
- [34] Christopher A. PISSARIDES. *Equilibrium Unemployment Theory*. 2^e éd. Cambridge, MA : MIT Press, 2000.
- [35] Colin RAFFEL et al. "Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer". In : *Journal of Machine Learning Research* 21.140 (2020), p. 1-67. URL : <https://www.jmlr.org/papers/v21/20-074.html>.
- [36] Nils REIMERS et Iryna GUREVYCH. "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks". In : *Proceedings of EMNLP-IJCNLP*. 2019, p. 3982-3992. URL : <https://aclanthology.org/D19-1410/>.
- [37] Francesco RICCI et al., éd. *Recommender Systems Handbook*. Springer, 2011. DOI : [10.1007/978-0-387-85820-3](https://doi.org/10.1007/978-0-387-85820-3).
- [38] Michael SPENCE. "Job Market Signaling". In : *The Quarterly Journal of Economics* 87.3 (1973), p. 355-374. DOI : [10.2307/1882010](https://doi.org/10.2307/1882010).
- [39] Nandan THAKUR et al. "BEIR: A Heterogeneous Benchmark for Zero-shot Evaluation of Information Retrieval Models". In : *Advances in Neural Information Processing Systems*. T. 34. 2021, p. 28974-28989. URL : <https://arxiv.org/abs/2104.08663>.
- [40] Ashish VASWANI et al. "Attention Is All You Need". In : *Advances in Neural Information Processing Systems*. 2017, p. 5998-6008.
- [41] Petar VELIČKOVIĆ et al. "Graph Attention Networks". In : *International Conference on Learning Representations*. 2018.
- [42] A. T. WASI. "HRGraph: Leveraging LLMs for HR Data Knowledge Graphs with Information Propagation-based Job Recommendation". In : *Proceedings of the 1st Workshop on Knowledge Graphs and Large Language Models*. 2024, p. 56-62.

- [43] B. YANG et Z. SHEN. “Knowledge Graph Construction and Talent Competency Prediction for Human Resource Management”. In : *Alexandria Engineering Journal* 121 (2025), p. 223-235.
- [44] Chen YANG et al. “Modeling Two-way Selection Preference for Person-Job Fit”. In : *Proceedings of the 16th ACM Conference on Recommender Systems*. 2022, p. 102-112.
- [45] Shunyu YAO et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In : *International Conference on Learning Representations*. 2023. URL : https://openreview.net/forum?id=WE_vluYUL-X.

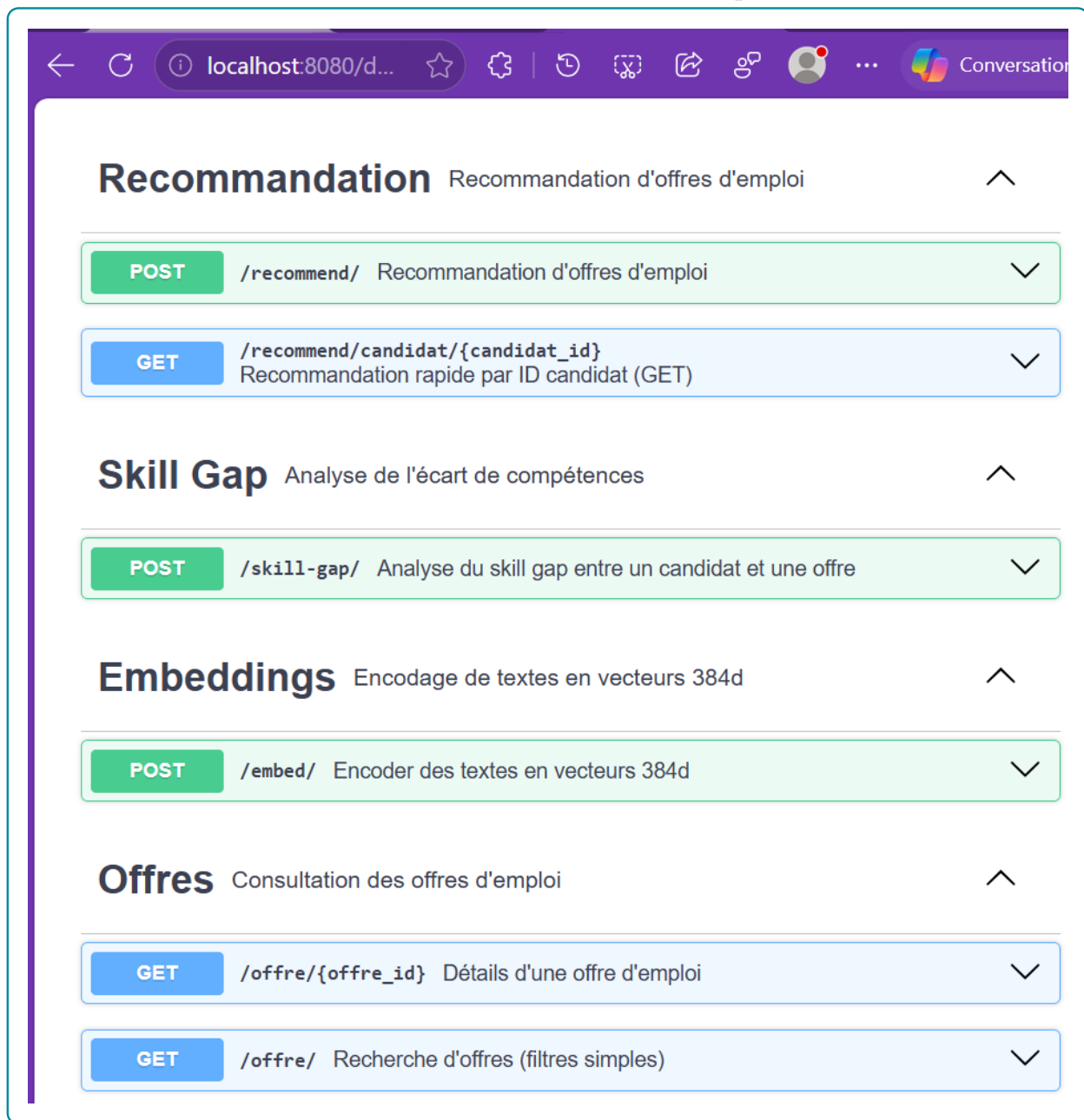
ANNEXES

Figure A.1 – Interface Streamlit du chatbot Agentic GraphRAG



Source : Nos travaux, capture locale de l'interface

Figure A.2 – Documentation interactive des endpoints FastAPI



Source : Nos travaux, capture locale de la documentation FastAPI

Table A.1 – Labels de nœuds et relations structurantes du graphe

Bloc	Labels de nœuds	Relations principales et interprétation
Données d'emploi	OffreEmploi, Candidat, Employeur, Secteur, Localisation	PUBLIEE_PAR relie une offre à son employeur ; DANS_SECTEUR la rattache à un secteur ; LOCALISEE_A indique la ville ou zone d'exercice ; A_NIVEAU relie le candidat à son niveau de formation.
Compétences et métiers	Compétence, Métier, GroupeCompétences, GroupeISCO	NECESSITE relie un métier aux compétences attendues ; PARTIE_DE rattache une compétence à son groupe ; CLASSIFIE_DANS relie un métier à un groupe ISCO ; PLUS_LARGE_QUE représente les relations hiérarchiques entre compétences.
Référentiel MEPC	GrandGroupeMEPC, SousGroupeMEPC, GroupeBaseMEPC	CONTIENT représente la hiérarchie interne de la nomenclature ; ALIGNE_AVEC rapproche les groupes MEPC des groupes ISCO ; CORRESPOND_MEPC rattache un métier ESCO à un groupe de base MEPC.
Référentiel NCF	NiveauFormationNCF, GrandDomaineNCF, DomaineSpécialiséNCF, DomaineDétailléNCF	CONTIENT représente la hiérarchie formation ; REQUIERT_NIVEAU_NCF relie une offre au niveau demandé ; A_FORMATION rattache un candidat à son domaine de formation ; PREPARE_POUR relie un domaine de formation aux métiers visés.
Recommandation	OffreEmploi, Candidat, Compétence, Métier, DomaineDétailléNCF	Les chemins candidat-compétence-offre permettent d'identifier les compétences acquises et manquantes ; les chemins offre-niveau-formation servent à contrôler la compatibilité de qualification ; les chemins formation-métier soutiennent la proposition de montée en compétences.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.2 – Volume d’entités indexées dans la base vectorielle pgvector

Type d’entité	Nombre	Source principale
OFFRE_EMPLOI	13 957	Scraping KmerAI
COMPETENCE	13 939	Référentiel ESCO v1.1
METIER	3 039	Référentiel ESCO v1.1
CANDIDAT	41 298	Base locale de profils demandeurs
GROUPE_BASE_MEPC	209	INS Cameroun - MEPC 2013
DOMAINE_DETAILLE_NCF	201	INS Cameroun - NCF 2017
Total entités structurées	72 643	Hors 142 chunks documentaires indexés séparément.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.3 – Pipeline général implémenté : étapes, entrées et sorties

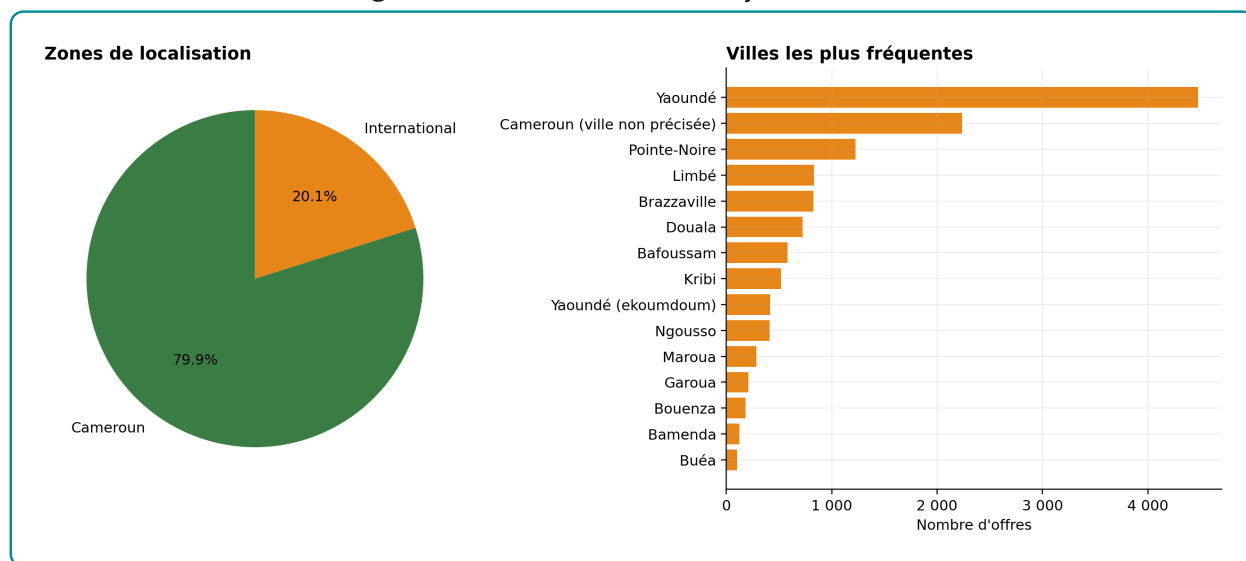
Ordre	Étape	Entrées principales	Sorties concrètes
1	ETL	Données brutes d’offres, candidats et référentiels	Fichiers normalisés, champs <code>text_to_embed</code> , paires <code>pairs_train/val/test.jsonl</code> .
2	Fine-tuning	Paires offre–description et modèle <code>all-MiniLM-L6-v2</code>	Modèle sauvegardé dans <code>models/st_finetuned/final</code> et métriques d’entraînement.
3	Indexation vectorielle	Entités nettoyées et encodeur fine-tuné	Table <code>embeddings</code> , table <code>doc_chunks</code> , index HNSW et index textuels PostgreSQL.
4	Graphe de connaissances	Offres, candidats, compétences, métiers, niveaux et référentiels	Noeuds Neo4j, relations métier-compétence-offre-candidat, contraintes et chemins explicatifs.
5	GraphRAG hybride	Résultats pgvector, chemins Neo4j et règles métier	Contexte candidat-offre, score hybride, analyse de <i>skill gap</i> , critic et Text2Cypher.
6	Orchestration LangGraph	Question utilisateur et registre d’outils	État partagé, routage, exécution des outils, génération finale et traces du workflow.
7	Exposition applicative	Workflow agentique et services locaux	API FastAPI, interface en ligne de commande de test et interface Streamlit.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.4 – Correspondance entre objectifs spécifiques et choix méthodologiques

Objectif spécifique	Choix méthodologique retenu
Normaliser les offres, candidats et référentiels	Construction d'une couche de données standardisée : identifiants stables, champs nettoyés, niveaux de formation, secteurs, compétences et textes d'embedding.
Adapter la recherche sémantique au domaine emploi-compétences	Fine-tuning contrastif d'un modèle SentenceTransformer, puis indexation des embeddings dans PostgreSQL avec pgvector pour le retrieval top-K [36, 33].
Représenter les relations entre métiers, compétences et profils	Construction d'un graphe de connaissances Neo4j reliant candidats, offres, compétences, métiers, secteurs, localisations et référentiels [19, 16].
Produire une recommandation explicable	Score hybride combinant similarité vectorielle, couverture de compétences, compatibilité de niveau et alignement sectoriel, complété par une analyse de <i>skill gap</i> .
Orchestrer la réponse utilisateur	Workflow LangGraph routant la requête vers les outils adaptés : pgvector, Neo4j, recherche documentaire, Text2Cypher, critic et génération finale.
Evaluer la qualité du système	Protocole séparant l'évaluation du retrieval, du graphe, du classement et de la réponse générée.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Figure A.3 – Localisation nettoyée des offres

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.5 – Volumes réellement exploités par les modules de recommandation

Objet exploité	Volume	Utilité pour le système
Offres normalisées	13 957	Base principale à recommander, indexée dans pgvector et reliée au graphe.
Candidats normalisés	41 298	Vivier à partir duquel sont calculés les rapprochements offre–profil.
Compétences indexées	13 939	Signal de matching, de filtrage et de justification du <i>skill gap</i> .
Secteurs nettoyés	1 039	Signal de contexte pour limiter les rapprochements absurdes entre domaines trop éloignés.
Métiers indexés	3 039	Niveau d’agrégation utile pour stabiliser les recommandations au-delà des intitulés d’offres.
Groupes métiers MEPC	209	Référentiel externe pour structurer les proximités métier.
Domaines détaillés NCF	201	Référentiel de formation utilisé pour contextualiser les niveaux et domaines.
Localisations représentées dans Neo4j	312	Signal géographique pour filtrage souple et explication des résultats.

Source : Nos travaux, diagnostics PostgreSQL

Table A.6 – Comparaison directe entre le baseline et le modèle fine-tuné

Métrique test	Baseline	Fine-tuné	Gain
NDCG@10	0,1681	0,6232	+0,4552
MRR@10	0,1410	0,5874	+0,4463
Recall@1	0,0986	0,5268	+0,4281
Recall@5	0,1983	0,6632	+0,4648
Recall@10	0,2560	0,7398	+0,4837
Precision@1	0,0986	0,5268	+0,4281

Source : Nos travaux, traitements Python et SentenceTransformer

Table A.7 – Synthèse des traitements de nettoyage utiles au moteur de recommandation

Traitement	Décision implémentée	Effet attendu
Casse et formats	Harmonisation des libellés pour réduire les variantes d'écriture.	Moins de fragmentation dans les compétences, secteurs, villes et sources.
Doublons	Validation uniquement si toute la ligne est identique sur toutes les variables.	Suppression prudente des répétitions exactes, sans effacer des offres proches mais distinctes.
Localisation	Séparation Cameroun, international et non précisée ; conservation des localités fines disponibles.	Filtrage géographique possible sans perte des offres incomplètes.
Champs vides	Modalités explicites lorsque le champ reste exploitable comme absence d'information.	Le système peut pénaliser ou contextualiser le manque d'information au lieu d'ignorer l'offre.
Descriptions faibles	Enrichissement de la représentation par métadonnées et compétences.	Robustesse accrue pour les sources où le détail de l'offre est absent ou court.

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.8 – Diagnostic statistique du chunking documentaire

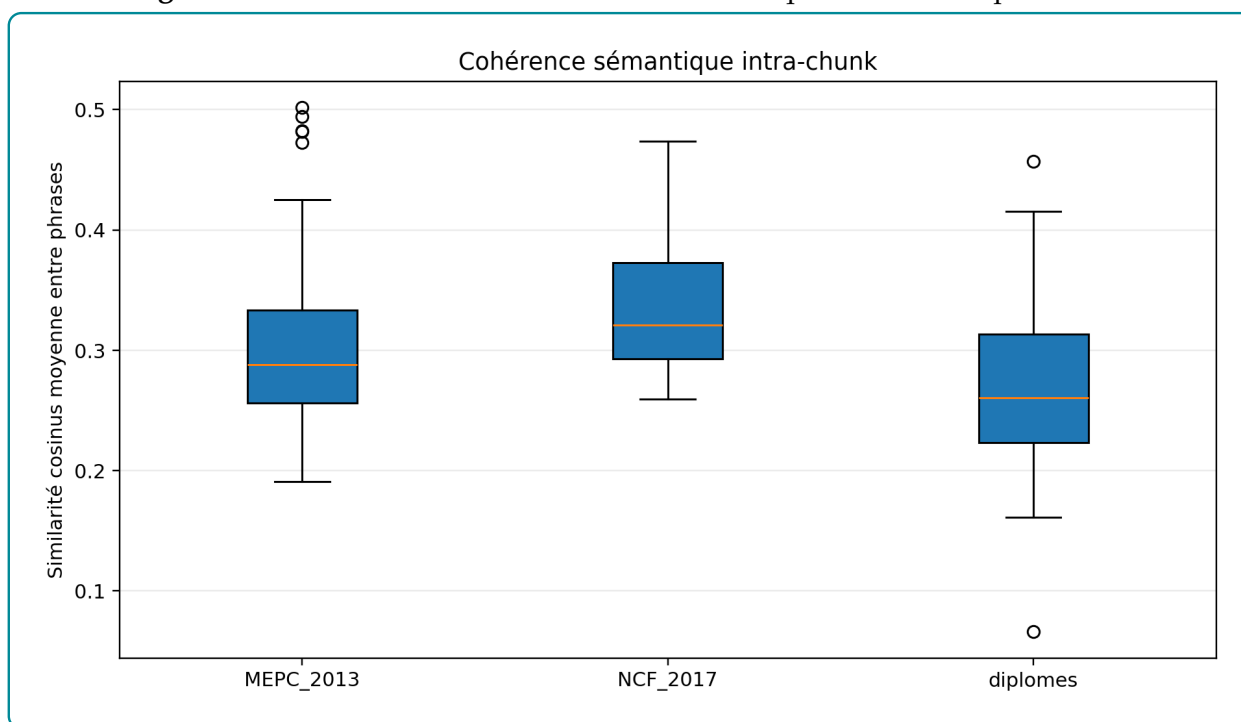
Indicateur	Valeur observée
Documents sources indexés	3
Chunks documentaires indexés	142
Stratégie réellement utilisée	100 % structurelle
Taille moyenne	187,1 tokens
Taille médiane	116,0 tokens
Écart-type	169,6 tokens
Minimum / maximum	6 / 492 tokens
Coefficient de variation	0,91
Chunks avec embedding	142 / 142
Chunks synchronisés avec Neo4j	142 / 142

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Table A.9 – Couverture documentaire et duplication des chunks

Source	Couverture approximative	Chunks
NCF 2017	25,9 %	38
MEPC 2013	23,6 %	61
Référentiel des diplômes	79,3 %	43
Taux de duplication cosinus > 0,95	0,0 %	142
Similarité maximale hors diagonale	0,921	–

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

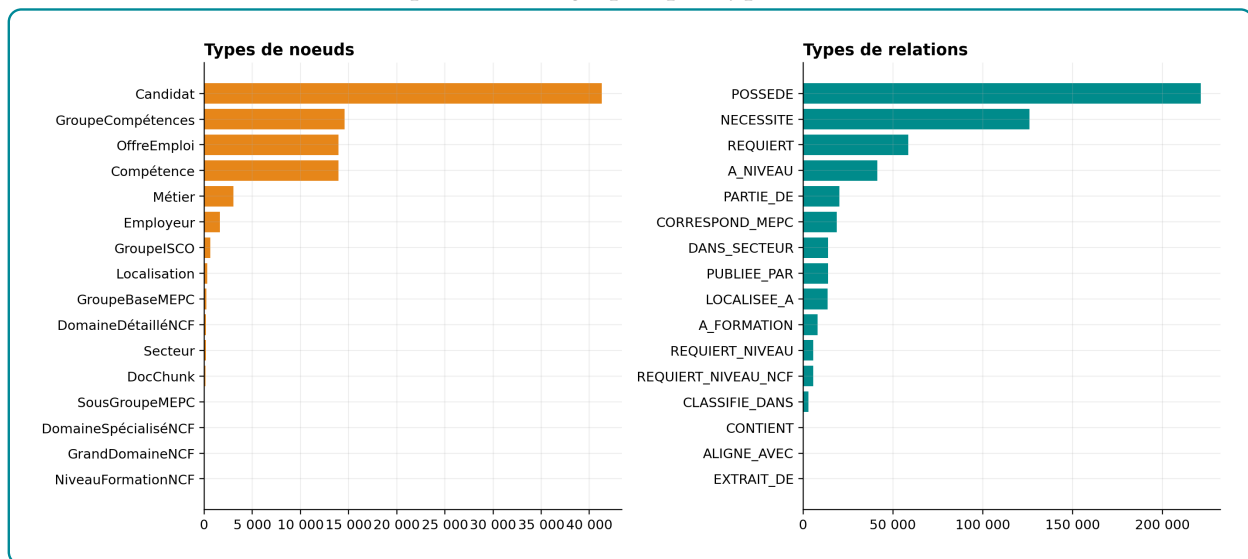
Figure A.4 – Distribution de la cohérence sémantique intra-chunk par source

Source : Nos travaux, mesure Python de la similarité cosinus moyenne entre les phrases de chaque chunk.

Table A.10 – Cohérence sémantique intra-chunk

Source	Moyenne	Médiane	Intervalle observé
MEPC 2013	0,307	0,288	0,191–0,502
NCF 2017	0,336	0,321	0,259–0,474
Référentiel des diplômes	0,268	0,260	0,066–0,457

Source : Nos travaux, traitements Python de nettoyage et de diagnostic, à partir des données collectées du projet.

Figure A.5 – Composition du graphe par types de noeuds et relations

Source : Nos travaux, diagnostics Neo4j.

Table A.11 – Hétérogénéité des degrés par famille de noeuds

Label	Moyenne	Médiane	p90	Max
NiveauFormationNCF	5 847,88	5 924,5	10 056,6	10 261
GroupeBaseMEPC	95,11	74	200	275
Secteur	72,69	5,5	151,6	1 548
Métier	47,61	43	76	185
Localisation	43,96	1	8	4 467
Compétence	30,04	4	27	6 787
OffreEmploi	6,57	8	9	9
Candidat	6,36	7	7	7

Source : Nos travaux, diagnostics Neo4j, à partir du graphe de connaissances emploi-compétences construit dans le projet.

Table A.12 – Résumé du déplacement de rang après reranking Neo4j

Indicateurs	Valeur
Couples candidat-offre évalués	690
Candidats distincts	69
Rang moyen avant reranking pgvector	9,51
Rang moyen après reranking Neo4j/Optuna	5,50
Déplacement moyen de rang	+4,01
Déplacement médian	+1,00
Offres remontées	59,86 %
Offres rétrogradées	13,77 %
Offres inchangées	26,38 %

TABLE DES MATIÈRES

Dédicace	i
Remerciements	ii
Liste des sigles et abréviations	v
Liste des tableaux	vii
Liste des figures	viii
Avant-propos	ix
Résumé	x
Abstract	xi
Introduction générale	1
 I CADRE THÉORIQUE ET EXPÉRIMENTAL	 5
1 FONDEMENTS THÉORIQUES ET REVUE DE LA LITTÉRATURE SUR L'IA GÉNÉRATIVE ET LES SYSTÈMES DE RECOMMANDATION	6
1.1 Repères conceptuels de l'étude	6
1.1.1 Vocabulaire opérationnel : du matching à l'Agentic RAG	6
1.2 Fondements économiques et problématique d'appariement	8
1.2.1 Marché du travail, recherche d'emploi et frictions	8
1.2.2 Capital humain, tâches et inadéquation des compétences	9
1.3 Systèmes de recommandation et person-job fit	9
1.3.1 Approches par contenu, collaboratives et hybrides	9
1.3.2 Spécificité du person-job fit	11
1.4 LLMs, embeddings et graphes de connaissances	12
1.4.1 Grands modèles de langage et génération contrôlée	12
1.4.2 Embeddings, recherche sémantique et bases vectorielles	13
1.4.3 Graphes de connaissances et recommandation relationnelle	14
1.5 RAG, GraphRAG et orchestration agentique	15
1.5.1 De la RAG au GraphRAG	15

1.5.2	Agentic RAG, critic et traçabilité	16
1.5.3	Évaluer la performance et la factualité	17
1.6	Positionnement de l'étude	19
2	SOURCES DE DONNÉES ET APPROCHE MÉTHODOLOGIQUE	21
2.1	Cadre méthodologique et sources de données	21
2.1.1	Démarche d'ingénierie appliquée retenue	21
2.1.2	Sources de données et leur rôle dans le système	23
2.1.3	Préparation et normalisation des données	24
2.1.4	Alignement des référentiels métiers et formations	24
2.2	Représentation sémantique et graphe de connaissances	24
2.2.1	Choix du modèle d'embeddings et fine-tuning	24
2.2.2	Indexation vectorielle dans PostgreSQL et pgvector	25
2.2.3	Construction du graphe de connaissances sous Neo4j	25
2.2.4	Score hybride et logique de skill gap	25
2.2.5	Construction du proxy de pertinence pour l'évaluation	25
2.3	Orchestration agentique et génération contrôlée	27
2.3.1	Architecture Agentic RAG et orchestration LangGraph	27
2.3.2	Text2Cypher et rôle du LLM	27
2.3.3	Modes d'accès et traçabilité des décisions	27
2.3.4	Protocole d'évaluation prévu du système	28
II	PRÉSENTATION DES RÉSULTATS ET DISCUSSION	29
3	CONSTRUCTION DU SYSTÈME HYBRIDE EMPLOI-COMPÉTENCES	30
3.1	Données exploitables et contraintes de qualité du système	30
3.1.1	Volumes disponibles après nettoyage	30
3.1.2	Qualité des variables utiles au matching emploi-compétences	31
3.1.2.1	Localisation exploitable pour filtrer sans exclure	31
3.1.2.2	Compétences et descriptions : signal utile mais hétérogène	32
3.1.3	Nettoyage réalisé et limites conservées	32
3.2	Adaptation du modèle d'embeddings au domaine emploi-compétences	33
3.2.1	Encodeur retenu pour les paires contrastives	33
3.2.2	Fine-tuning : contraintes de calcul et preuve de gain	34
3.2.2.1	Paramétrage contraint et dynamique d'apprentissage	34
3.2.2.2	Preuve de gain sur le ranking sémantique	35
3.2.3	Structure de l'espace vectoriel après fine-tuning	36
3.3	Mémoire vectorielle pgvector pour le retrieval	37

3.3.1	Schéma physique, tables et volumes indexés	37
3.3.2	Référentiels découpés en chunks interrogeables	39
3.4	Neo4j : mémoire relationnelle du matching	41
3.4.1	Schéma, noeuds et relations mobilisés par le matching	42
3.4.2	Structure du graphe et dépendance aux hubs	42
3.5	Score hybride et reranking des recommandations	45
3.5.1	Formalisation du score hybride	45
3.5.2	Effet du reranking hybride	46
3.6	Orchestration agentique du système par LangGraph	46
3.6.1	Perception de la requête et planification des outils	47
3.6.2	Action, observation et construction du contexte	48
3.6.3	Génération contrôlée et boucle de révision	48
4	ÉVALUATION DE LA PERFORMANCE INTERNE DU SYSTÈME	50
4.1	Protocole expérimental et métriques retenues	50
4.2	Retrieval sémantique et apport mesuré du graphe	51
4.2.1	Modèle d’embeddings spécialisé	51
4.2.2	Ablation pgvector seul contre pgvector enrichi par Neo4j	52
4.3	Validation fonctionnelle du GraphRAG	53
4.3.1	Requêtes relationnelles métier, compétence et offre	54
4.3.2	Skill gap, niveau NCF et comparaison GraphRAG	55
4.4	Reranking hybride et calibration du score	56
4.4.1	Déplacements de rang après reranking par le graphe	56
4.4.2	Pondérations optimales avec Optuna (optimisation bayésienne)	57
4.5	Contrôle de génération par critic	60
4.5.1	Calibration contrastive du seuil de fidélité	60
4.5.2	Interprétation des décisions accept/revise	61
5	SYNTHÈSE ET DISCUSSION DES RÉSULTATS	64
5.1	Résultats majeurs de l’étude	64
5.2	Confrontation avec la littérature	65
5.3	Discussion des hypothèses de recherche	65
5.4	Portée, limites et conditions d’usage	66
	Conclusion générale et perspectives	68
	Références bibliographiques	II
A	ANNEXES	VII